

Spring Security Course

Course Agenda - <https://github.com/eazybytes/spring-security>

Basic Concepts

1. Welcome to the world of Spring Security
2. Securing a web app using Spring Security
3. Important interfaces, Classes, Annotations of Spring Security
4. Configuring Authentication & Authorization for web app

Advanced Concepts

1. Implementing role-based access using ROLES, AUTHORITIES
2. Different Strategies that Spring Security provides when coming to passwords
3. Method level security using Spring Security
4. How to handle most common attacks like CORS, CSRF with Spring Security
5. Deep dive on JWT & its role in Authentication & Authorization
6. Deep dive on OAUTH2, OpenID & Securing a web app using the same
7. Exploring Authorization servers like Keycloak, Spring Authorization server
8. Important topics of Security like Hashing, Tokens & many more.

MILESTONE 1 – Firstly, will create a simple spring security project & enable basic security in it.

MILESTONE 2 – Authentication using login screen will be implemented along with the URLs security, CSRF & CORS protection.

MILESTONE 3 – Authorization will be implemented using ROLES, AUTHORITIES

MILESTONE 4 – Method level security will be implemented along with custom filters in the authentication & authorization process.

MILESTONE 5 – Finally, the application will be enhanced with OAUTH2/OpenID using JWT which is the most common security approach in the industry.

application.properties

- We can add properties using below expression to have a default value in absence of the environment variable.

```
application.properties x
1 spring.application.name=${SPRING_APP_NAME:springSection1}
2
3 logging.pattern.console=${LOGPATTERN_CONSOLE:%green(%d{HH:mm:ss.SSS}) %blue(%-5level) %red([%thread]) %yellow(%logger{15})- %msg%n}
```

Package declared outside of main package

- Main package is the package where the Spring boot main class is present.
- For some reason, if we've created these controller package outside of this main package where the spring boot main class is present, then in such scenarios, by default our spring boot framework is not going to scan the controller classes.
- So, in all such scenarios, we need to make sure we're mentioning @ComponentScan along with the package detail which we want to scan.

```
@SpringBootApplication  Shivam-Omen *
@ComponentScan("com.geek.spring.security.controller")
public class EasyBankBackendApplication {

    public static void main(String[] args) { SpringApplication
}
}
```

Spring Security

- Once we add the spring-boot-starter-security dependency to pom.xml, by default, Spring Security will assume & protect each & every API & MVC paths available inside our project.
- By default, username is user & default password will be generated on the console.
- Once we logged in, even if we refresh the browser multiple times still Spring security will not ask for relog in. As behind the scene, Spring security does a lot of work in remembering the session within the browser.
- In case we use different browser with no session, Spring security will expect you to login again.
- We can refer SecurityProperties to view the default user & default password configuration

```
SecurityProperties.java x
40 public class SecurityProperties {
78     public static class Filter {
106     }
107
108     public static class User {
109
110         Default user name.
113         private String name = "user";
114
115         Password for the default user name.
118         private String password = UUID.randomUUID().toString();
119     }
}
```

- Default Password is not really applicable for our real time scenarios or even for POCs. We can fix this by defining the properties inside the application.properties file

- For the specific application.properties keys, we can
 - Either refer the specific properties class in the Spring boot application
 - Or refer the spring boot reference documentation: <https://docs.spring.io/spring-boot/appendix/application-properties/index.html>. We can utilize right side option to choose security properties.

Spring Boot / Appendix / Common Application Properties

	ken introspection endpoint.
<code>spring.security.oauth2.resourceserver.opaquetoken.client-secret</code>	Client secret used to authenticate with the token introspection endpoint.
<code>spring.security.oauth2.resourceserver.opaquetoken.introspection-uri</code>	OAuth 2.0 endpoint through which token introspection is accomplished.
<code>spring.security.saml2.relyingparty.registration.*</code>	SAML2 relying party registrations.
<code>spring.security.user.name</code>	Default user name. <code>user</code>
<code>spring.security.user.password</code>	Password for the default user name.
<code>spring.security.user.roles</code>	Granted roles for the default user name.

Common Application Properties

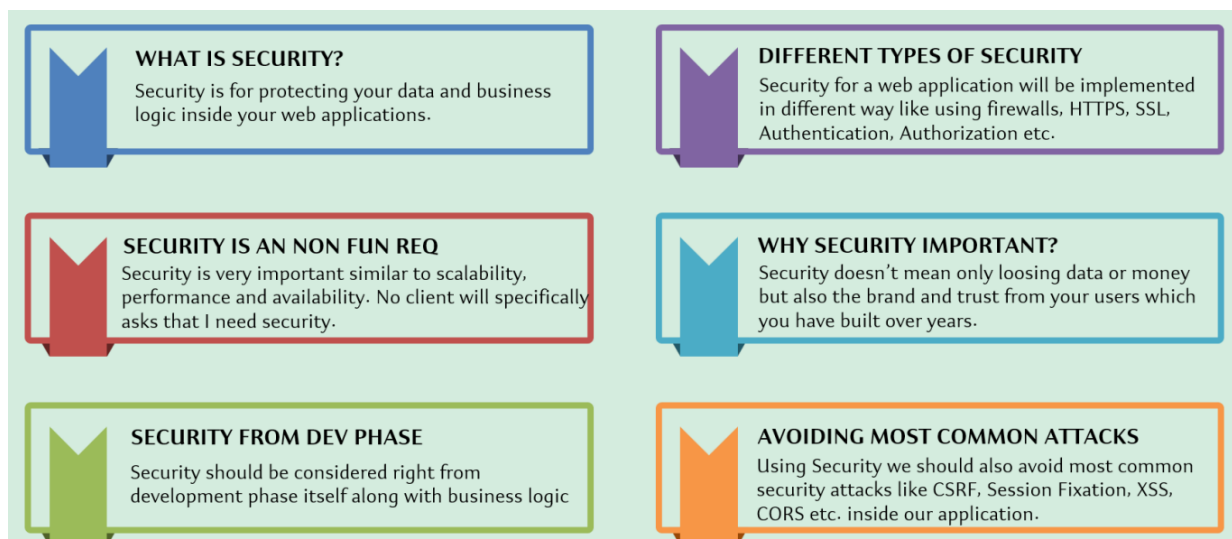
- Actuator Properties
- Cache Properties
- Core Properties
- Data Migration Properties
- Data Properties
- Devtools Properties
- Docker Compose Properties
- Integration Properties
- JSON Properties
- Mail Properties
- RSocket Properties
- Security Properties**
- Server Properties
- Templating Properties
- Testcontainers Properties
- Testing Properties
- Transaction Properties
- Web Properties

 Edit this Page

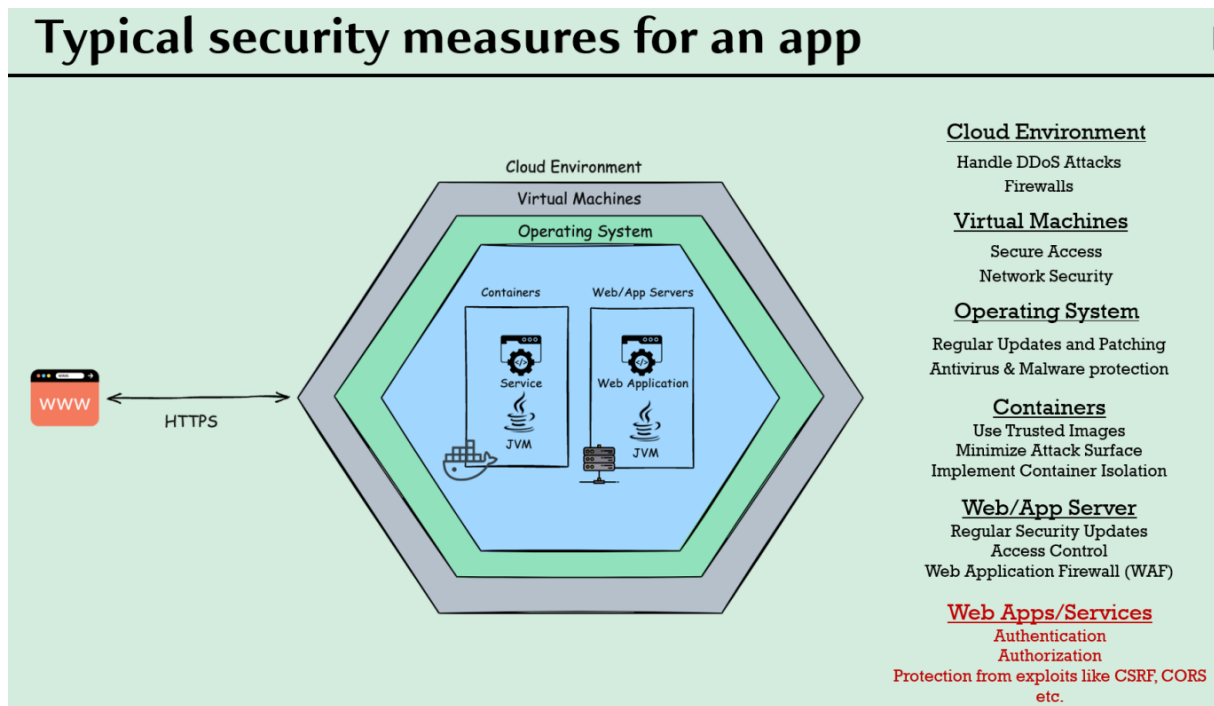
 GitHub Project

What is Security & Why we need it?

- For e.g., Banks are protected by physical security say 24x7, strong lockers, sensors & CCTV camera 24x7 & they are protected because Banks hold “Valuable Assets” that includes cash, gold, some important papers. Because of all these reasons, Banks try to protect themselves with various security mechanism.
- Similarly, our web applications can also hold valuable data like Credit Card details, transactions, health records or business logics etc. & they should be protected via proper security mechanism like Authentication, Authorization.
- Security is a non-functional requirement. There can be 2 types of requirements an organization can have:
 1. **Functional Requirements (FR)** – Under FR, we have various aspects like business logic, how data to be stored, how UI should be displayed etc.
 2. **Non-Functional Requirements (NFR)** – Under NFR, we’ll have topics like Security, Performance, Scalability, Availability etc.
- No client will ask specifically/explicitly to build a proper secure, performing web application. By default, every developer or every team should assume all these NFRs.
- We should implement security right from the development phase. In fact, we should move the security to the left side of the SDLC.



Imp – Spring Security Framework is going to help the developers in securing a web application from all the most common attack.



- Inside an organization infrastructure, the very 1st layer will be Cloud Environment -> virtual M/c -> Operation System -> Docker containers or Web/App Servers.
- At each layer, there is a security mechanism, and as a developer, we should be very clear about our responsibilities & Operation/DevOps team's responsibilities.
- Right from the Client Application, we're trying to implement the security with the help of HTTPS security protocol.
- For all the layer like Cloud, VM, OS, Containers, Web/App Server -> Operation team uses various mechanism to secure them.
- But coming to the last layer i.e., about securing the web application or the APIs, Developers need to implement various mechanism like Authentication, Authorization & Protection from exploits like CSRF, CORS etc.
- This is where Spring Security framework plays a major role to implement security with ease as it's super tough to implement security from scratch inside any web application.

Why Spring Security ?



Application security is not fun and challenging to implement with our custom code/framework.

Spring Security built by a team at Spring who are good at security by considering all the security scenarios. Using Spring Security, we can secure web apps with minimum configurations. So there is no need to re-invent the wheel here.



Spring Security handles the common security vulnerabilities like CSRF, CORs etc. For any security vulnerabilities identified, the framework will be patched immediately as it is being used by many organizations.

Using Spring Security we can secure our pages/API paths, enforce roles, method level security etc. with minimum configurations easily.



Spring Security supports various standards of security to implement authentication, like using username/password authentication, JWT tokens, OAuth2, OpenID etc.

Servlets & Filters

- Before we try to understand the Spring Security internal flow, first we need to understand what're servlets & filters inside Java ecosystem.
- Inside Spring Security framework, lot of security related functionality is implemented with the help of filters & servlets.

Servlet

- We can easily invoke the REST API or MVC path with the help of the Browser (Client App), the request (message) will be sent via HTTP or HTTPS protocol but the request will not make any sense to the backend server or backend logic.
- Inside Java Programming language, anything we want to process, it has to be converted into java object so, this is where Servlet container comes into the picture.

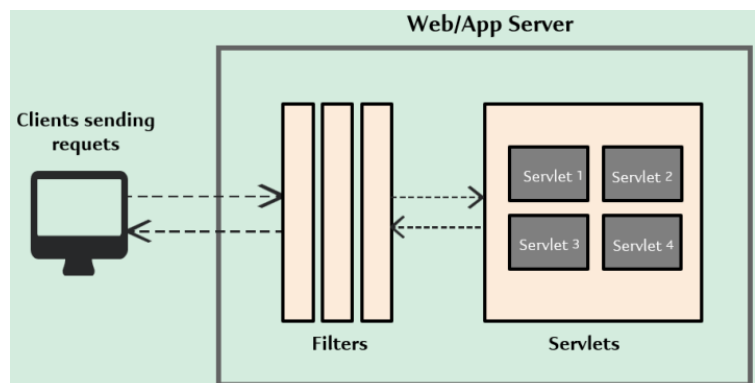
Servlet Container

- One of the mostly used servlet container is Apache Tomcat.
- In Java web apps, Servlet Container (Web Servers) takes care of translating the HTTP messages for Java code to understand.
- Servlet container converts the HTTP messages into ServletRequest & hand over to Servlet method as a parameter.
- Imp. Point – Once the Servlet request object is prepared, the same will be forwarded to next servlet that can be leveraged by the frameworks like Spring Boot or Spring Security.
- From these servlets, only the actual business logic invocation is going to happen i.e., whatever logic we write inside the controller or service classes will be invoked via these servlets only.
- Similarly, Servlets returns ServletResponse as the output to the Servlet Container & the container will convert the ServletResponse object to the HTTP or HTTPS protocol before forwarding the response to the client application.
- So, everything we write inside java web apps are driven by Servlets.

Note – Building a business logic or web application with servlets is very complex, Frameworks like Spring Boot or Spring Security does a lot of heavy lifting for us so that dev can focus on writing business logic.

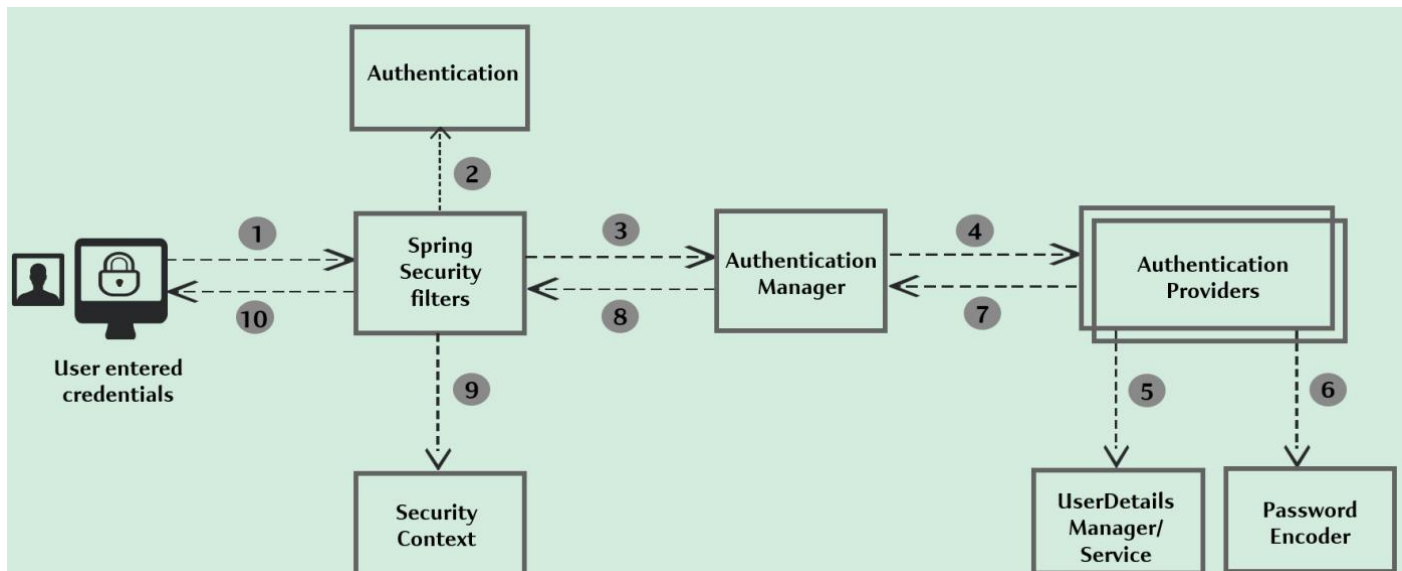
Role of Filters

- We can see in the picture that before our request reaches to the actual servlet, there are filters in between.
- Filters inside java web application can be used to intercept each request/response & do some pre work before our business logic.
- Using these filters only, frameworks like Spring Security implement lot of security related logic.
- The purpose of Spring Security framework is whenever we're trying to access a REST API or a MVC path, it has to intercept the request & identify if the user is authenticated or not. If not, then it has to redirect the user to the login page.
- Since the Spring Security has to intercept each & every request to enforce the security inside a web apps, all the security logic is implemented with the help of filters, only.



Note – Always the filters are going to be executed first, followed by the servlets or the actual business logic execution. **Once all the Spring Security related filters are executed, the request will be forwarded to the DispatcherServlet.**

Spring Security Internal Flow/Architecture



1) Through client applications like Browser, Postman etc.,

- The end users send the request to access REST API or a MVC path & if the API is protected, the end user will have to provide their credential as part of the Request Header or Request Body based on the configuration done behind the scenes.
- If the user is trying to access a protected API without mentioning credentials, the Spring Security framework will intercept the request by using Spring Security filters & will redirect the user to the login page.
- The job of identifying if the user is authenticated or not is handled by Spring Security filters (intercepts each & every request)
- For e.g., A filter can be for authentication, another filter can be CSRF attack etc.
- Based on our security filters configuration – 5 or 10 or 20 filters, the request will go through each filter & at any filter layer, if there is an error then end user will get a 401 or 403 error based upon the exception type.

2) In case the end user provides the proper credential inside the request while accessing a secured API, the Spring Security filters will populate the Authentication object.

- By default, the credentials received from the end user as part of HTTP Request object can't be forwarded directly to the framework code or business logic.
- Spring Security filters will first covert the credentials from the HTTP Servlet Request to Authentication object.
- Inside Authentication object, there will be username, password & boolean value to identify whether the user is authenticated or not. Initially when the object is created 1st time, isAuthenticated value will be set to false & username, password will be set to the default or the one we provided in application.properties
- The same Authentication object will be used by all other components inside the Spring Security framework.
- The Authentication object represents a user credentials & information to identify if the user is authenticated or not.
- Once the Authentication object is created & populated, the filters will forward the request to the Authentication Manager.

3) Authentication Manager

- It is responsible for completing the authentication by taking help from other components & conveying the results back to the filters whether the authentication is successful or not.
- **Imp Point** – Authentication Manager will not be performing actual authentication. So, in order to perform actual authentication, it will forward the request to Authentication Provider available inside the Spring Security framework. (We can use already existing or can define our own Authentication Providers)
- So, a project can have multiple Authentication Providers to identify whether the authentication is successful or failed.

4 – 5) Once the Authentication Providers receive the Authentication object to perform the actual authentication, these Authentication Providers are going to take help from 2 other component i.e.,

- UserDetailsManager or UserDetailsService and Password Encoder component.
- The Authentication Provider uses UserDetailsManager component or UserDetailsService implementation classes to load the user details based upon the username result.
- Once the user details will be sent back to the Authentication provider, it won't compare the password provided by the user with the stored password (from database or inside the memory) by itself rather it takes help from Password Encoder Component.

6) Password Encoder component

- Is responsible for the password comparison.
- Using these Password Encoder, we can store the password using plain text or using the hashing algorithm.
- Since the Password encoder is a specialist, the Authentication Provider will ask the PE to compare the user provided password with the loaded password from storage system with help of UserDetailsManager.
- Once the PE says the passwords matched accordingly the Authentication Provider conveys back to the Authentication Manager saying "The Authentication is successful"

7) Authentication Provider will convey whether the Authentication was successful or not via setting the boolean value isAuthenticated present in the Authentication object.

- Based on the value of isAuthenticated attribute, the Authentication manager will know whether the authentication is successful or not.

8) The same Authentication object will be sent back to the Spring Security filters.

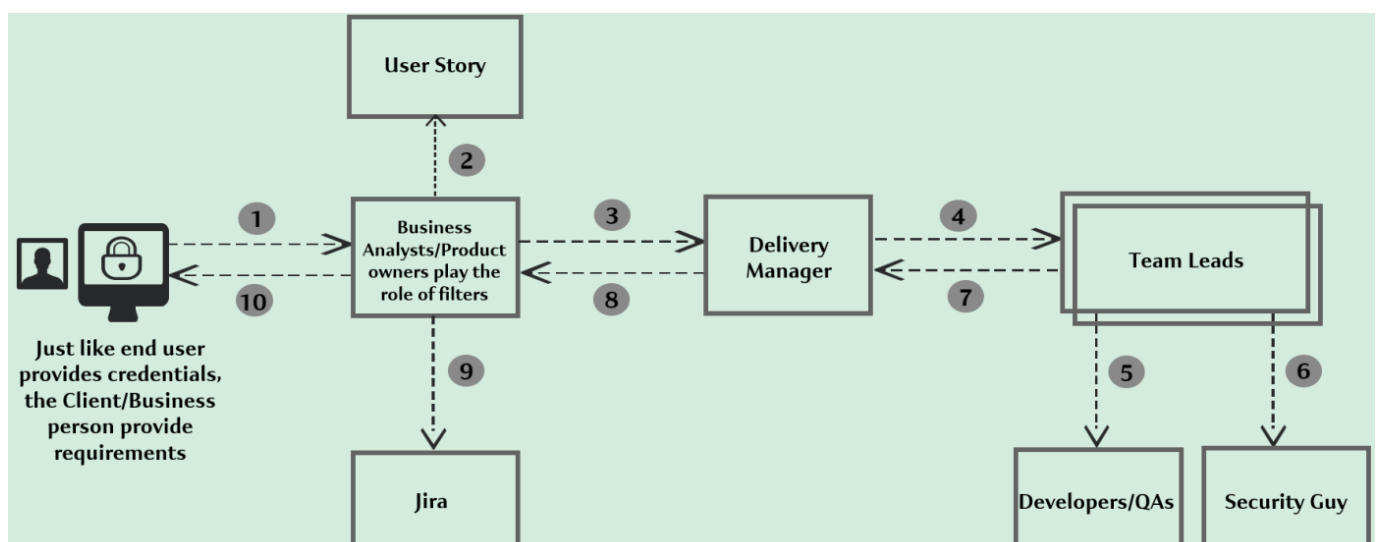
9) Regardless of whether the Authentication is successful or not, the Authentication object will be stored in the Security Context against a sessionId (created for a given browser)

- So, if the user is trying to access the same protected page from the same browser then, based upon the given session ID, the Spring Security filters will load the authentication object details from the security context & accordingly they will show either successful or error message.

Imp Conclusion – All the actual authentication execution is only going to happen for the very 1st request as there is no entry for a given session ID in Security Context. Once the 1st request is processed & there is an entry inside the security context for the same session ID, from next time onwards, the security filters will not be invoking the Authentication manager, instead they will leverage the details present inside the security context to send the successful or error response.

10) A response will be sent to the client app based on the authentication result. In case of successful authentication, a response from the REST API will be sent else 401 or 403 error.

Analogy of Spring Security Internal flow with SDLC



★ Spring Security Filters

A series of Spring Security filters intercept each request & work together to identify if Authentication is required or not. If authentication is required, accordingly navigate the user to login page or use the existing details stored during initial authentication.

★ Authentication

Filters like UsernamePasswordAuthenticationFilter will extract username/password from HTTP request & prepare Authentication type object. Because Authentication is the core standard of storing authenticated user details inside Spring Security framework.

★ AuthenticationManager

Once received request from filter, it delegates the validating of the user details to the authentication providers available. Since there can be multiple providers inside an app, it is the responsibility of the AuthenticationManager to manage all the authentication providers available. In simple words, the authentication manager takes the responsibility for authentication.

★ AuthenticationProvider

AuthenticationProviders has all the core logic of validating user details for authentication.

★ UserDetailsManager/UserDetailsService

UserDetailsManager/UserDetailsService helps in retrieving, creating, updating, deleting the User Details from the DB/storage systems.

★ PasswordEncoder

Service interface that helps in encoding & hashing passwords. Otherwise we may have to live with plain text passwords ☹

★ SecurityContext

Once the request has been authenticated, the Authentication will usually be stored in a thread-local SecurityContext managed by the SecurityContextHolder. This helps during the upcoming requests from the same user.

Reference Classes in Spring Security framework to understand the Internal Flow

Suppose End user is trying to access a protected API (e.g., welcome API without providing any credentials), there are few below filters that are going to play a role.

1) org.springframework.security.web.access.intercept.**AuthorizationFilter**

- Responsible to identify whether a user is trying to access a protected API or path without credentials or without authenticated session.
- AuthorizationFilter class contains business logic to identify whether to deny the request or not.

```
public class AuthorizationFilter extends GenericFilterBean {  
    @Override  
    public void doFilter(ServletRequest servletRequest, ServletResponse servletResponse, FilterChain chain)  
        throws ServletException, IOException {  
  
        HttpServletRequest request = (HttpServletRequest) servletRequest;  
        HttpServletResponse response = (HttpServletResponse) servletResponse;  
  
        if (this.observeOncePerRequest && isApplied(request)) {...}  
  
        if (skipDispatch(request)) {...}  
  
        String alreadyFilteredAttributeName = getAlreadyFilteredAttributeName();  
        request.setAttribute(alreadyFilteredAttributeName, Boolean.TRUE);  
        try {  
            AuthorizationResult result = this.authorizationManager.authorize(this::getAuthentication, request);  
            this.eventPublisher.publishAuthorizationEvent(this::getAuthentication, request, result);  
            if (result != null && !result.isGranted()) {  
                throw new AuthorizationDeniedException("Access Denied", result);  
            }  
            chain.doFilter(request, response);  
        }  
        finally {  
            request.removeAttribute(alreadyFilteredAttributeName);  
        }  
    }  
}
```


2) org.springframework.security.web.authentication.ui.DefaultLoginPageGeneratingFilter

- The purpose of this filter is to redirect the user to the login page by showing a page with username & password fields.
- DefaultLoginPageGeneratingFilter class contains business logic to build the login page.

```
public class DefaultLoginPageGeneratingFilter extends GenericFilterBean {  
  
    private void doFilter(HttpServletRequest request, HttpServletResponse response, FilterChain chain)  
        throws IOException, ServletException {  
        boolean loginError = isErrorPage(request);  
        boolean logoutSuccess = isLogoutSuccess(request);  
        if (isLoginUrlRequest(request) || loginError || logoutSuccess) {  
            String loginPageHtml = generateLoginPageHtml(request, loginError, logoutSuccess);  
            response.setContentType("text/html;charset=UTF-8");  
            response.setContentLength(loginPageHtml.getBytes(StandardCharsets.UTF_8).length);  
            response.getWriter().write(loginPageHtml);  
            return;  
        }  
        chain.doFilter(request, response);  
    }  
}
```

Note: We can add debug point to doFilter methods in DefaultLoginPageGeneratingFilter & AuthorizationFilter class.

3) org.springframework.security.web.authentication.AbstractAuthenticationProcessingFilter

- Implementation class – OneTimeTokenAuthenticationFilter, UsernamePasswordAuthenticationFilter, WebAuthnAuthenticationFilter
- **Contains majority of authentication logic.**

Note: We can debug from FilterChainProxy to understand the internal flow. Some important classes while debugging

- FilterChainProxy – doFilter () method
- VirtualFilterChain implements FilterChain – doFilter () method: In this method, a doFilter() call will be made to each of the 16 default filters in default order
- Below default Order (1st Parent abstract class doFilter () method will execute then child specific doFilter() method will execute)
 1. DisableEncodeUrlFilter extends OncePerRequestFilter
 2. WebAsyncManagerIntegrationFilter extends OncePerRequestFilter
 3. SecurityContextHolderFilter extends GenericFilterBean
 4. HeaderWriterFilter extends OncePerRequestFilter
 5. CsrfFilter extends OncePerRequestFilter
 6. LogoutFilter extends GenericFilterBean
 7. **UsernamePasswordAuthenticationFilter** extends **AbstractAuthenticationProcessingFilter**
 8. DefaultResourceFilter extends GenericFilterBean
 9. DefaultLoginPageGeneratingFilter extends GenericFilterBean
 10. DefaultLogoutPageGeneratingFilter extends GenericFilterBean
 11. BasicAuthenticationFilter extends OncePerRequestFilter
 12. RequestCacheAwareFilter extends GenericFilterBean
 13. SecurityContextHolderAwareRequestFilter extends GenericFilterBean
 14. AnonymousAuthenticationFilter extends GenericFilterBean
 15. ExceptionTranslationFilter extends GenericFilterBean
 16. AuthorizationFilter extends GenericFilterBean

Authentication Object:

- Authentication is an interface present org.springframework.security.core &
- Implementation classes – AbstractAuthenticationToken, OneTimeTokenAuthenticationToken, UsernamePasswordAuthenticationToken (used as default), etc.
- AbstractAuthenticationToken acts as base class for Authentication objects.

AuthenticationManager:

- It is a functional interface with authentication (Authentication authObj) method present in org.springframework.security.authentication
- Implementation classes – ProviderManager (used as default) etc.
- ProviderManager – Iterates an authentication request through a list of AuthenticationProviders.

Authentication Provider:

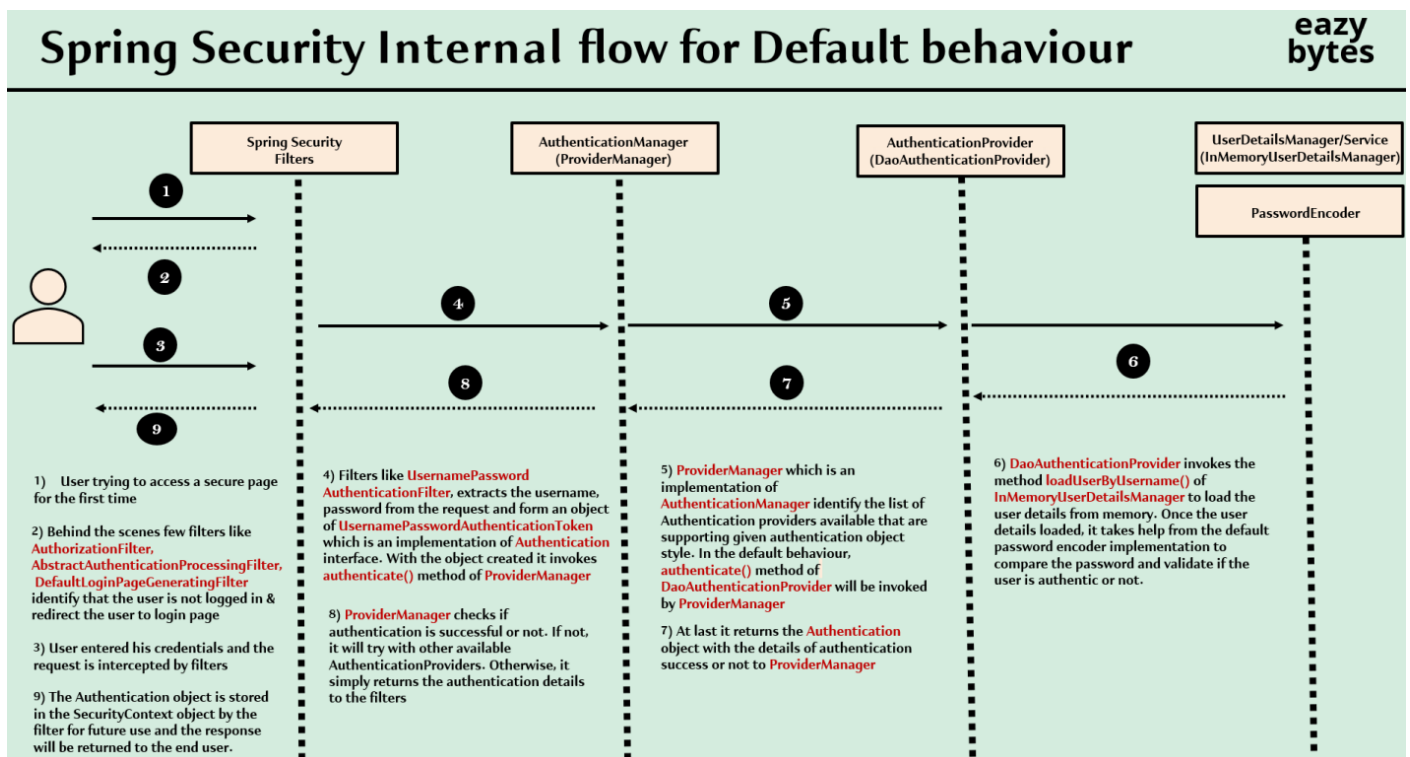
- It is an interface present in org.springframework.security.authentication
- Indicates a class can process a specific Authentication implementation.
- Implementation classes: AbstractUserDetailsAuthenticationProvider, DaoAuthenticationProvider (used as default), etc.
- AbstractUserDetailsAuthenticationProvider – Acts a base class & allows subclasses to override & work with UserDetails objects.

UserDetailsService:

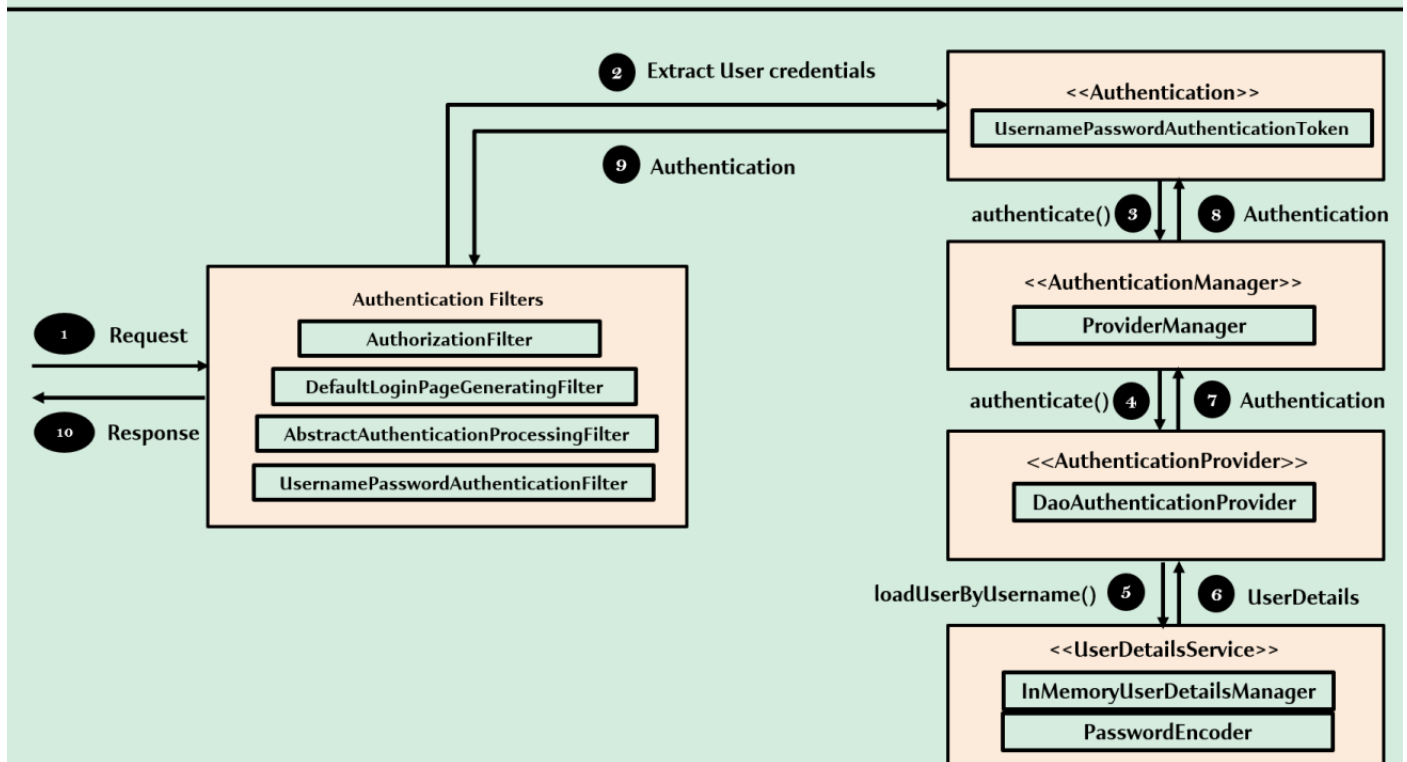
- It is a core interface that loads user-specific data present in org.springframework.security.core.userdetails.
- It is used throughout the framework as a user DAO & is the strategy used by the DaoAuthenticationProvider.
- Implementation classes – UserDetailsService, InMemoryUserDetailsService, JdbcUserDetailsService, CachingUserDetailsService etc.
- UserDetailsService interface acts as base to InMemoryUserDetailsService & JdbcUserDetailsService.

PasswordEncoder:

- Service interface for encoding passwords present in org.springframework.security.crypto.password
- Implementation classes – BcryptPasswordEncoder, DelegatingPasswordEncoder, Pbkdf2PasswordEncoder etc.



Spring Security Internal flow for Default behaviour



Note:

- Spring Security framework handles all the subsequent requests by creating & publishing the JSESSIONID in the browser.
- As long as a valid JSESSIONID cookie mapped to backend server security context details is present in the browser, we won't need to login again and we can access any protected API or MVC path without re login.

Please sign in

Sign in

Application

Manifest

Service workers

Storage

Storage

Local storage

Session storage

Extension storage

IndexedDB

Cookies

Cache storage

Storage buckets

Filter

Name

Value

Dom...

Path

E

JSESSIONID

A3260BE6E837D9A316AA7507AAC448EA

local...

/

S

No cookie selected

Notes

- This way, Spring Security framework is doing smart job of not invoking the actual authentication every time. If we try to invoke actual authentication for each & every request then definitely it's going to have some performance impact but by using this JSESSIONID cookie, Spring Security framework is doing a smart job here.
- This is a default behavior of Spring Security framework. There are better & advanced approaches also available like JWT tokens, OAuth2.0 framework, OpenID framework.

Changing the default Security Configuration

- By default, the Spring Security tries to protect all the APIs but we want to customize Spring Security behaviour based upon the business requirement.
- Before we get started with the Customizations, we need to understand the piece of code responsible for default behaviour.
- Refer org.springframework.boot.autoconfigure.security.servlet.[SpringBootWebSecurityConfiguration](#) class & method defaultSecurityFilterChain (HttpSecurity http)

```
class SpringBootWebSecurityConfiguration {  
  
    The default configuration for web security. It relies on Spring Security's content-negotiation  
    strategy to determine what sort of authentication to use. If the user specifies their own  
    SecurityFilterChain bean, this will back-off completely and the users should specify all the bits  
    that they want to configure as part of the custom security configuration.  
  
    @Configuration(proxyBeanMethods = false)  
    @ConditionalOnDefaultWebSecurity  
    static class SecurityFilterChainConfiguration {  
  
        @Bean  
        @Order(SecurityProperties.BASIC_AUTH_ORDER)  
        SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {  
            http.authorizeHttpRequests(( AuthorizationManagerRequestMat... requests) -> requests.anyRequest().authenticated());  
            http.formLogin(withDefaults());  
            http.httpBasic(withDefaults());  
            return http.build();  
        }  
    }  
}
```

- The code present in defaultSecurityFilterChain () method is trying to create a bean of SecurityFilterChain & is responsible to secure all the APIs in our application.
- By default, formLogin & httpBasic login are enabled.
- **Form** – Any UI application that can have forms like HTML. The end user can enter the credential inside the forms. So, this style of login we call as Form Login.
- **HttpBasic** – If someone sends the credentials inside the http Request headers by Base64 encoding them, we call it standard as http basic. HttpBasic style of authentication is used in scenarios where we want to authenticate an API invocation.
- Beside these 2 login styles, we have other advanced style of logins such as OAuth2 login.
- If we want to customize the Spring Security behaviour then we have to declare similar kind of method, which is going to create a bean of SecurityFilterChain.
- Whenever we enable the **FormLogin** inside our web application, then the extraction of the credentials from the request is going to be taken care by the [UsernamePasswordAuthenticationFilter](#). For more understanding, we can debug same class.
- Whereas, if we're using the **HttpBasic** style of login, then the same credential extraction activities will be taken care by [BasicAuthenticationFilter](#). For more understanding, we can debug same class.

Modifying the security config as per our custom requirements

1. permitAll ()

- By using anyRequest ().permitAll(), we can permit all the requests coming towards our web application APIs, Paths using Spring Security framework like shown below:

```
@Configuration
public class ProjectSecurityConfig {

    @Bean
    SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {

        http.authorizeHttpRequests((requests) -> requests.anyRequest().permitAll())
            .formLogin(Customizer.withDefaults())
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }
}
```

- Not recommended for Production

2. denyAll ()

- By using anyRequest().denyAll(), we can deny all the requests coming towards our web application APIs, Paths using Spring Security framework like below:

```
@Configuration
public class ProjectSecurityConfig {

    @Bean
    SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {

        http.authorizeHttpRequests((requests) -> requests.anyRequest().denyAll())
            .formLogin(Customizer.withDefaults())
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }
}
```

- Not recommended for Production
- denyAll () method is not going to be used very commonly in the real projects, but we can use in case we want to deny the upcoming request towards few APIs for a specific set of time.

Note:

- If we disable both Form Login & Http Basic login, then we will get 403 error

Custom Security Configurations inside Spring Security Framework

- By using `requestMatchers().authenticated()` & `requestMatchers().permitAll()`, we can secure the web application APIs, Paths as per our custom requirements using Spring Security framework like shown below:

```
@Configuration
public class ProjectSecurityConfig {

    @Bean
    SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {

        http.authorizeHttpRequests((requests) -> requests
            .requestMatchers("/myAccount", "/myBalance", "/myLoans", "/myCards").authenticated()
            .requestMatchers("/notices", "/contact").permitAll()
            .formLogin(Customizer.withDefaults())
            .httpBasic(Customizer.withDefaults());
        return http.build();
    }
}
```

Disable Form Login inside Spring Security Framework

- If your Spring Security-protected endpoints are only accessed programmatically (using APIs) by clients (such as mobile apps or other services) and there's no need for a web-based login form, disabling form login can be appropriate.
- This can be done in the below way:

```
http.authorizeHttpRequests((requests) -> requests
    .requestMatchers("/myAccount", "/myBalance", "/myLoans", "/myCards").authenticated()
    .requestMatchers("/notices", "/contact", "/error").permitAll()
    .formLogin(flc -> flc.disable())
    .httpBasic(Customizer.withDefaults());
```

Note:

- By default, Spring Security framework will always redirect to the error page if any runtime exception happens behind the scenes. But this error page is also going to be secured by default.
- We should always permit these error page to everyone so that we can see the runtime exceptions inside our application at any time.

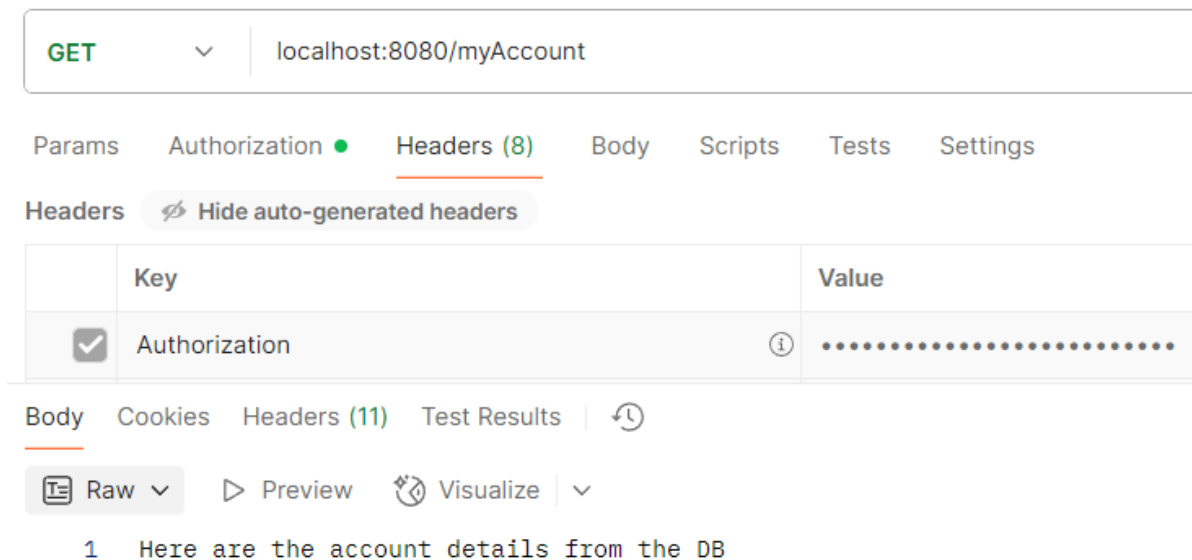
Disable Http Basic Login inside Spring Security Framework

- If your application primarily serves web pages & requires users to interact via a browser, then HTTP Basic authentication may not be required.
- HTTP Basic style logic can be disabled in the following way:

```
http.authorizeHttpRequests((requests) -> requests
    .requestMatchers("/myAccount", "/myBalance", "/myLoans", "/myCards").authenticated()
    .requestMatchers("/notices", "/contact", "/error").permitAll()
    .formLogin(Customizer.withDefaults())
    .httpBasic(hbc -> hbc.disable());
```

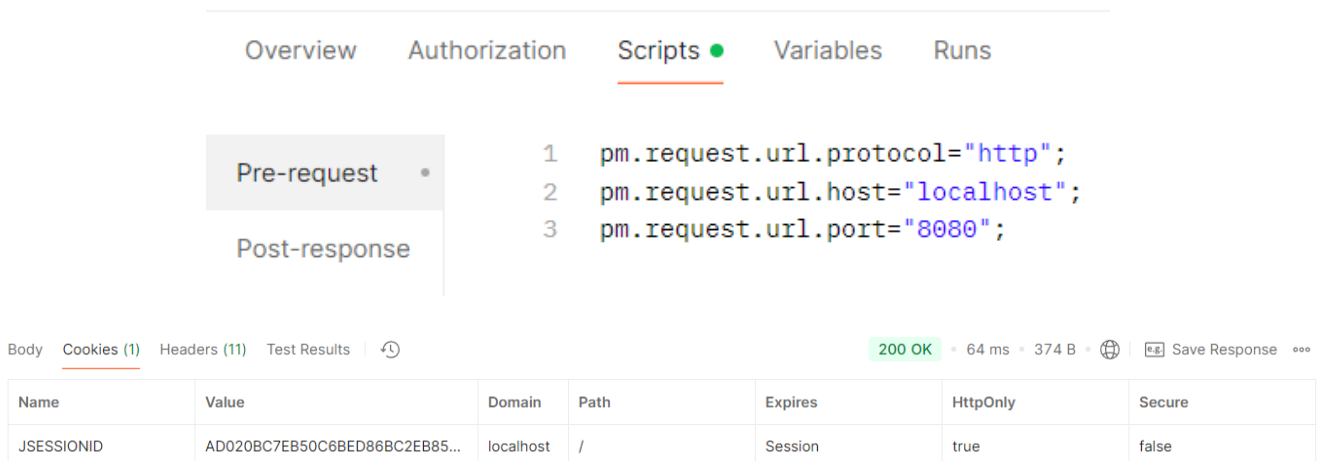
HttpBasic Authentication testing via Postman

- If some API can't be invoked from the Browser, then we can use Postman for all such scenarios.
- While sending Basic Auth via Request header, a key named "Authorization" & value prefixed with {Basic} string will be sent to backend for authentication.
- In Backend, BasicAuthenticationFilter will be invoked.



- **Trick:** We can use different ways to configure base url in postman
 1. We can set base url as variable with scope as Collection, Environment or global. We can check the set variable name on the collection or selected scope level.
 2. We can write JS scripts on Collection level -> Pre-request (executed before API invocation) or post-request

Spring Security

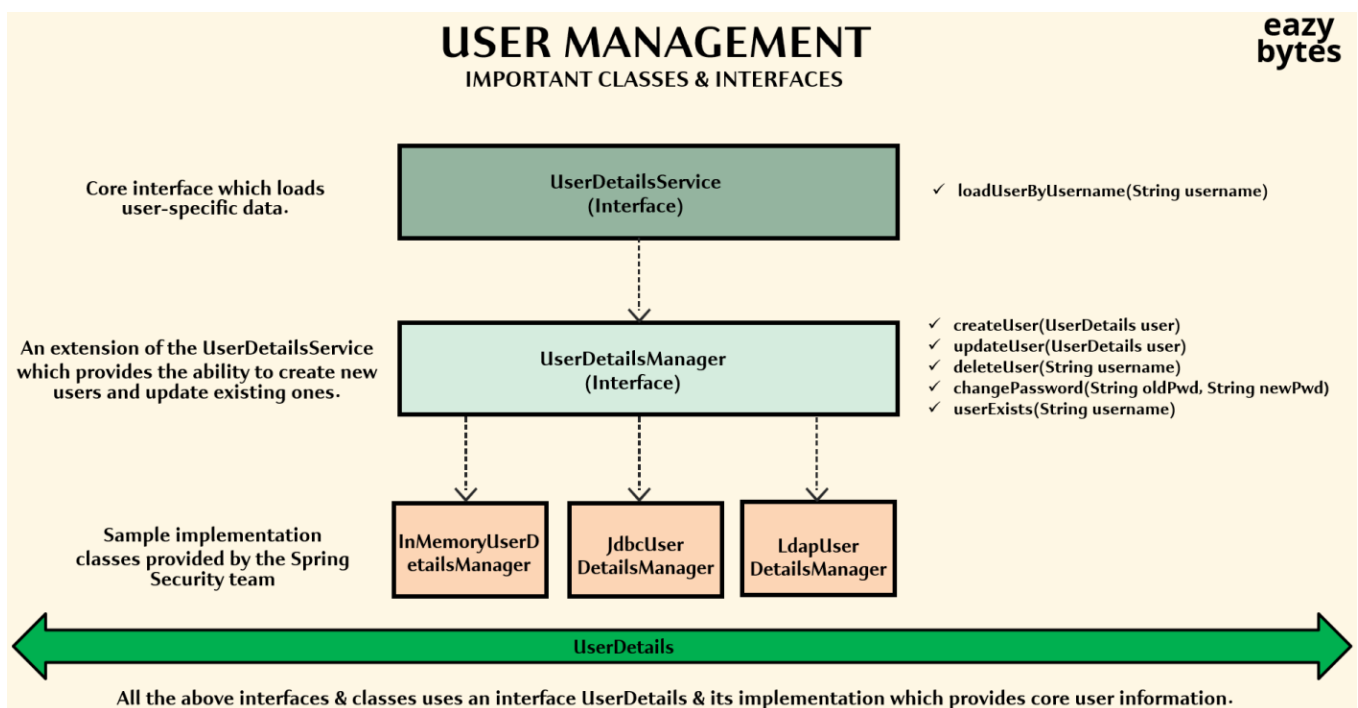


Configure Users using InMemoryUserDetailsManager

- Instead of defining a single user inside application.properties, as next step we can define multiple users along with their authorities with the help of InMemoryUserDetailsManager & UserDetails
- In this section, we will focus on creating multiple users by using the memory of the Spring Boot application. Later, we will be going to enhance this application by creating multiple users inside a storage system like database.
- All user-related operations are supported by UserDetailsService interface & its child i/f UserDetailsManager interface.
- Whenever we want to store the credentials inside the memory of the application, we need to create the object of one of the implementations of UserDetailsService i.e., InMemoryUserDetailsManager.

UserDetailsService interface

- Core interface with loadUserByUsername (String username) method, which loads user-specific data based on username from a storage system.
- The storage system can be in-memory or a database.



UserDetailsManager interface

- ALL CRUD Operations: It contains the methods to create a new user, to delete end user, to update the user details, to change the password, and to check whether the user exists with the given username.
- It is also going to inherit the fetch related method from the parent interface (UserDetailsService)
- **Implementation classes** – InMemoryUserDetailsManager, JdbcUserDetailsManager

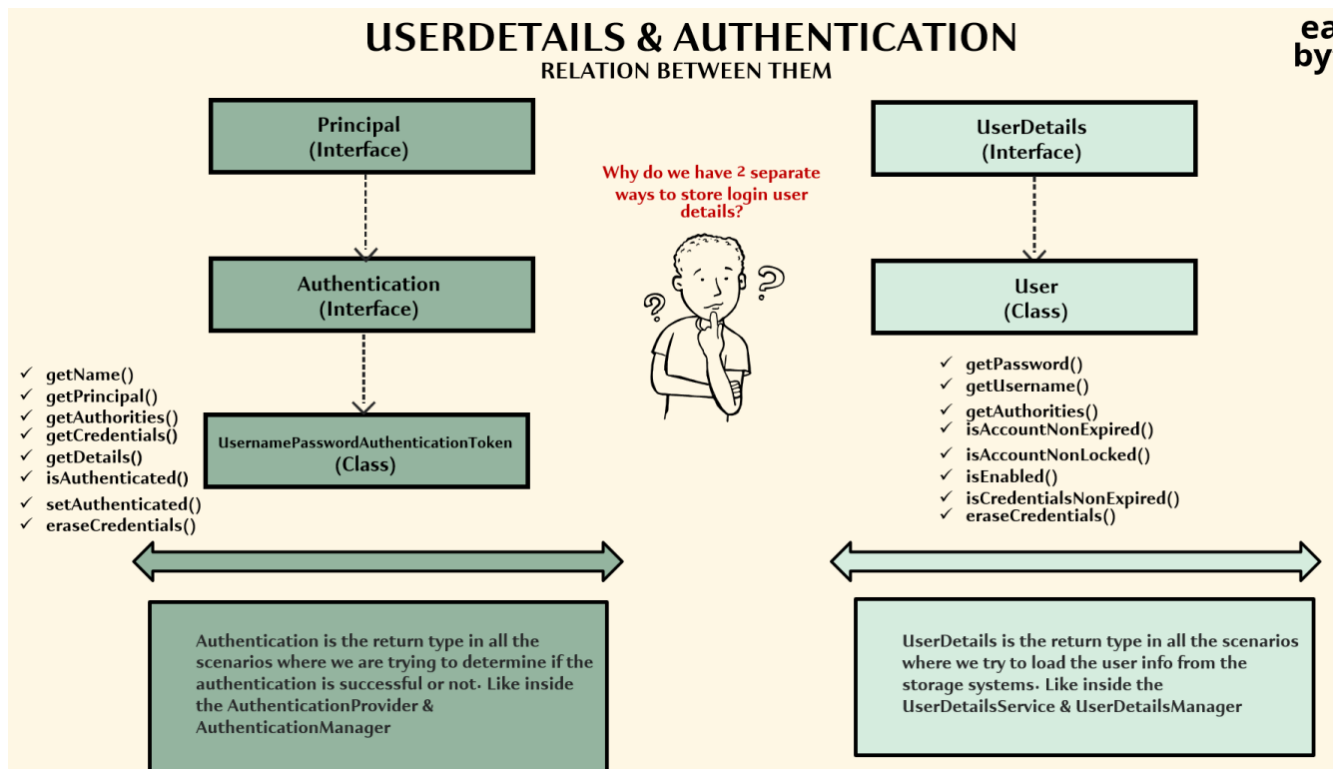
Q. Why Spring Security team separated user management methods into 2 different interfaces i.e., **UserDetailsService** i/f & **UserDetailsManager** i/f

Ans:

- UserDetailsService i/f has only method to load user details by username i.e., Read Operation as there can be use cases where sometimes organizations only want to authenticate the user but never have a scenario where the end user can register or perform any other (create, delete, update) operations directly.
- UserDetailsManager i/f allows all other operations in addition to read i.e., create, delete, update. This can be used where organization wants to have complete functionality.

Note: In case we're not satisfied with already inbuilt UserDetailsManager implementation, we can create custom UserDetailsService as well.

- Inside Spring Security, an end-user is represented by UserDetails interface.
- UserDetails i/f implementation classes: User
- The magic of converting the UserDetails object i.e., User into the Authentication object i.e., UsernamePasswordAuthenticationToken is going to happen inside the AuthenticationProvider implementation classes: DaoAuthenticationProvider
- UserDetails i/f implementation methods are specifically required once the Authentication is successful.



- By default, if we create user like below, we're not conveying anything to the Spring Security framework, it will store the credential in a plain text format.
- This is not recommended approach for any kind of application because anyone can see your actual password.

```

ProjectSecurityConfig.java
15 public class ProjectSecurityConfig {
16
17
18
19
20
21
22
23
24
25
26
27 @Bean no usages new *
28 public UserDetailsService userDetailsService() {
29     UserDetails user = User.withUsername("user").password("12345").authorities("read").build();
30     UserDetails admin = User.withUsername("admin").password("54321").authorities("read").build();
31     return new InMemoryUserDetailsManager(user, admin);
32 }
33 }

User.java
63 public class User implements UserDetails, CredentialsContainer {
326 public static final class UserBuilder {
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
1001
1002
1003
1004
1005
1006
1007
1008
1009
1010
1011
1012
1013
1014
1015
1016
1017
1018
1019
1020
1021
1022
1023
1024
1025
1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079
1080
1081
1082
1083
1084
1085
1086
1087
1088
1089
1090
1091
1092
1093
1094
1095
1096
1097
1098
1099
1100
1101
1102
1103
1104
1105
1106
1107
1108
1109
1110
1111
1112
1113
1114
1115
1116
1117
1118
1119
1120
1121
1122
1123
1124
1125
1126
1127
1128
1129
1130
1131
1132
1133
1134
1135
1136
1137
1138
1139
1140
1141
1142
1143
1144
1145
1146
1147
1148
1149
1150
1151
1152
1153
1154
1155
1156
1157
1158
1159
1160
1161
1162
1163
1164
1165
1166
1167
1168
1169
1170
1171
1172
1173
1174
1175
1176
1177
1178
1179
1180
1181
1182
1183
1184
1185
1186
1187
1188
1189
1190
1191
1192
1193
1194
1195
1196
1197
1198
1199
1200
1201
1202
1203
1204
1205
1206
1207
1208
1209
1210
1211
1212
1213
1214
1215
1216
1217
1218
1219
1220
1221
1222
1223
1224
1225
1226
1227
1228
1229
1230
1231
1232
1233
1234
1235
1236
1237
1238
1239
1240
1241
1242
1243
1244
1245
1246
1247
1248
1249
1250
1251
1252
1253
1254
1255
1256
1257
1258
1259
1260
1261
1262
1263
1264
1265
1266
1267
1268
1269
1270
1271
1272
1273
1274
1275
1276
1277
1278
1279
1280
1281
1282
1283
1284
1285
1286
1287
1288
1289
1290
1291
1292
1293
1294
1295
1296
1297
1298
1299
1300
1301
1302
1303
1304
1305
1306
1307
1308
1309
1310
1311
1312
1313
1314
1315
1316
1317
1318
1319
1320
1321
1322
1323
1324
1325
1326
1327
1328
1329
1330
1331
1332
1333
1334
1335
1336
1337
1338
1339
1340
1341
1342
1343
1344
1345
1346
1347
1348
1349
1350
1351
1352
1353
1354
1355
1356
1357
1358
1359
1360
1361
1362
1363
1364
1365
1366
1367
1368
1369
1370
1371
1372
1373
1374
1375
1376
1377
1378
1379
1380
1381
1382
1383
1384
1385
1386
1387
1388
1389
1390
1391
1392
1393
1394
1395
1396
1397
1398
1399
1400
1401
1402
1403
1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
2662
2663
2664
2665
2666
2667
2668
2669
2670
2671
2672
2673
2674
2675
2676
2677
2678
2679
2680
2681
2682
2683
2684
2685
2686
2687
2688
2689
2690
2691
2692
2693
2694
2695
2696
2697
2698
2699
2700
2701
2702
2703
2704
2705
2706
2707
2708
2709
2710
2711
2712
2713
2714
2715
2716
2717
2718
2719
2720
2721
2722
2723
2724
2725
2726
2727
2728
2729
2730
2731
2732
2733
2734
2735
2736
2737
2738
2739
2
```


There was an unexpected error (type=Internal Server Error, status=500).

Given that there is no default password encoder configured, each password must have a password encoding prefix. Please either prefix this password with '{noop}' or set a default password encoder in 'DelegatingPasswordEncoder'.

java.lang.IllegalArgumentException: Given that there is no default password encoder configured, each password must have a password encoding prefix. Please either prefix this password with '{noop}' or set a default password encoder in 'DelegatingPasswordEncoder'.

- Without configuring the PasswordEncoder bean, Spring Security will throw an error. Either we configure a PasswordEncoder bean or try to prefix our passwords with the {noop}

Configuring PasswordEncoder using PasswordEncoderFactories

- Problem with above approach:
 1. Hard coding the credentials inside the source code. Anyone having access to source code can easily see the password.
 2. Password is being stored as a plain text internally inside the application memory. If some hacker has access to our application head dump, the hacker can easily get what's the password by analyzing the heap dump.
- In order to solve above 2 problems, we need to tell Spring Security framework to use the PasswordEncoder to store the passwords in an encrypted or in a hashing format.

```
@Bean
public PasswordEncoder passwordEncoder() {
    return PasswordEncoderFactories.createDelegatingPasswordEncoder();
}

@Bean
public UserDetailsService userDetailsService() {
    UserDetails user = User.withUsername("user")
        .password("{noop}12345")
        .authorities("read")
        .build();

    UserDetails admin = User.withUsername("admin")
        .password("{bcrypt}$2a$12$v/X62C2bs.UrdI41uqHS/e1KgYR9iOKzMsNUKLrZok00QRwUWS1VW")
        .authorities("admin")
        .build();

    return new InMemoryUserDetailsManager(user, admin);
}
```

- PasswordEncoderFactories.createDelegatingPasswordEncoder() gives the flexibility to store the password in either Spring Security recommended default format or any specific format by adding the specific prefix to the password.
- We can refer this site to generate bcrypt format password - <https://bcrypt-generator.com/>

CompromisedPasswordChecker

- Using CompromisedPasswordChecker implementation, we can check if a password has been compromised. This feature is released as part of Spring Security 6.3 version.

```
@Bean
public CompromisedPasswordChecker compromisedPasswordChecker() {
    return new HaveIBeenPwnedRestApiPasswordChecker();
}
```

- **Implementation classes** - HaveIBeenPwnedRestApiPasswordChecker
- Recommended for Production

Tips: We can use warp terminal for better UI experience compared to our traditional terminals that comes along with OS. Using this terminal, we can also ask AI if we've any questions around a command.

Docker command to add my SQL:

- `docker run -p 3306:3306 --name springsecurity -e MYSQL_ROOT_PASSWORD=root -e MYSQL_DATABASE=eazybank -d mysql`

SQL Client - <https://github.com/sqllectron/sqllectron-gui/releases/tag/v1.38.0>

Understanding JdbcUserDetailsManager & creating Users inside the DB

- JdbcUserDetailsManager is a JDBC implementation of the UserDetailsManager.
- It takes care of all the CRUD operations for managing users in a database.
- It uses a JDBC template to execute the SQL queries.

```
@Bean no usages new *
public UserDetailsService userDetailsService(DataSource dataSource) {
    return new JdbcUserDetailsManager(dataSource);
}
```

Using JdbcUserDetailsManager to perform authentication

- Add spring-boot-starter-jdbc, mysql-connector-j, spring-boot-starter-data-jpa to pom.xml to resolve JdbcUserDetailsManager class compilation errors.

```
public class JdbcUserDetailsManager extends JdbcDaoImpl implements UserDetailsManager, GroupManager {
```

- **JdbcDaoImpl** provides default user schema DDL location - `org/springframework/security/core/userdetails/jdbc/users.ddl`
- We need to add few properties to application.properties to help spring boot application to connect to the MySQL DB.

```
spring.datasource.url=jdbc:mysql://${DATABASE_HOST:localhost}:${DATABASE_PORT:3306}/${DATABASE_NAME:eazybank}
spring.datasource.username=${DATABASE_USERNAME:root}
spring.datasource.password=${DATABASE_PASSWORD:root}
spring.jpa.show-sql=${JPA_SHOW_SQL:true}
spring.jpa.properties.hibernate.format_sql=${HIBERNATE_FORMAT_SQL:true}
```

- `spring.jpa.show-sql`: JPA Framework will print all the SQL statements that are going to be executed against the DB on the console
- `spring.jpa.properties.hibernate.format_sql`: The queries that are going to be printed on the console, are not in a readable format by default. This property is used to make queries print in readable format.

Authorities Table

username	authority
admin	admin
user	read

Users Table

username	password	enabled
admin	{bcrypt}\$2a\$12\$88.f6upbBvy0okEa7OfHFuorV29qeK.sVbB9VQ6J6dWM1bW6Qef8m	1
user	{noop}EazyBytes@12345	1

Note: This way is good option for beginners but enterprise applications may not accept to this proposal i.e., using already existing & recommended User table with pre-defined schema by Spring Security. They would require a custom table for Authentication.

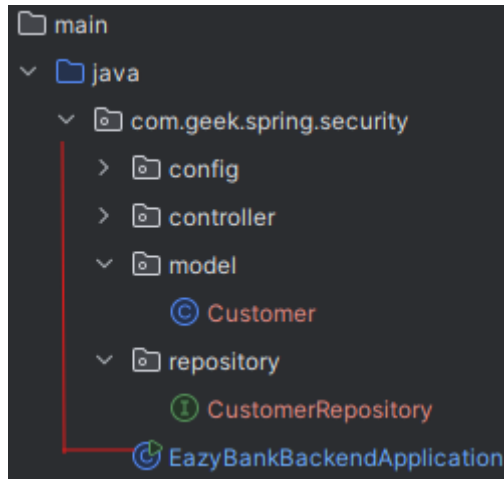
Custom Tables for Authentication

Customer Table

id	email	pwd	role
1	happy@example...	{noop}EazyBytes@12345	read
2	admin@example...	{bcrypt}\$2a\$12\$88.f6upbBvy0okEa7OfHFuorV29qeK.sVbB9VQ6J6dWM1bW6Qef8m	admin

- We will use Spring Data JPA framework to execute DB related logic
- If we've created entities like Customer & repositories under the main package only then both annotations are optional.

```
@EnableJpaRepositories("com.geek.spring.security.repository")
@EntityScan("com.geek.spring.security.model")
```



```
@Entity 3 usages
@Table(name = "customer")
@Getter @Setter
public class Customer {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private long id;
    private String email;
    private String pwd;
    @Column(name = "role")
    private String role;
}
```

Note:

- **@EnableWebSecurity** is completely optional in the Spring Boot environment as spring boot is smart enough to enable the security based upon the dependencies that it sees inside the pom.xml
- **@EnableWebSecurity** is mandatory in case we're trying to implement the Spring Security in a normal Spring framework application which is not using any Spring boot functionality.

Creating our own custom implementation of UserDetailsService

Authentication Providers depend on the UserDetailsService/Manager implementation to fetch the user details either from Memory using InMemoryUserDetailsManager bean or from database using JdbcUserDetailsManager bean.

The problem with JdbcUserDetailsManager:

- It forces us to use the predefined schema, predefined table structure.

In order to let the DaoAuthenticationProvider to use our own custom tables for the authentication, we should create our own bean by either implementing the UserDetailsManager or UserDetailsService.

Case 1 – Use UserDetailsManager: Then we will have to define all the CRUD method inside UserDetailsManager i/f

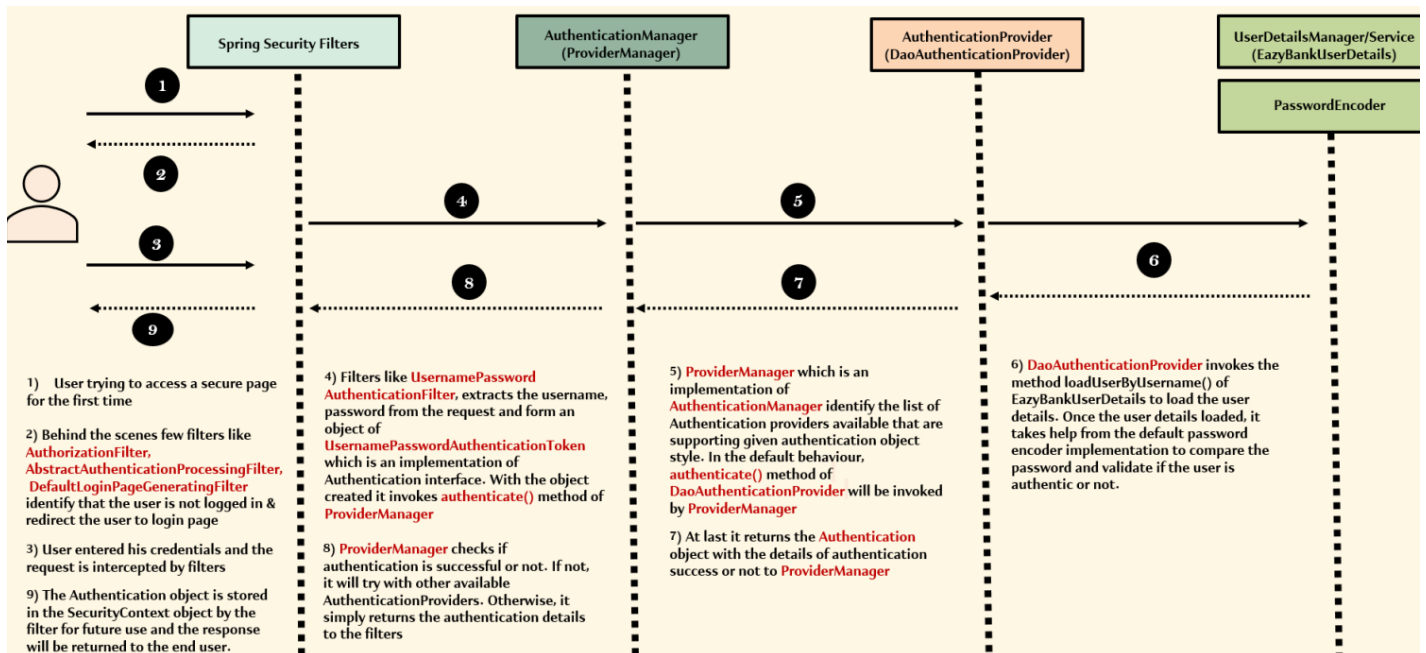
Case 2 – Use UserDetailsService: Then we will have to only define the loadUserByUsername () method.

```
@Service
@RequiredArgsConstructor
public class EazyBankUserDetailsService implements UserDetailsService {

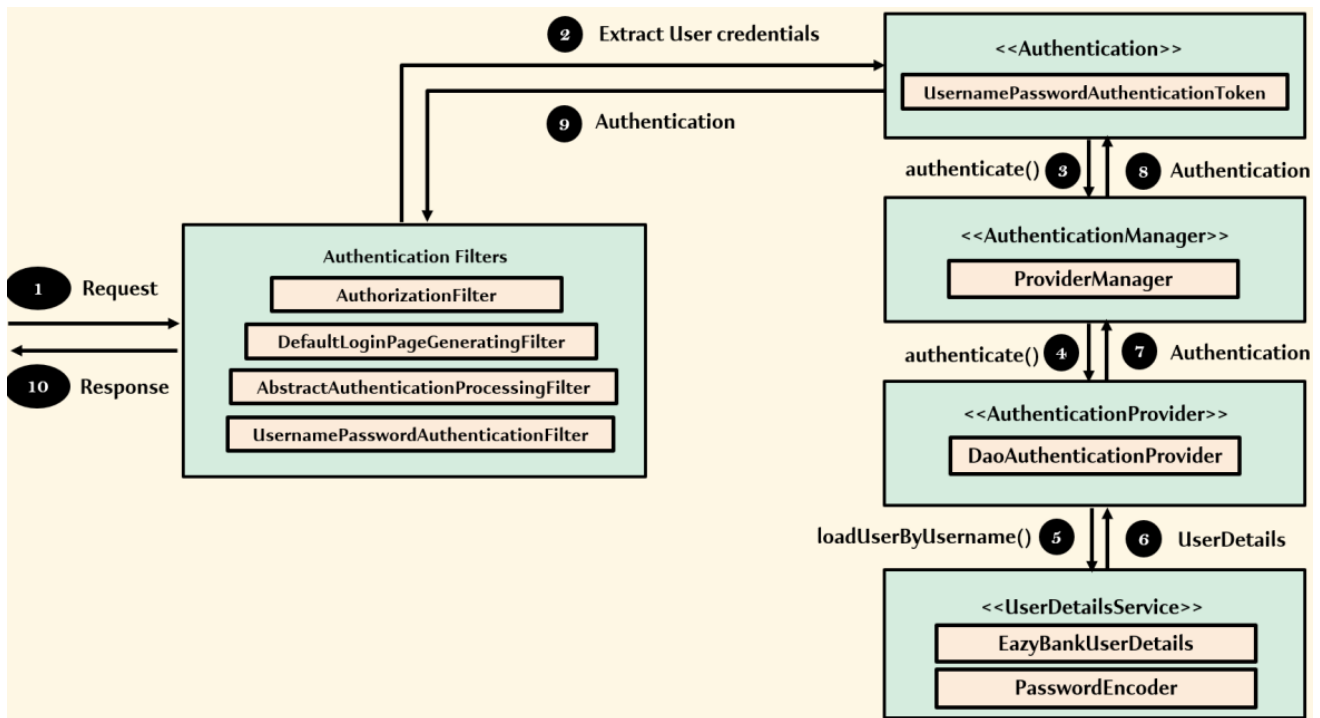
    private final CustomerRepository customerRepository;

    @Override
    public UserDetails loadUserByUsername(String username) throws UsernameNotFoundException {
        Customer customer = customerRepository.findByEmail(username).orElseThrow(() -> new
            UsernameNotFoundException("User details not found for the user: " + username));
        List<GrantedAuthority> authorities = List.of(new SimpleGrantedAuthority(customer.getRole()));
        return new User(customer.getEmail(), customer.getPwd(), authorities);
    }
}
```

Sequence Flow with our own UserDetailsService implementation:



Sequence Flow with our own UserDetailsServiceImpl Implementation:



Building a new REST API to allow the registration of a new user

- UserDetailsManager i/f provides a method: createUser (UserDetails user) to create user but we should not involve Spring Security Framework during the registration of a new user due to below reasons:
 1. If we start involving UserDetailsManager i/f, obviously we need to support lot many other functionalities that may not be needed.
 2. When we're using above i/f method, we're restricted to the input parameter of type UserDetails. If we want to accept any other parameter, then this is not a friendly option.
 3. Organizations should always leverage the REST APIs or MVC paths that allow the registration of the user, CRUD operations. Recommended to only involve the Spring Security framework during the authentication process. That's why we've created our own UserDetailsServiceImpl implementation above.
- There are certain modifications that we've to do in order to support /register user API
 1. Permit "/register" API as public API
 2. Inside Spring Security framework, whenever we're trying to invoke an API path with Create, Update or delete data functionality, in such scenarios, by default Spring Security framework will stop such request due to the CSRF protection that it's providing.
 3. Spring Security for the reading operations (GET), it won't enforce the CSRF protection whereas for the data-alternating methods (POST, PUT, DELETE), it will enforce the CSRF protection.

To Resolve CSRF issue, either we need to properly implement the CSRF protection inside our application or we need to disable the CSRF protection provided by the Spring Security framework.

```
http.csrf( CsrfConfigurer<HttpSecurity> csrfConfigurer -> csrfConfigurer.disable())
```

POST /register

Params Authorization Headers (9) Body Scripts Setting

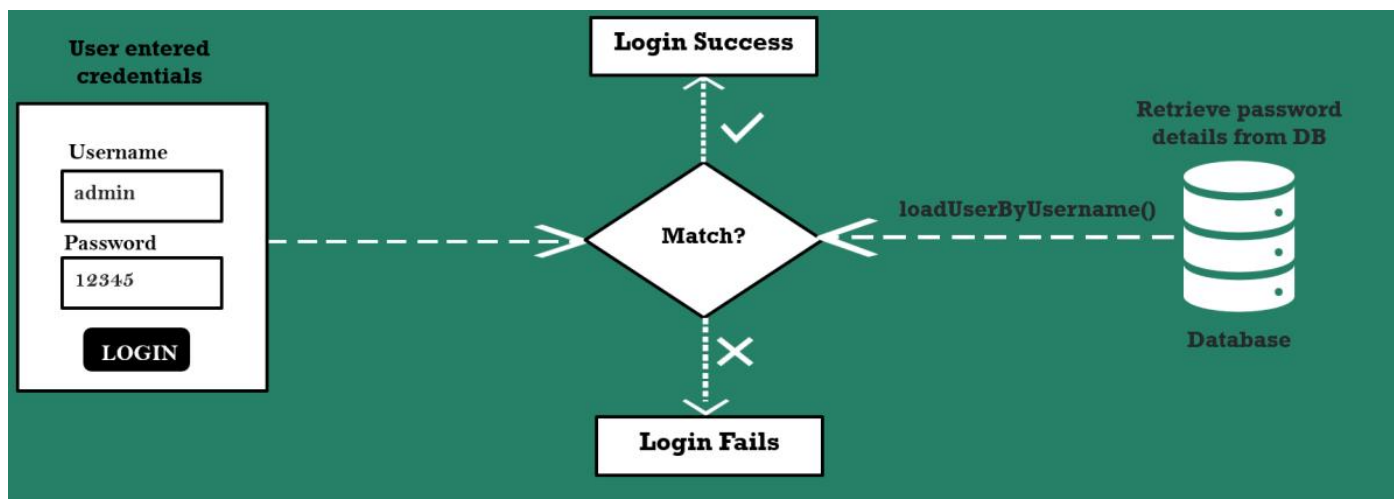
☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ bin

```
1 {
2   "email": "john@example.com",
3   "pwd": "EazyBytes@12345",
4   "role": "user"
5 }
```

Note: There is no need customize AuthenticationManager or ProviderManager

Password Management with Password Encoders

How our passwords validated without PasswordEncoders – NoOpPasswordEncoder



Storing the passwords in a plain text inside a storage system like DB will have Integrity & Confidentiality issues. So this is not a recommended approach for Production applications.

Encoding Vs Encryption Vs Hashing

Different ways for Data Privacy

Encoding

- ✓ Encoding is defined as the process of converting data from one form to another and has nothing to do with cryptography.
- ✓ It involves no secret and completely reversible.
- ✓ Encoding can't be used for securing data. Below are the various publicly available algorithms used for encoding.

Ex: ASCII, BASE64, UNICODE

Encryption

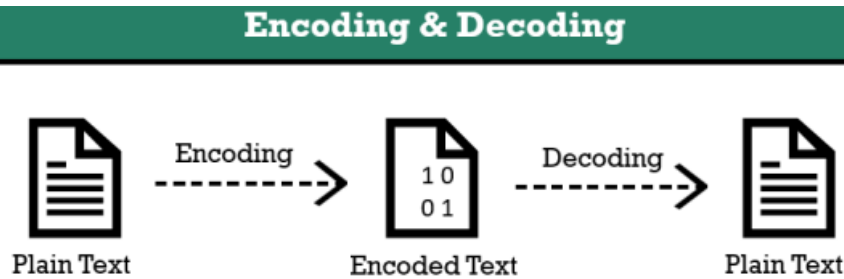
- ✓ Encryption is defined as the process of transforming data in such a way that guarantees confidentiality.
- ✓ To achieve confidentiality, encryption requires the use of a secret which, in cryptographic terms, we call a "key".
- ✓ Encryption can be reversible by using decryption with the help of the "key". As long as the "key" is confidential, encryption can be considered as secured.

Hashing

- ✓ In hashing, data is converted to the hash value using some hashing function.
- ✓ Data once hashed is non-reversible. One cannot determine the original data from a hash value generated.
- ✓ Given some arbitrary data along with the output of a hashing algorithm, one can verify whether this data matches the original input data without needing to see the original data

Encoding & Decoding Process

- Not recommended approach for password management.

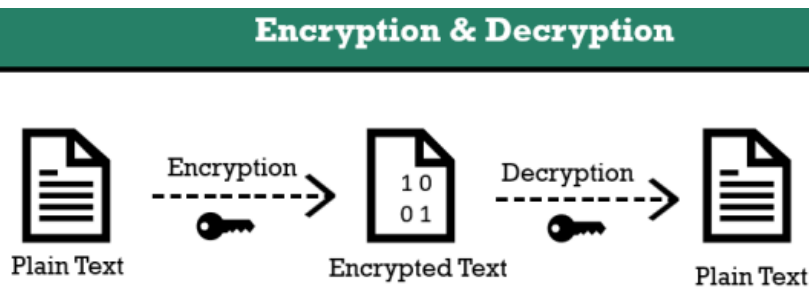


One of the most common use cases of Encoding & Decoding

- Whenever a system is not capable of dealing with the binary data like images etc. then for all such scenarios Encoding & Decoding will be used.
- For e.g., Outlook application. Since mail applications are not capable of storing, processing or dealing with binary data like images, videos as attachment, behind the scene, all these binary data is going to be encoded with the help of formats like BASE64 encoding.
- Once the encoding is completed, these files will look like a normal text file. On other side, these normal text files will be decoded to the original binary data.
- **Command:**
 1. `openssl base64 -in plainText.txt -out encode.txt`
 2. `openssl base64 -d -in encode.txt -out decode.txt`

Encryption & Decryption Process

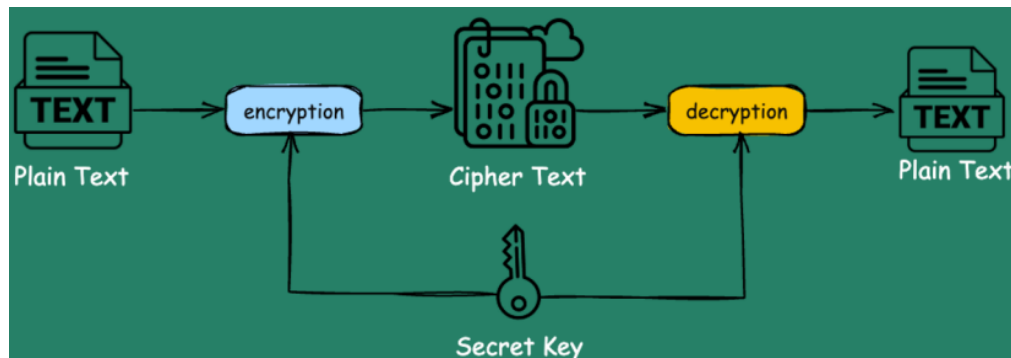
- Not recommended approach for password management.
- Encrypted text is also called Cipher text.



- Encryption is essential in our daily routines,
 - a) whether we're messaging friends on WhatsApp, ensuring a website's authenticity through our browser
 - b) Entering our credit card details while shopping online.
- It safeguards our data from potential threats & unauthorized access, ensuring privacy & security.
- Central to any encryption process are the Encryption Algorithm & the key. While numerous encryption algorithm exists, there are generally 2 types of keys: Symmetric & Asymmetric

1. Symmetric Encryption

- In Symmetric key encryption, the same key used to encrypt the data is used to decrypt the data. It is generally used for encrypting **data at rest**.
- Some famous symmetric encryption algorithms include
 - a) **AES** – known for its security & used globally.
 - b) **Triple DES** – an enhancement of DES providing better security
 - c) **Blowfish** – Notable for its speed & variable key length.



- Each offers distinct advantages for protecting data.

Drawback with Symmetric Encryption

- Though it's trying to provide the confidentiality, if someone loses the secret key to unauthorized persons, then they should be able to do both encryption & decryption.
- That's why the strength of our encryption depends on how strongly we're protecting our secret key.

Problem Scenario:

There is an application "A" & it wants to communicate with various 3rd party applications like P1, P2, P3 etc. using Symmetric encryption. A will share the secret key to P1, P2, & P3. Application "A" can ensure that this key is well secured but can't guarantee if the other third-party applications can ensure the safety.

In this case, entire encryption process may get compromised.

What is data at rest?

Ans: Data at rest refers to the data stored on a device, such as hard drive, smartphone or cloud storage & is not actively being transmitted or processed.

For e.g., AWS S3 buckets

Inside S3 bucket, we can store any amount of data. AWS gives the option to the customers if they want to encrypt the data at rest. If opted, then AWS is going to encrypt the new data with the secret key known only to AWS.

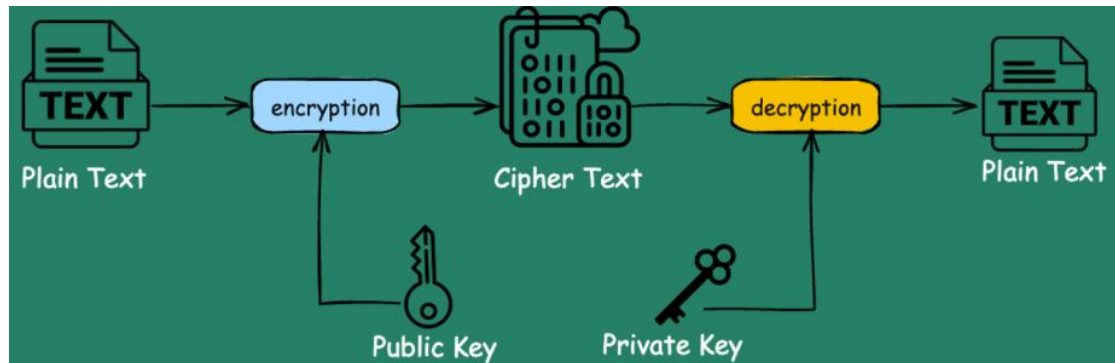
- With that even if someone gets hold onto the data present inside the s3 bucket, it is going to be useless for them because the data is encrypted & they can't get the secret key either from the AWS or from the organization.
- In case the organization is trying to read the data from these S3 buckets, AWS behind the scenes is going to decrypt the data using the secret key & send the data in a plain format to the organization who owns the data.

Use case Recommendation:

- Use the Symmetric encryption only in the scenarios of data at rest & to an extent where there is only a single application use involved.
- Don't use Symmetric encryption if there are multiple parties involved during the encryption & decryption process.

2. Asymmetric Encryption

- In Asymmetric key encryption, a unique lock with 2 distinct keys is used. One for locking (encryption) & one for unlocking (decryption).
- We can keep the private key (decryption) secure with us while freely distributing the public key (encryption) to anyone who want to send us an encrypted message.
- Public key is used to encrypt the messages on sender side & only the private key will be used to decrypt the message on receiver side.



- Asymmetric key encryption is ideal for encrypting data in transit, where we need to share the key with another system.
- The most commonly used asymmetrical algorithms include:
 1. RSA – Rivest-Shamir-Adleman
 2. Diffie-Hellman
 3. ECC - Elliptic Curve Cryptography
 4. PGP – Pretty Good Privacy
- Still, we can't use encryption for password management – We may have people inside the organization only to steal the secret key.
- Though, this encryption option is providing confidentiality option, it's not providing integrity option.

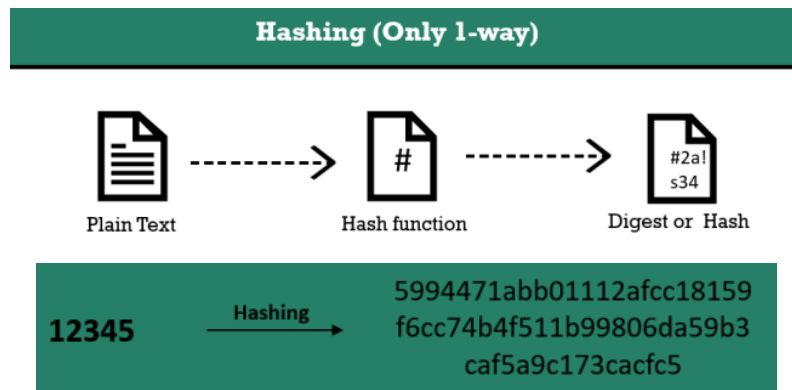
```
Encryption algorithm: AES 256
Secret key: 12345
pbkdf2 (Password based key derived function 2)
- makes encryption more stronger (it is going to add a random salt to derive another secret key)
base64 - to convert the encrypted data to readable format

Command to encrypt
1. openssl enc -aes-256-cbc -pass pass:12345 -pbkdf2 -in plainText.txt -out encrypt.txt
2. openssl enc -aes-256-cbc -pass pass:12345 -pbkdf2 -in plainText.txt -out encrypt.txt -base64

Command to decrypt
1. openssl enc -aes-256-cbc -pass pass:12345 -d -pbkdf2 -in encryptNonReadable.txt -out decryptNonReadable.txt
2. openssl enc -aes-256-cbc -base64 -pass pass:12345 -d -pbkdf2 -in encrypt.txt -out decrypt.txt
```

Hashing

- In hashing, data is converted to the hash value/digest value using some hash function.
- A hash function transforms input of any length (such as a file, message, or video) into a consistent, fixed-length output (e.g., 256 bits for SHA-256).
- When the same input is hashed, it always generated the same resulting digest or hash.



- A fundamental characteristic of a hash function is its irreversibility, meaning it is not possible to deduce the input from its output alone. This property is often referred as “one-way”.



- Suitable for password management as integrity of data is maintained using hashing.
- We can also validate genuine application by comparing the generated SHA with provided SHA
- **Command:**

```
openssl dgst -sha256 anyfile.extension  
  
echo -n "EazyBytes@12345" | openssl dgst -sha256
```

Drawbacks of Hashing

- Based on our discussion about the hashing, it is clear that no one can derive the plain text value based upon a given hash value becoz it's irreversible. With this, we might be under assumption that hashing is 100% suitable for the password management but don't come to this conclusion becoz there are certain drawbacks of hashing we need to discuss to making hashing 100% suitable for Password management.

While this may look like a perfect solution as no one can get to know the plain password from the hash or digest value even if they manage to steal the user data from DB. It has some loop holes. Let's imagine few users they used simple passwords and there is also a good chance that multiple user choosed to have the same password like **123456** or **password** etc. Below is how, it is going to look,

User ID	Username	Plain password	Hash password
1	johndoe	12345	5994471abb01112afcc18159f6cc74b4f511b99806da59b3caf5a9c173cacfc5
2	mike123	password	5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8
3	pavan189	12345	5994471abb01112afcc18159f6cc74b4f511b99806da59b3caf5a9c173cacfc5
4	saanvi180	password	5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8

1. Hashing is always going to give same hash value if the input is same

- If an attacker gains access to hashed password in a DB, they could potentially launch a brute force attack or an exhaustive search (on Rainbow table) by trying all possible passwords.
- This method tests each attempt against the entire database of hashed passwords. Ideally, we aim to limit attackers to targeting only 1 hashed password at a time.

2. Hash functions are designed to be fast, which attackers can exploit to try numerous passwords per second during a brute force attack.

- Ideally, we should implement mechanisms to impede such attacks, slowing down the process significantly.

What is a Brute force attack ?

A brute force attack is a trial-and-error method used by hackers to decode encrypted data, such as passwords or encryption keys. In this type of attack, the attacker systematically tries all possible combinations of characters until the correct one is found. It's essentially like trying every possible key in a lock until the right one opens it.

What is a Rainbow table or Dictionary table attack ?

A rainbow table or Dictionary table attacks are a type of cryptographic attack used to crack password hashes. It involves precomputing and storing a large number of password hashes along with their corresponding plaintext passwords in a table known as a "rainbow table" or a "dictionary table"

When a hashed password is obtained, instead of computing hashes for all possible passwords as in a brute force attack, the attacker can simply look up the hash in the rainbow or dictionary table to find the corresponding plaintext password. This significantly reduces the time and computational resources required to crack the password.

Solutions to overcome the drawbacks of Hashing

1. Using Salt during the password hashing

- To prevent Rainbow table attacks, Salts are commonly used.
- Salts are random values that're unique for each user & are made public & generated at the time of user registration. The salt value is stored as part of resulting hash itself. Specifically, the salt is included in the hash string along with the actual hashed password.
- This salt can be stored as plain text value or encode it or encrypt it.
- The salt is typically stored/concatenated at the beginning of the hash string & is used during the password verification process to generate the same hash again for comparison.
- Since each user has a unique hash function, attackers can't precompute large table of passwords (rainbow tables) to test against the entire database of stolen password hashes.

Plain text password + Random salt value + Hashing Algo = Protection from Rainbow attacks

- Salt is not going to protect us from Brute force attack.

How brute force attack will be avoided?

Ans: Let's understand when the brute force attack is going to happen – 2 such scenarios can be:

Case 1: The hacker doesn't have original hash value of the end user & he just wants to hack someone's account. He will keep on trying some random passwords via UI application or whatever mechanism the app has exposed.

- In this scenario, the salt is not going to protect us.
- Becoz during the login operation, the backend API is going to derive the salt value from the value that we've stored in DB & append to the value entered by the hacker inside the UI application.
- For e.g., Hacker guessed the password as 12345 for user1 & entered the password on UI then behind the scene, the BE is going to leverage the salt value & append to this user-entered value & it's going to generate the overall hash value & hash is going to match.
- Here the salt is not protecting from brute force.

To avoid this kind of scenario, the organization is going to use various approaches:

- As soon as app detect someone enter 3 consecutive times wrong password, they will lock the account. With this hacker is going to have just 3 attempts. So, we're trying to limit the hacker's option here.
- Website will not accept the simple passwords like 12345 rather they will add criteria like at least 1 uppercase, 1 lowercase, min 8 digits, 1 special character etc. Due to this, the end user will end up picking up some strong password.

Case 2: Suppose hacker steals all the data present inside our organization DB. He has all the hash values of all the users along with salt i.e., salt + total hash value.

- In this scenario, hacker has to guess user original password & append it to salt value & check for the match with total hash value.
- This operation he has to perform for millions of guesses & it's not going to be problem for him as hash function is super-fast.

To overcome this challenge, we need to make the hash functions intentionally slow.

2. Using Slow hashing algorithms to overcome brute force

- Brute force attacks issue can be solved with password hashes, which are designed to be slow.
- The normal hashing algorithm/process is quite fast, due to which attackers can do the trial & error by trying any number of plain passwords to check against hashed password.
- We can make attackers life tough by using password hashing algorithms like PBKDF2, bcrypt, scrypt & Argon2. These algorithms slow the hashing process & demands lots of CPU cost, memory cost.
- Based on the business requirement, we can slow the algorithm by 2 or 3 .. seconds.

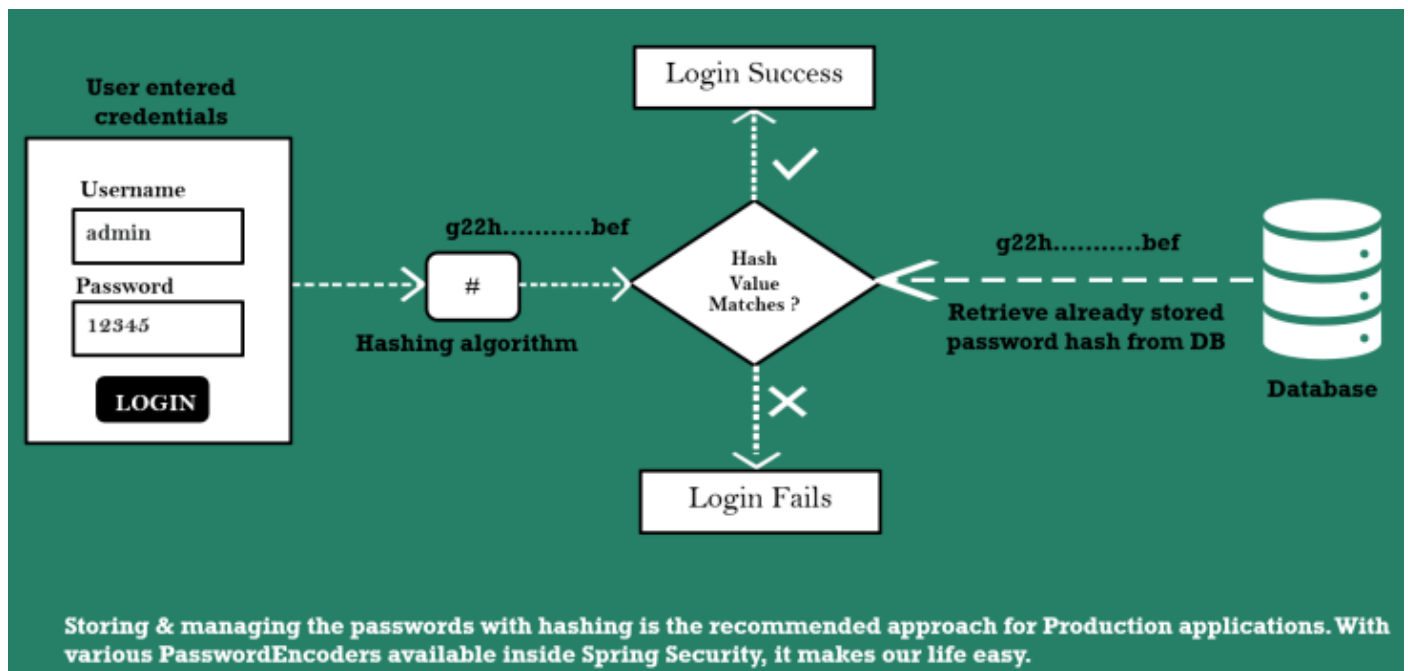
These algorithms are specifically designed to slow down the hashing process, thereby increasing the time and resources required for an attacker to guess passwords. They achieve this by introducing additional computational costs, such as CPU and memory, into the hashing process. By making it more computationally expensive to hash passwords, these algorithms make brute force attacks less practical and deter attackers from attempting them.

Spring Security provides industry recommended Password Encoders that are capable of generating random salt and leverage password hashing algorithms like bcrypt

Q. If we slow the hashing algorithm, won't it affect the organization itself?

- No, it's not as a user is going to register or login not too many times & he can easily wait for 1 or 2 seconds.

How Password validated with Hashing & PasswordEncoders



Password Encoders in Spring Security

```
public interface PasswordEncoder {

    String encode(CharSequence rawPassword);

    boolean matches(CharSequence rawPassword, String encodedPassword);

    default boolean upgradeEncoding(String encodedPassword) {
        return false;
    }
}
```

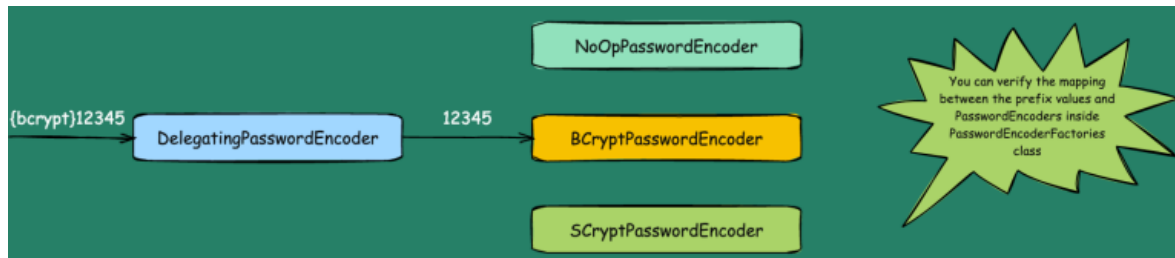
PasswordEncoder Interface implementations (1 – 6 in strength & complexity order)

1. NoOpPasswordEncoder (Not recommended for Prod apps)
2. StandardPasswordEncoder (Not recommended for Prod apps)
3. Pbkdf2PasswordEncoder
4. BCryptPasswordEncoder
5. SCryptPasswordEncoder
6. Argon2PasswordEncoder

Note: We can debug each of these encoders of understand the internal functionality. If we try to use the SCrypt or Argon2 Password Encoders, the configuration is going to be more tough becoz we need to right-tune the CPU cost, Memory cost, Parallelization cost so, we need to find the balance or sweet spot which is going to work for us.

Multiple Encoding Strategies with Delegating Password Encoder (PasswordEncoderFactories)

- When needing to switch password hashing algorithms in an application, such as due to a vulnerability, the DelegatingPasswordEncoder is handy.
- It allows managing multiple hash types simultaneously. For instance, it enables a new algorithm for new users while maintaining the old one for existing credentials.
- The DelegatingPasswordEncoder delegates to the correct implementation of the PasswordEncoder based on the prefix of the password.



- During the user registration process, the DelegatingPasswordEncoder uses the default password encoder defined in the framework & appends the corresponding prefix to the encoded password.
- During the login operation, when we call the matches () method with a password that has prefix {bcrypt}, the call is delegated to the BCryptPasswordEncoder class.
- Similarly, based on the other prefix values, the corresponding password encoder will be selected.

Conclusion about Encoding, Encryption & Hashing

- For password, always go with the Hashing
- Symmetric encryption we can use in the scenarios where we want to encrypt the data in rest.
- Asymmetric encryption we can use in the scenarios where we want to encrypt the data in transit.
- BASIC 64, we just need to use in the scenarios where we can't deal with the binary data.

Understanding Authentication Provider & implementing it

- The AuthenticationProvider in Spring Security takes care of the Authentication logic.
- The default implementation of the AuthenticationProvider is to delegate the responsibility of finding the user in the system to a UserDetailsServiceImpl implementation & PasswordEncoder for password validation.
- But in case of below custom authentication requirement that is not fulfilled by Spring Security framework, then we can build our own authentication logic by implementing the AuthenticationProvider interface.

Imp. Point: It is the responsibility of the ProviderManager (an implementation of AuthenticationManager) to check all the implementation of AuthenticationProvider & try to authenticate the user based on the Authentication object type.

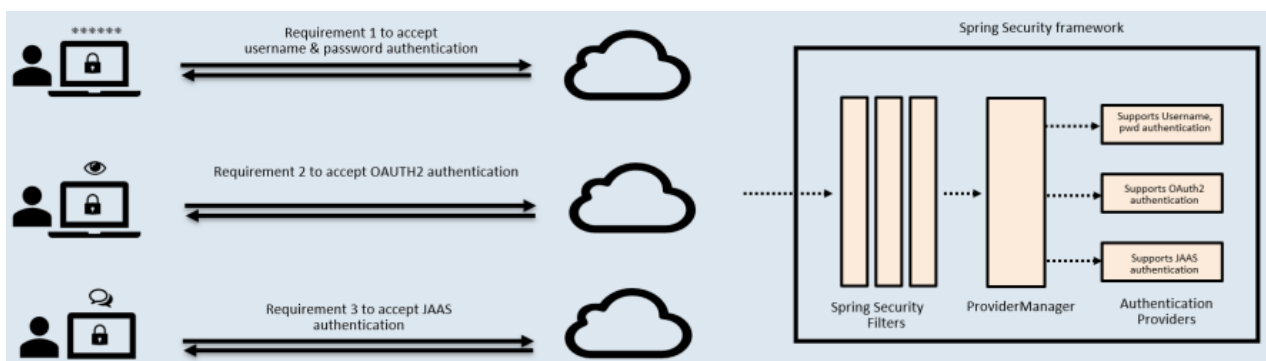
Why should we consider creating our own AuthenticationProvider (inbuilt is DaoAuthenticationProvider)

- With the help of already inbuilt Authentication Provider i.e., DaoAuthenticationProvider, we're able to perform the authentication of the end user by loading the user details from DB with our own UserDetailsServiceImpl implementation.
- As of now, we're building small applications but in real projects/enterprise applications, the security requirement may not look very simple.

Scenario 1: We have a scenario where an enterprise application wants to provide multiple style of Authentication i.e., multiple Authentication Provider to allow the end user to authenticate via:

1. **Authentication Provider 1** - Username & password authentication for certain application
2. **Authentication Provider 2** - OAuth2 authentication
3. **Authentication Provider 3** - JAAS authentication (Java Authentication & Authorization Service)

This scenario has a lot of complex security requirement.



- For all the complex security scenarios, we need to write our own Authentication Provider implementations.

Scenario 2: By default, implementation DaoAuthenticationProvider is taking help from UserDetailsServiceImpl implementation to the user service details.

Once the user details are loaded from the storage system, the actual authentication of password comparison & the decision of Authentication is going to happen inside the AuthenticationProvider but there will some scenarios where we can have our own custom Authentication requirements like:

1. To allow the authentication of a user only if the user's age is greater than 18.
 2. To allow the authentication only from certain countries.
- For this kind of custom requirement, DaoAuthenticationProvider is not going to help becoz whatever logic that we're going to write inside the UserDetailsServiceImpl implementation will load the user details from storage system but not help us to validate the age details or country details.
 - To accommodate such requirements, we have to define own AuthenticationProvider implementation

Understanding AuthenticationProvider methods

```
public interface AuthenticationProvider {

    /** Performs authentication with the same contract as ...*/
    Authentication authenticate(Authentication authentication) throws AuthenticationException;

    /** Returns <code>true</code> if this <Code>AuthenticationProvider</code> supports the ...*/
    boolean supports(Class<?> authentication);

}
```

1. authenticate method

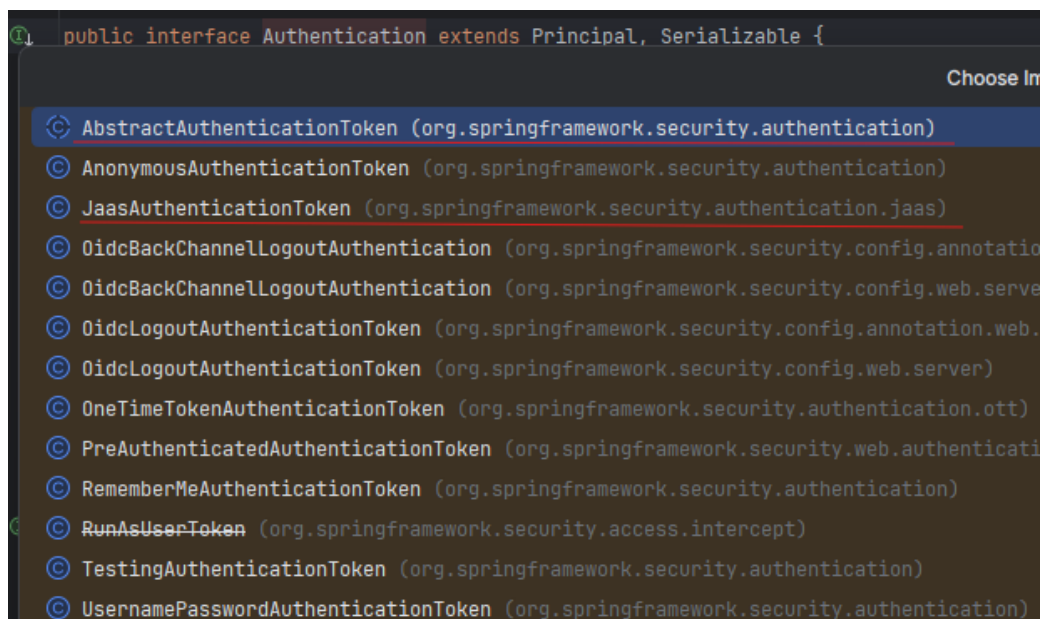
- Inside this method, the authentication logic is defined.
- The ProviderManager is going to invoke the authenticate () method with the Authentication object. The Authentication object has username, password, or any other details along with the authenticated boolean set to false.
- So, when we define our own custom AuthenticationProvider, we need to make sure we're loading the user details from a storage system based on the authentication details that we receive & once we perform the credential matching, we can perform any other extra validation logic & at last, Authentication object will be returned indicating whether the authentication is successful or not.
- We can debug & check the above logic in AbstractUserDetailsAuthenticationProvider, DaoAuthenticationProvider

2. supports method

- Whenever we're defining our own custom AuthenticationProvider, we need to tell the framework what type of authentication that our AuthenticationProvider is going to support.
- We can check DaoAuthenticationProvider for reference. It is going to support UsernamePasswordAuthenticationToken style of authentication.
- This means when our ProviderManager receives the Authentication object of type UsernamePasswordAuthenticationToken (implementation of Authentication object) then the ProviderManager is going to invoke the DaoAuthenticationProvider for the actual implementation.

```
@Override
public boolean supports(Class<?> authentication) {
    return (UsernamePasswordAuthenticationToken.class.isAssignableFrom(authentication));
}
```

- Above kind of similar logic, we need to define under the supports () method, indicating what style of Authentication (refer this interface & its implementation) that our AuthenticationProvider is going to support.



```
public interface Authentication extends Principal, Serializable {

    Choose In

    AbstractAuthenticationToken (org.springframework.security.authentication)
    AnonymousAuthenticationToken (org.springframework.security.authentication)
    JaasAuthenticationToken (org.springframework.security.authentication.jaas)
    OidcBackChannelLogoutAuthentication (org.springframework.security.config.annotation)
    OidcBackChannelLogoutAuthentication (org.springframework.security.config.web.server)
    OidcLogoutAuthenticationToken (org.springframework.security.config.annotation.web)
    OidcLogoutAuthenticationToken (org.springframework.security.config.web.server)
    OneTimeTokenAuthenticationToken (org.springframework.security.authentication.ott)
    PreAuthenticatedAuthenticationToken (org.springframework.security.web.authentication)
    RememberMeAuthenticationToken (org.springframework.security.authentication)
    RunAsUserToken (org.springframework.security.access.intercept)
    TestingAuthenticationToken (org.springframework.security.authentication)
    UsernamePasswordAuthenticationToken (org.springframework.security.authentication)
```

UsernamePasswordAuthenticationToken

- Whenever we're trying to use login page / HTTP basic style of authentication to send the credentials, Spring Security behind the scene will create the object of UsernamePasswordAuthenticationToken.
- Same token will be forwarded to ProviderManager to DaoAuthenticationProvider.
- DaoAuthenticationProvider is the only AuthenticationProvider supporting Username, Password style of Authentication.

Oidc Token

- Oidc token is the token which is going to be used in the scenario of OAuth2.0 authentication

Jaas Token

- For applications supporting Jaas style.

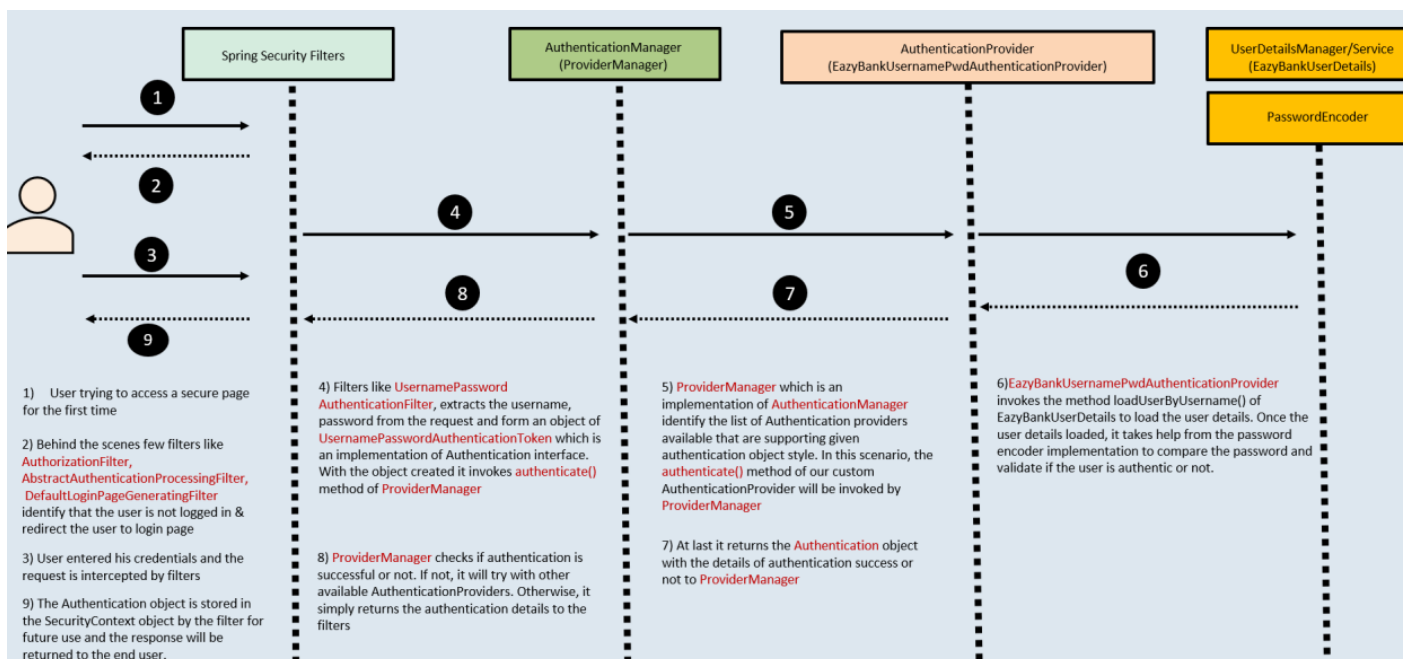
AnonymousAuthenticationToken

- This token represents an end user who is anonymous i.e., user is trying to access the application without any security.
- For e.g., a public page or a public API

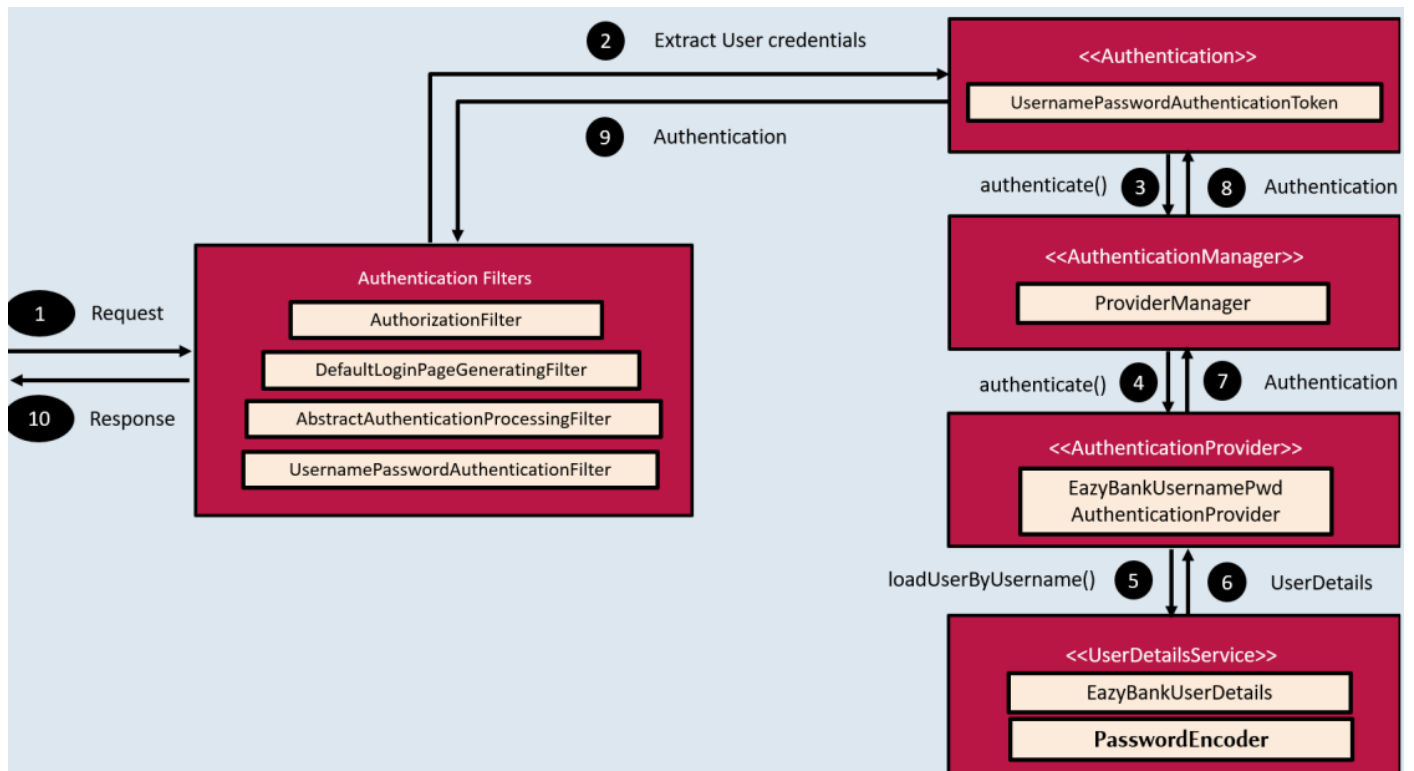
TestingAuthenticationToken

- This token object we can try to create in the scenario of Unit testing.
- If an application has unit testing, we don't have to actually authenticate the end user every time for each & every scenario. To avoid the actual authentication scenarios, we can mark the entire authentication flow with the help of TestingAuthenticationToken

Sequence Flow with our own AuthenticationProvider implementation



Sequence Flow with our own AuthenticationProvider implementation



Environment Specific Security Configuration

- We can conditionally authenticate the end user based upon the environment where authentication is happening using the spring boot concept of Profiles.
- Using Profiles concept, we can conditionally load the properties, beans, create the beans.

Way 1: (Not Recommended) Activating specific profile by hardcoding the value inside the application.properties.

```
application.properties
1 spring.application.name=${SPRING_APP_NAME:springSection6}
2
3 # import application_prod.properties
4 spring.config.import=application_prod.properties
5 # profiles can be default, prod
6 spring.profiles.active=prod
7

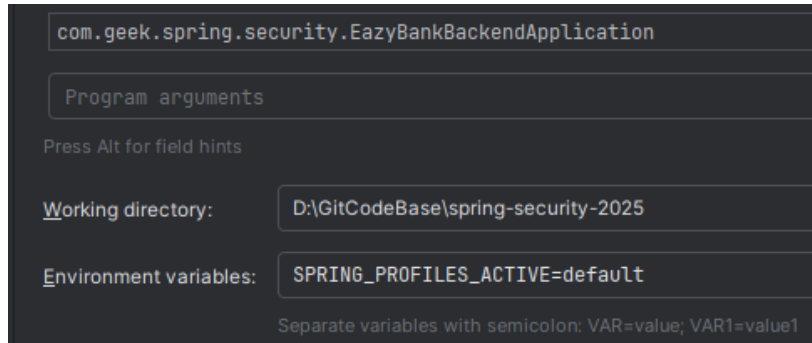
application_prod.properties
1 spring.config.activate.on-profile=prod
2
3 spring.application.name=${SPRING_APP_NAME:springSection6}
4
```

Disadvantage of Way 1

- Every time we need to change the profile, we have to manually update application.properties & build, generate jar & then deploy the same.

Way 2: Spring Boot allows to override properties from outside the application.

- i.e., we can have the same application jar or build file. The behaviour can be changed from outside by activating the profile.
- Using the environment variable, we can change the profile.
- The operation team can always create the environment variables & using the same they can always set specific profile values. Based upon the value present inside the environment variable, the spring boot will give priority to the same & it will not give priority to the values present inside the application.properties file.



We can activate Spring Security configuration or beans conditionally using Spring Boot profiles concept. Below is the sample snippet on how we can activate a Bean or configuration based on a profile,

```
@Configuration
@Profile("!prod")
public class ProjectSecurityConfig {

}
```

```
@Configuration
@Profile("prod")
public class ProjectSecurityProdConfig {

}
```

```
@Component
@Profile("!prod")
public class EazyBankUsernamePwdAuthenticationProvider
    implements AuthenticationProvider {

}
```

```
@Component
@Profile("prod")
public class EazyBankProdUsernamePwdAuthenticationProvider
    implements AuthenticationProvider {

}
```


Spring Security Customizations for most common use cases

Accepting only HTTPS Traffic using Spring Security

- Any person who is trying to use a website or mobile application will only be comfortable if app accepts & process the request using the HTTPS protocol.
- With the help of HTTPS, every data that end user sends to the backend application will be properly secured.
- The end user can submit credit card details as well confidently.
- So, if you build a production app which is only accepting plain HTTP requests, then obviously no one is going to use your website.
- That's why most of the organization will make sure that their Backend application accepts only the HTTPS protocol but not HTTP protocol.
- This can be achieved easily with the help of Spring Security Framework.

Note:

- By default, if we're not making any HTTPS related configurations, then the Spring Security will accept both kinds of traffic (HTTP & HTTPS)
- We can use below config to allow only HTTPS protocol requests.

```
@Bean
no usages  Shivam-Omen *
SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {
    http.redirectToHttps( HttpsRedirectConfigurer<HttpSecurity> https -> https.requestMatchers(AnyRequestMatcher.INSTANCE))
        .csrf(AbstractHttpConfigurer::disable)
        .authorizeHttpRequests( AuthorizationManagerRequestMat... requests -> requests
            .requestMatchers( ...patterns: "/myAccount", "/myBalance", "/myLoans", "/myCards").authenticated()
            .requestMatchers( ...patterns: "/notices", "/contact", "/error", "/register").permitAll()
        )
        .formLogin(withDefaults())
        .httpBasic(withDefaults());
    return http.build();
}
```

Enabling HTTPS in your application is crucial for safeguarding data confidentiality and integrity during communication between clients and servers. By encrypting the data transmitted, HTTPS prevents eavesdropping and man-in-the-middle attacks. Spring Security offers effortless integration to ensure only HTTPS traffic is accepted in your application.

Below are the Spring Security configurations to accept HTTPS traffic only,

```
@Bean
SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {
    http.requiresChannel(rcf -> rcf.anyRequest().requiresSecure())
        .authorizeHttpRequests((requests) -> requests
            .requestMatchers("/myAccount", "/myBalance", "/myLoans", "/myCards").authenticated()
            .requestMatchers("/notices", "/contact", "/error").permitAll());
    return http.build();
}
```

If you are looking for an option to accept only HTTP traffic, then you can invoke `requiresInsecure()` Method instead of `requiresSecure()` method

Exception Handling in Spring Security Framework

- If a client attempts to access a protected resource without authentication or proper authorization, they should be prompted to provide credentials to access the resource.
- This can be achieved by either redirecting the client to a login page in the UI or by informing them about the situation through the API Response (Custom Response instead of inbuilt one)
- Inside Spring Security authentication & authorization related exceptions are handled by **ExceptionTranslationFilter**

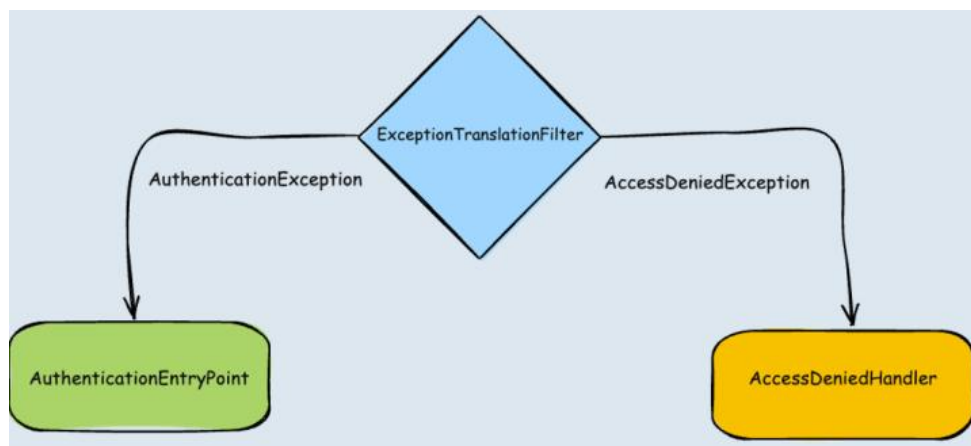
In general, Spring Security framework doesn't worry about all kind of business exceptions or runtime exceptions raised in our application. It is only going to worry & handle Security related exceptions. Inside the Security framework, there are 2 types of exceptions that may happen:

1. AuthenticationException

- Result in status as 401.
- 401 indicates that the person/client application is not authenticated.
- **Extended exception classes** – BadCredentialsException, UsernameNotFoundException etc.

2. AccessDeniedException

- Result in status as 403 (Forbidden).
- 403 indicates that the person/client application is properly authenticated but they don't have enough privileges or roles to access a secured API.
- **Extended exception classes** – AuthorizationDeniedException, CsrfException etc.



Spring Security is going to worry on how to handle only these 2 types of exceptions. Inside Spring Security we've a filter which is going to monitor these 2 exceptions & this filter name is **ExceptionTranslationFilter**

The responsibility of **ExceptionTranslationFilter** is to check what is the type of exception raised.

Case 1: If it's related to the AuthenticationException, it's going to invoke the **AuthenticationEntryPoint** related implementation.

Case 2: If it's related to the AccessDeniedException, it's going to invoke the **AccessDeniedHandler** related implementation.

```
ExceptionTranslationFilter.java x
83 public class ExceptionTranslationFilter extends GenericFilterBean implements MessageSourceAware {
171 @ private void handleSpringSecurityException(HttpServletRequest request, HttpServletResponse response,
172         FilterChain chain, RuntimeException exception) throws IOException, ServletException {
173     if (exception instanceof AuthenticationException) {
174         handleAuthenticationException(request, response, chain, (AuthenticationException) exception);
175     }
176     else if (exception instanceof AccessDeniedException) {
177         handleAccessDeniedException(request, response, chain, (AccessDeniedException) exception);
178     }
179 }
```

AuthenticationEntryPoint

- AuthenticationEntryPoint is an interface inside the Spring Security framework, which is responsible to handle all Authentication related exceptions.
- AuthenticationEntryPoint serves as a handler for unauthenticated access attempts, determining how to respond when someone tries to access a part of our side or API without the authentication.
- It might redirect users to a login page or return an error message indicating that authentication is necessary.

```
public interface AuthenticationEntryPoint {  
    void commence(HttpServletRequest request, HttpServletResponse response,  
        AuthenticationException authException)  
        throws IOException, ServletException;  
}
```

- **LoginUrlAuthenticationEntryPoint**, **BasicAuthenticationEntryPoint** are the few of the Spring Security provider implementations of AuthenticationEntryPoint interface that gets executed by default in case of AuthenticationException.

Defining Custom AuthenticationEntryPoint inside Spring Security Framework

Step 1: We need to create an implementation class using AuthenticationEntryPoint interface & by overriding the commence () method. Inside the commence () method, we can build logic on what response to send back to clients based on our requirements,

```
public class CustomBasicAuthenticationEntryPoint implements AuthenticationEntryPoint {  
    @Override  
    public void commence(HttpServletRequest request, HttpServletResponse response,  
        AuthenticationException authException) throws IOException {  
        response.setHeader("eazybank-error-reason", "Authentication failed");  
        response.sendError(HttpStatus.UNAUTHORIZED.value(), authException.getMessage());  
    }  
}
```

Step 2: The **CustomBasicAuthenticationEntryPoint** created in the above step can be configured for the httpBasic flow like shown below,

```
http.httpBasic(hbc -> hbc.authenticationEntryPoint(  
    new CustomBasicAuthenticationEntryPoint()));
```

- We can also configure custom AuthenticationEntryPoint globally that is common for all login types using exceptionHandling configurations.

```
http.exceptionHandling(ehc -> ehc.authenticationEntryPoint(  
    new CustomBasicAuthenticationEntryPoint())); // It is an Global Config
```

Note:

The authenticationEntryPoint() we're using for httpBasic configuration is not applicable for formLogin as during the form login, we're going to deal with UI pages but not with Json response, the UI end user doesn't bother about Response Header, Response Body, that's why whenever we're trying to use the form login, we have other types of config we can use like login page etc.

AccessDeniedHandler

- AccessDeniedHandler interface serves as handler for AccessDeniedExceptions

```
public interface AccessDeniedHandler {  
  
    void handle(HttpServletRequest request, HttpServletResponse response,  
                AccessDeniedException accessDeniedException)  
        throws IOException, ServletException;  
  
}
```

- AccessDeniedHandlerImpl is the most commonly used handler by the Spring Security framework.

Define Custom AccessDeniedHandler inside the Spring Security framework

Step 1: We need to create an implementation class using AccessDeniedHandler i/f & by overriding the handle () method. Inside the handle () method, we can build logic on what response to send back to clients based on our requirements.

```
public class CustomAccessDeniedHandler implements AccessDeniedHandler {  
  
    @Override  
    public void handle(HttpServletRequest request, HttpServletResponse response,  
                      AccessDeniedException accessDeniedException)  
        throws IOException, ServletException {  
        response.setHeader("eazybank-denied-reason", "Authorization failed");  
        response.setStatus(HttpStatus.FORBIDDEN.value());  
    }  
  
}
```

Step 2: The CustomAccessDeniedHandler created in the above step can be configured using exceptionHandling ()

```
http.exceptionHandling(ehc -> ehc.accessDeniedHandler  
    (new CustomAccessDeniedHandler()));
```

Note:

Getting JSON response may not be an ideal response for an UI application. So, your application is strictly an UI based application, then there is an option available that we can explore.

```
// Access Denied handler config is done at global level only  
http.exceptionHandling( ExceptionHandlingConfigurer<HttpSecurity> ehc -> ehc.accessDeniedHandler(new CustomAccessDeniedHandler())  
    .accessDeniedPage( accessDeniedUrl: "/denied"));
```

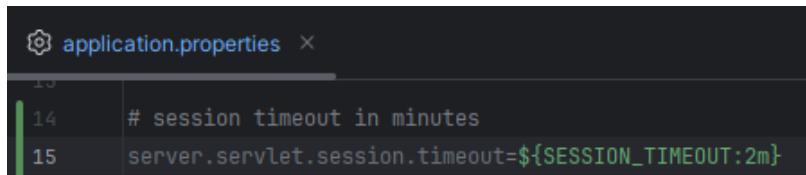
By using above config, whenever in our application if there is 403 error, the end user will be redirected to the MVC path “/denied”

Session Timeout & Invalid Session configurations

- Session created post login will have default timeout of 30 minutes i.e., the JSESSIONID created by Spring Security framework, by default it's going to have 30 minutes as a session timeout.
- Post 30 minutes, the end user will be redirected to the login page.
- We can customize this timeout based on our requirement & criticality of the application.

Note: Spring boot never allows a timeout less than 2 minutes.

- Once the session expired, the end user will be redirected to the login page. This behavior may not be user friendly becoz the end user will be clueless on why the login page is being displayed suddenly.
- To handle the invalid sessions/tampered session id properly, we can redirect the end user to a different URL or page with session timeout details.



```
13
14 # session timeout in minutes
15 server.servlet.session.timeout=${SESSION_TIMEOUT:2m}
```

Handling Invalid Session

Using the below config, the user can be redirected to the given URL when an invalid session is detected,

```
http.sessionManagement(sessionConfig ->
    sessionConfig.invalidSessionUrl("/invalidSession"))
```

Concurrent Session Control Configuration

- By default, the Spring Security framework is not going to enforce any constraints or any control on how many concurrent sessions an end user can have. The end user can have any no. of sessions by default.
- But in real application, we may want to restrict the end user to have only a single or may be 2 or 3 concurrent sessions depending upon the business requirements.

We can control the maximum sessions allowed for a user and what should happen in the case of invalid session due to too many sessions for the current user,

```
http.sessionManagement(sessionConfig ->
    sessionConfig.maximumSessions(1)
        .maxSessionsPreventsLogin(true)
        .expiredUrl("/expiredSession"))
```


Session Hijacking Attack

- In this attack, the attacker steals the session ids for a web application by eavesdropping on the communication b/w a user & an application or by gaining access to the user's computer or web browser data.
- Mostly this happens in 2 scenarios
 1. Session Id mentioned in the URL itself
 2. Session Id saved as a cookie in the browser
- In above 2 scenarios, if the hacker steals the session Id, then using the same, he can perform some action on the behalf of the user.
- For e.g., we've used a public computer in one to login somewhere then the session id associated with the login will be available inside the browser as a cookie or URL or hacker can steal the session id while our request is traveling through the network.
- To avoid session hijacking attack, there are multiple approaches:
 1. Use HTTPS protocol for communication (hackers can't steal the session id during the network traffic or while the request is traveling through the internet)
 2. Limit session id timeout for short time (10 min – 15 min) so that public login session will expire ASAP.
 3. Add checkbox (if using public computer) for end user during login operation. Accordingly, organization can be cautious while creating cookies.
- Imp. Point – In case of Session Hijacking, Spring Security directly doesn't play any role. Using above approaches we can secure our application against the same.

Session Fixation Attack

- Session Fixation attacks are a potential risk where it is possible for a malicious attacker to create a session by accessing a site, then persuade another user to log in with the same session (by sending them a link containing the session identifier as a parameter).
- Spring Security protects against this attack automatically by creating a new session or otherwise changing the session ID when a user logs in.
- One such scenario example: (Session Fixation Attack Flow)

Below are the steps that happens during the Session Fixation attack,

1) Emily often shops online at <http://fashionmart.com/> where she stores her payment information for quick purchases.

2) Eva, a hacker, discovers a vulnerability in <http://fashionmart.com/> that allows her to manipulate session identifiers. So Eva logged into the App with her credentials which will provide a valid session identifier like 34hjhj454h8efop43

3) Eva sends Emily an email pretending to be from the fashion website, saying: "Exciting news! We're offering exclusive discounts to our loyal customers. Click here to access your personalized discount: [http://fashionmart.com/?session= 34hjhj454h8efop43](http://fashionmart.com/?session=34hjhj454h8efop43)

4) Intrigued by the promise of discounts, Emily clicks the link and unknowingly logs into her shopping account with the session ID " 34hjhj454h8efop43 " provided by Eva.

5) With this fixed session ID, Eva gains unauthorized access to Emily's shopping account and can view her personal information, make purchases using her stored payment details, and potentially steal her identity.

- We have 3 options to control the strategy for Session fixation protection by following the below configuration:

changeSessionId: This option utilizes the session fixation protection provided by the Servlet container. It does not create a new session but instead changes the session ID. Note that this option is only available in Servlet 3.1 (Java EE 7) and newer containers.

newSession: With this option, a new "clean" session is created, without copying the existing session data. However, Spring Security-related attributes will still be copied.

migrateSession: This option creates a new session and copies all existing session attributes to the new session. It is the default in Servlet 3.0 or older containers.

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http) {
    http
        .sessionManagement((session) -> session
            .sessionFixation((sessionFixation) -> sessionFixation
                .newSession()
            )
        );
    return http.build();
}
```

We can also set the session fixation protection to **none** to disable it, but this is not recommended as it leaves your application vulnerable.

Authentication Events

- For each authentication that succeeds or fails, a **AuthenticationSuccessEvent** or **AbstractAuthenticationFailureEvent** respectively is fired by Spring Security framework.
- Spring Security provided **DefaultAuthenticationEventPublisher** takes care of publishing these events to the listeners.
- We as developers just need to build listeners to listen to these events & execute business logic based on our requirement.
- By using Spring's **@EventListener** support, we can listen to the Authentication success event & failure event.

```
@Component
@Slf4j
public class AuthenticationEvents {

    @EventListener
    public void onSuccess(AuthenticationSuccessEvent success) {
        log.info("Login successful for the user : {}",
            success.getAuthentication().getName());
    }

    @EventListener
    public void onFailure(AbstractAuthenticationFailureEvent failures) {
        log.error("Login failed for the user : {} due to : {}",
            failures.getAuthentication().getName(),
            failures.getException().getMessage());
    }
}
```

Use cases:

Many time inside real applications, we may have requirements to handle authentication event. For e.g., on successful or failed authentication, we may want to send an email to the end user. This may be required for super critical application.

Form Login Configurations for MVC or Monolithic apps

- For form-based logins, Spring Security already provides a simple login page. But this behavior can be changed with the below configurations:

```
.formLogin(formLoginConfig -> formLoginConfig
    .loginPage("/login")
    .usernameParameter("userid")
    .passwordParameter("secretPwd")
    .defaultSuccessUrl("/dashboard")
    .failureUrl("/login?error=true").permitAll())
```

loginPage -> Specifies the URL to send users to if login is required. The default is /login

usernameParameter - The HTTP parameter to look for the username when performing authentication. Default is "username"

passwordParameter - The HTTP parameter to look for the password when performing authentication. Default is "password"

defaultSuccessUrl - Specifies where users will be redirected after authenticating successfully if they have not visited a secured page prior to authenticating. This is a shortcut for calling `successHandler(AuthenticationSuccessHandler)`.

failureUrl - The URL to send users if authentication fails. This is a shortcut for invoking `failureHandler(AuthenticationFailureHandler)`. The default is "/login?error".

- If we want more control on the successful authentication & failure authentication of form login, we can configure the `successHandler` & `failureHandler` like below:

```
.formLogin(formLoginConfig -> formLoginConfig
    .loginPage("/login")
    .usernameParameter("userid")
    .passwordParameter("secretPwd")
    .successHandler(authenticationSuccessHandler)
    .failureHandler(authenticationFailureHandler))
```

Here the `authenticationSuccessHandler` and `authenticationFailureHandler` are the beans of the implementation classes of `AuthenticationSuccessHandler` and `AuthenticationFailureHandler` interfaces

```
public class CustomAuthenticationFailureHandler implements
    AuthenticationFailureHandler {

    @Override
    public void onAuthenticationFailure(HttpServletRequest request,
        HttpServletResponse response,
        AuthenticationException exception) throws IOException {
        ....
    }
}
```

```
public class CustomAuthenticationSuccessHandler implements
    AuthenticationSuccessHandler {

    @Override
    public void onAuthenticationSuccess(HttpServletRequest request,
        HttpServletResponse response,
        Authentication authentication) throws IOException {
        ...
    }
}
```

Logout Configuration for MVC or Monolithic apps

With the below configurations, we can configure what should happen during the logout operation of the user,

```
.logout(logoutConfigurer -> logoutConfigurer
        .logoutSuccessUrl("/login?logout=true").permitAll()
        .invalidateHttpSession(true)
        .clearAuthentication(true)
        .deleteCookies("JSESSIONID"))
```

logoutSuccessUrl -> The URL to redirect to after logout has occurred. The default is "/login?logout". This is a shortcut for invoking logoutSuccessHandler(LogoutSuccessHandler) with a SimpleUrlLogoutSuccessHandler.

invalidateHttpSession - Configures SecurityContextLogoutHandler to invalidate the HttpSession at the time of logout.

clearAuthentication - Specifies if SecurityContextLogoutHandler should clear the Authentication at the time of logout.

deleteCookies - Allows specifying the names of cookies to be removed on logout success. This is a shortcut to easily invoke addLogoutHandler(LogoutHandler) with a CookieClearingLogoutHandler.

Spring Security Thymeleaf Integration

We can get the currently authenticated user details in Thymeleaf pages by following the below steps,

First, we need to add the below dependency inside pom.xml:

```
<dependency>
    <groupId>org.thymeleaf.extras</groupId>
    <artifactId>thymeleaf-extras-springsecurity6</artifactId>
</dependency>
```

Next, we can load the username like shown below:

```
<html xmlns:th="https://www.thymeleaf.org"
      xmlns:sec="https://www.thymeleaf.org/thymeleaf-extras-springsecurity6">
<body>
    <div sec:authorize="isAuthenticated()">
        Authenticated as <span sec:authentication="name"></span></div>
</body>
</html>
```

Spring Security JSP Integration

We can get the currently authenticated user details in JSP pages by leveraging the Spring Security Taglib support.

First, we need to define the tag on the JSP page like shown below:

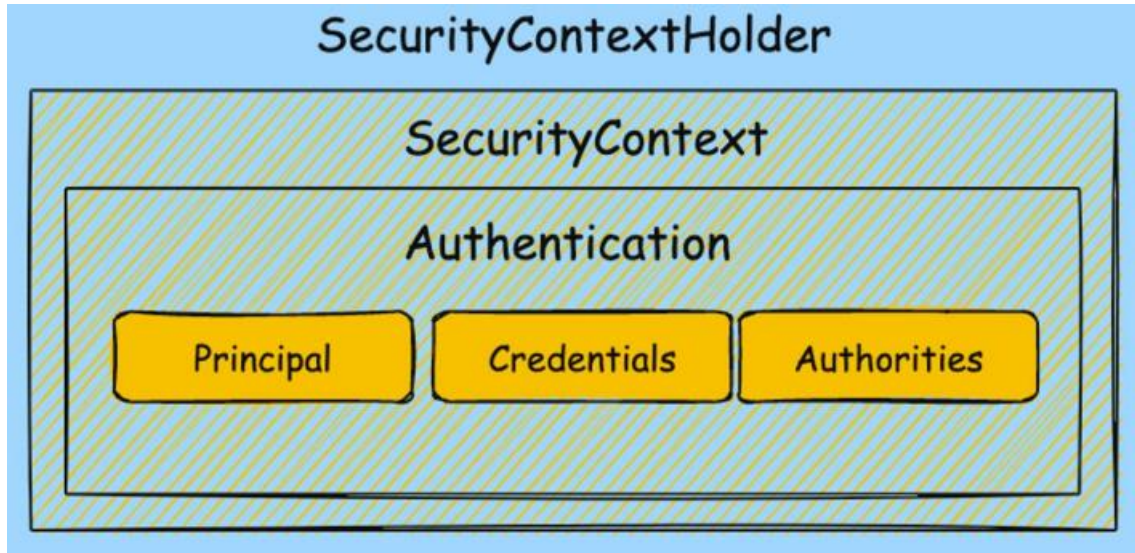
```
<%@ taglib prefix="security" uri="http://www.springframework.org/security/tags" %>
```

Next, we can load the username like shown below:

```
<security:authorize access="isAuthenticated()">
    authenticated as <security:authentication property="principal.username" />
</security:authorize>
```

Role of SecurityContext & SecurityContextHolder

- At the heart of Spring Security's authentication model is the SecurityContextHolder. It contains the SecurityContext.
- The SecurityContextHolder is where Spring Security stores the details of who is authenticated like Principal or username, Credentials like password & Authorities.
- Spring Security doesn't care how the SecurityContextHolder is populated. If it contains a value, it is used as the currently authenticated user.



- After the authentication process, we may need details about the authenticated entity, such as username or authorities of the current user. This information remains accessible even after authentication is complete.
- Once the **AuthenticationManager** successfully completes the authentication process, it stores the **Authentication** instance for the rest of the request. The component responsible for storing this object is called the **SecurityContext**.
- The security context is stored in a **SecurityContextHolder**.
- **SecurityContext** is an interface defining the minimum-security information associated with the current thread of execution.

```
public interface SecurityContext {
    Authentication getAuthentication();
    void setAuthentication(Authentication authentication);
}
```

- SecurityContextHolder is a helper class that provides access to the security context. By default, it uses a ThreadLocal object to store the security context, ensuring it is always available to methods in the same thread, even if you don't explicitly pass it around.
- We don't need to worry about ThreadLocal memory leaks in web applications, as Spring Security handles the cleanup.
- Additionally, SecurityContextHolder can be configured with different strategies for storing the security context.

Note:

- Once the authentication action is completed, the Spring Security framework is going to store the authentication details inside the object of SecurityContext. (SecurityContext is an interface with SecurityContextImpl as its implementation).
- This SecurityContext is going to be managed by a class "SecurityContextHolder". This holder class is responsible to manage the details inside the SecurityContext. It exposes various methods to clear the SecurityContext, add new details to the SecurityContext, update i.e., all the CRUD operation related to the SecurityContext along with other utility methods.

- Spring Security framework is going to leverage this holder class to a great extent whenever it's trying to store or deal with the SecurityContext object.
- We don't have to implement our own SecurityContext or SecurityContextHolder.

Holding Strategies for the Security Context

To specify which strategy should be used, you must provide a mode setting. Below are the three valid mode settings,

MODE_THREADLOCAL—Allows each thread to store its own details in the security context. In a thread-per-request web application, this is a common approach, as each request has an individual thread. This is the **default** mode

MODE_INHERITABLETHREADLOCAL—Similar to MODE_THREADLOCAL, this strategy also instructs Spring Security to copy the security context to the next thread in the case of an asynchronous method. This ensures that the new thread running the @Async method inherits the security context. The @Async annotation is used on methods to instruct Spring to execute the annotated method on a separate thread.

MODE_GLOBAL—Makes all the threads of the application see the same security context instance.

As shown in the following code snippet, you can change the strategy to MODE_GLOBAL. You can use the method SecurityContextHolder.setStrategyName() or the system property `spring.security.strategy`:

```
@Bean
public InitializingBean initializingBean() {
    return () -> SecurityContextHolder.setStrategyName(
        SecurityContextHolder.MODE_GLOBAL);
}
```



Load login user details in Spring Security

Approach 1 — The simplest way to load the currently authenticated user details is via a static call to the SecurityContextHolder:

```
@RestController
public class SomeController {

    @GetMapping(value = "/username")
    public String currentUser() {
        Authentication authentication = SecurityContextHolder.getContext().getAuthentication();
        String currentPrincipalName = authentication.getName();
        return currentPrincipalName;
    }
}
```

Approach 2 — We can mention the Authentication directly as a method argument, and the framework will correctly resolve it by extracting from the SecurityContextHolder. Instead of Authentication, we can also mention Principal.

```
@RestController
public class SomeController {

    @GetMapping(value = "/username")
    public String currentUser(Authentication authentication) {
        return authentication.getName();
    }
}
```


Section 8: Understanding CORs & CSRF

Setting up the EazyBank UI Project

1. Download & install node.js from <https://nodejs.org/en/download/> as per your OS & check if the installation is successful using the command 'node -v' which will print the current installed version.
2. Run the command 'npm install -g @angular/cli' to install Angular CLI
3. Download the visual code using the link <https://code.visualstudio.com/> and import the angular provided.
4. Navigate to the folder inside the project where package.json is present and run the command 'npm install' by opening a terminal. This will install all the dependencies inside the new folder created with the name 'node_modules'
5. Run the command 'ng serve' & it will start the application at <http://localhost:4200/> by default.

Now we can run the UI & BE application.

Name	Status	Type	Initiator	Size	Time
notices	CORS error	xhr	notices.component.ts:16	0.0 kB	14 ms
notices	403	preflight	Preflight	0.0 kB	6 ms

Why our UI resource is not able to access the Backend resource?

Ans: CORS policy (A Specification from W3C implemented by most browsers)

Browser default behavior

- Firstly, browser will send a preflight request to the BE resource and in response the BE server will send the proper response header indicating all the accepted origin details
- Post Preflight success request, the actual request with the Request headers will be sent to the BE resource.
- In case Preflight Request fails, the actual request will fail with CORS error.

Cross – Origin Resource Sharing (CORS)

- CORS is a protocol that enables scripts running on a browser client to interact with resources from a different origin.
- For e.g., If a UI app wants to make an API call running on a different domain, it would be blocked from doing so by default due to CORS.
- So, CORS is not a security issue/attack rather the default protection provided by browsers to stop sharing the data/communication b/w different origins.

“Other origins” means the URL being accessed differs from the location that the JavaScript is running from, by having:

- A different scheme (HTTP or HTTPS)
- A different domain
- A different port



Note:

- Understanding Analogy: Indian mobile number receiving call from Indian number & London number.
- CORS policy only comes into pic when UI app & BE app are on different servers (not deployed on the same server/origin)

Solution to handle CORS

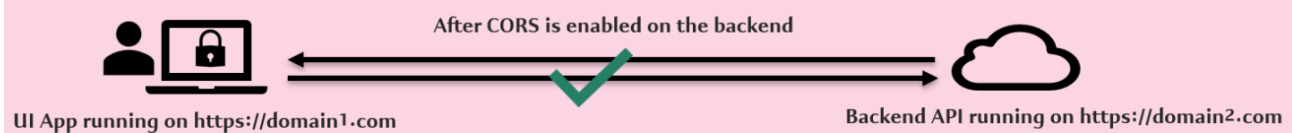
Way 1: Using @CrossOrigin annotation on class or method level **[Not Recommended]**

- If we've a valid scenario, where a web app UI deployed on a server is trying to communicate with a REST service deployed on another server, then this kind of communication we can allow with the help of @CrossOrigin annotation.
- @CrossOrigin allows clients from any domain to consume the API.

@CrossOrigin annotation can be mentioned on top of a class or method like mentioned below,

```
@CrossOrigin(origins = "http://localhost:4200") // Will allow on specified domain
```

```
@CrossOrigin(origins = "**") // Will allow any domain
```



Drawback of Way 1:

- In case we've 100 controllers in our project then we've to add @CrossOrigin annotation to each controller & in case in future it would be a tedious job to add new origin to each controller.
- This approach is not preferable.

Way 2: Defining CORS related configurations globally using Spring Security [Recommended Approach]

```
@Bean
SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {
    http.cors(corsCustomizer -> corsCustomizer.configurationSource(new CorsConfigurationSource() {
        @Override
        public CorsConfiguration getCorsConfiguration(HttpServletRequest request) {
            CorsConfiguration config = new CorsConfiguration();
            config.setAllowedOrigins(Collections.singletonList("http://localhost:4200"));
            config.setAllowedMethods(Collections.singletonList("*"));
            config.setAllowCredentials(true);
            config.setAllowedHeaders(Collections.singletonList("*"));
            config.setMaxAge(3600L);
            return config;
        }
    }));
    http.authorizeHttpRequests((requests) -> requests
```

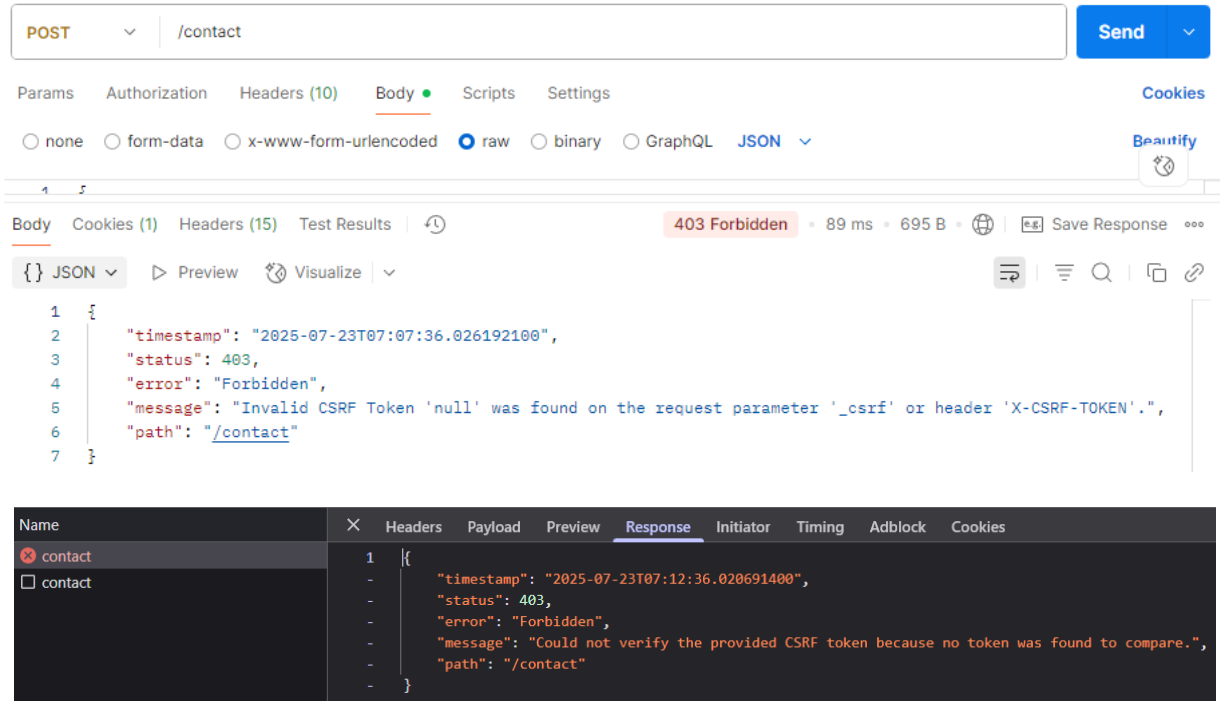
Name	×	Headers	Preview	Response	Initiator	Timing	Adblock
☒ styles.css	▼ General						
☐ notices	Request URL			http://localhost:8080/notices			
☒ notices	Request Method			OPTIONS			
A favicon.ico	Status Code			200 OK			
☐ fontawesome-webfont.woff2?v...	Remote Address			[::1]:8080			
	Referrer Policy			strict-origin-when-cross-origin			
	▼ Response Headers			☐ Raw			
	Access-Control-Allow-Credentials			true			
	Access-Control-Allow-Headers			x-requested-with			
	Access-Control-Allow-Methods			GET			
	Access-Control-Allow-Origin			http://localhost:4200			
	Access-Control-Max-Age			3600			

- Based on the configuration we've provided, all these CORS related details will be loaded inside the Response Header of Preflight Request with the help of **CorsFilter**.
- CORS consider localhost:3000 & 127.0.0.1:3000 as different origins.

CSRF Protection

Cross – Site Request Forgery (CSRF)

- Disabling the CSRF protection provided by Spring Security is not recommended for production applications. We need to properly implement a solution to handle the CSRF attacks.
- If you don't implement CSRF related solution inside your application, then by default Spring security is going to stop all the HTTP methods (altering or creating the data) like POST, PUT, DELETE. Only GET will be allowed.



- We're getting above error from both Postman & UI application becoz we're trying to create a new data with POST method but we're not passing any CSRF token as part of the Request.
- Whenever the CSRF token is missing, the Spring Security will assume that someone is trying to attack the application with CSRF attack. That's why it stops processing the request by throwing 403 statuses.

Introduction

- Forgery – If someone tries to exploit you without your knowledge we call such kind of activity as Forgery. For e.g., Someone got your cheque book & forge your signature.
- A typical Cross-Site Request Forgery (CSRF or XSRF) attack aims to perform an operation in a web application on behalf of a user without their explicit consent.
- In general, it doesn't directly steal the user's identity like credentials or JSession Id or Cookie instead it will trick/exploit the end user to carry out an action without their will.

- E.g., We're using a website netflix.com & the attack's website is evil.com

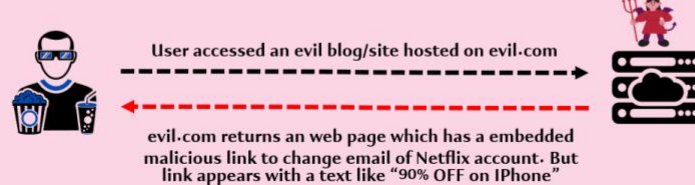
Step 1: As soon as the end user enters his credential, the netflix.com backend server is going to authenticate & post Authentication, the Backend server will provide a cookie against the domain name (netflix.com) that is going to be stored inside the browser.

Note: This cookie will be used by netflix.com for further actions (without asking for login again). The beauty of cookies are they're usually bound to a domain name (i.e., amazon.com will have different cookie than netflix.com). So, it's Browser's job to attach the cookie whenever a request is made to netflix.com

Step 1 : The Netflix user login to Netflix.com and the backend server of Netflix will provide a cookie which will store in the browser against the domain name Netflix.com



Step 2 : The same Netflix user opens an evil.com website in another tab of the browser.



Step 3: The problem here is netflix.com backend server is not able to differentiate whether the request is coming from evil.com or netflix.com becoz even the hacker is trying to send the request to the netflix.com only not to another domain.

Since this malicious form is getting invoked from the same browser (different tab), the browser is going to still attach the cookie of the netflix.com while it is making the request to BE of netflix.com

Step 3 : User tempted and clicked on the malicious link which makes a request to Netflix.com. And since the login cookie already present in the same browser and the request to change email is being made to the same domain Netflix.com, the backend server of Netflix.com can't differentiate from where the request came. So here the evil.com forged the request as if it is coming from a Netflix.com UI page.



```
<form action="https://netflix.com/changeEmail"
  method="POST" id="form">
  <input type="hidden" name="email" value="user@evil.com">
</form>

<script>
  document.getElementById('form').submit()
</script>
```

Note:

- Regardless whether the CORS is implemented or not, still the CSRF attack is possible in above scenarios.
- Becoz the malicious embedded form is maintained by the hacker inside evil.com & the advantage with embedded form is whenever someone is making a request to an action using an embedded form, the browser will think the request is originated from their domain (netflix.com) itself.
- netflix.com backend server never know that the embedded form is present inside another domain (i.e., evil.com)
- That's why CORS is not going to be helpful in this scenario.

Solution to CSRF Attack

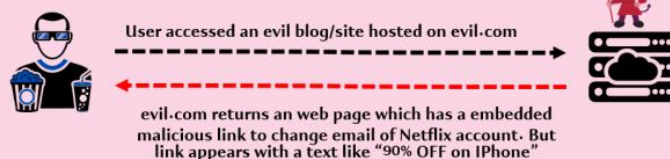
- To defeat a CSRF attack, applications need a way to determine if the HTTP request is legitimately generated via the application's user interface (i.e., netflix.com not from evil.com), the best way to achieve this is through a CSRF/XSRF token.
- CSRF token is a secure random token that is used to prevent CSRF attacks. The token needs to be unique per user session & should be of large random value to make it difficult to guess.

Step 1: Backend will generate CSRF/XSRF token along with JSESSION ID and attach & store them to the cookie inside browser.

Step 1 : The Netflix user login to Netflix.com and the backend server of Netflix will provide a cookie which will store in the browser against the domain name Netflix.com along with a randomly generated unique CSRF token for this particular user session.



Step 2 : The same Netflix user opens an evil.com website in another tab of the browser.



Step 3 : User tempted and clicked on the malicious link which makes a request to Netflix.com. And since the login cookie already present in the same browser and the request to change email is being made to the same domain Netflix.com. This time the Netflix.com backend server expects CSRF token along with the cookie. The CSRF token must be same as initial value generated during login operation



The CSRF token will be used by the application server to verify the legitimacy of the end-user request if it is coming from the same App UI or not. The application server rejects the request if the CSRF token fails to match the test.

Q. How the backend server is differentiating even though the hacker request has 2 cookies attached?

Ans:

- When the UI application is trying to make a POST/PUT/DELETE request, the Backend server is going to expect the CSRF/XSRF token values at 2 places
 1. CSRF token as part of Request (Header/Payload depending on UI/Backend server agreement)
 2. CSRF token as part of Cookie

1. CSRF token as part of the Request (Header/Payload)

- The hacker will not be able to send the CSRF token as the part of Request.
- In order to send the cookie value inside the Request Header/Body, the Client application have to manually read the cookie value using JavaScript. Once the cookie value is read by JS code, the CSRF token value will be populated to Header/Body in the same JS code.
- The Hacker won't be able to perform above operation from the JavaScript code inside evil.com becoz whenever JavaScript is being executed from a given domain, the JS code can read only the cookies related/belonging to the same domain.
- So though hacker can manage to build client-side code with JS to read cookie value inside the browser of the end user but this JS code will not work becoz the browser will stop evil.com JS code from reading the cookie belonging to netflix.com
- This way hacker would not be able to send cookie value inside the Request. With this, the backend server will easily identify that it got the token only as part of Cookie not as part of Request & with that it will throw 403.

2. CSRF token as part of the Cookie

- Sending the CSRF token as part of Cookie is taken care by the Browser automatically for the specific domain so by leveraging the same, the hacker will be able to send the token value inside the cookie.

Important interfaces/componet that handles CSRF attacks in Spring Security

1. CsrfToken interface

- org.springframework.security.web.csrf.CsrfToken
- Provides the information/contract about an expected CSRF token. Implementation classes:
 - a) DefaultCsrfToken – A CSRF token that is used to protect against CSRF attacks.

2. CsrfTokenRepository interface

- org.springframework.security.web.csrf.CsrfTokenRepository
- Provides the contract to create, store & load CSRF tokens. As a developer, we don't need to explicitly generate the CSRF token during the login operation. Implementation classes:
 - a) **HttpSessionCsrfTokenRepository** – Stores the CsrfToken in the HttpSession **[Not Recommended]**
 - b) **CookieCsrfTokenRepository** – Persists the CsrfToken in a cookie named "XSRF-TOKEN" & reads from the header "XSRF-TOKEN" following the conventions of Angular

3. CsrfTokenRequestHandler interface

- A callback interface that is used make the CsrfToken created by the CsrfTokenRepository available as a request attribute.
- Implementations of this interface may choose to perform additional tasks or customize how the token is made available to the application through request attributes.
- Implementation classes:
 - a) CsrfTokenRequestAttributeHandler – It is capable of making the CsrfToken available as a request attribute & resolving the token value as either a Header or Parameter value of the request.

4. CsrfFilter class

- Spring Security inbuilt filter responsible to validate the CSRF token received by client & to decide whether to throw an exception or not.
- All the validation logic is present inside CsrfFilter class doFilterInternal () method. We can debug & see.

5. CsrfCookieFilter (Custom Filter)

- Custom Filter written by Developers to send the generated CSRF token back to client side inside the response header.

Note:

- We will create CookieCsrfTokenRepository using CookieCsrfTokenRepository.withHttpOnlyFalse() method. WithHttpOnlyFalse () method will create the CookieCsrfTokenRepository instance & set a variable **cookieHttpOnly** as false.
- When we're setting cookieHttpOnly = false, we're letting the UI framework / JS code to read the cookie value. By default, all the cookies that're going to be generated will have cookieHttpOnly = true that signifies only browser can read & attach the cookie for each HTTP request but our JS/Angular code will not be able to read the cookie manually.

A `CsrfTokenRepository` that persists the CSRF token in a cookie named "XSRF-TOKEN" and reads from the header "X-XSRF-TOKEN" following the conventions of AngularJS. When using with AngularJS be sure to use `withHttpOnlyFalse()`.

Since: 4.1

Author: Rob Winch, Steve Riesenber, Alex Montoya

```
public final class CookieCsrfTokenRepository implements CsrfTokenRepository {
```

```
    public static CookieCsrfTokenRepository withHttpOnlyFalse() {  
        CookieCsrfTokenRepository result = new CookieCsrfTokenRepository();  
        result.cookieHttpOnly = false;  
        return result;  
    }
```

```
.csrf( CsrfConfigurer<HttpSecurity> csrfConfig -> csrfConfig.csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse()))
```

- Above config should be good to generate the CSRF token value.
- But whenever we're getting CsrfToken value using CookieCsrfTokenRepository class, behind the scene the token is going to be generated Lazily i.e., until unless someone is trying to read the token value manually, the token is never going to be generated & the cookie will also be not generated.
- This has been done intentionally by Spring Security framework to achieve some performance as not all the requests need the CSRF token & cookies. For e.g., GET APIs (unnecessarily loading & reading CSRF token is going to attract some performance issues)
- Since the token is getting generated lazily, the CSRF cookie we're expecting from the backend to UI will never be generated automatically. We need to write a piece of logic to read the token manually. Once the token is read manually, behind the scenes, Spring Security will take care of generating the token value, generating the cookie, & sending the same to the UI framework.
- In order to read the token value manually, we have to define a filter:

Whenever a filter is extending `OncePerRequestFilter`, the library will ensure that the filter is only executed once per request.

```
public class CsrfCookieFilter extends OncePerRequestFilter { 2 usages
```

```
    @Override
```

```
    protected void doFilterInternal(HttpServletRequest request, HttpServletResponse response, FilterChain filterChain)  
        throws ServletException, IOException {  
        CsrfToken csrf = (CsrfToken) request.getAttribute(CsrfToken.class.getName());  
        // Render the token value to a cookie by causing the deferred token to be loaded  
        csrf.getToken();  
        filterChain.doFilter(request, response);  
    }
```

```
.addFilterAfter(new CsrfCookieFilter(), BasicAuthenticationFilter.class)
```


- We also need to pass `CsrfTokenRequestHandler` to the security configuration so that csrf token from the Request Header/Body can be extracted. We can debug & see `CsrfTokenRequestAttributeHandler`, `CsrfFilter` class – `doFilterInternal ()` method.
- From now onwards, we're going to use a separate UI application (separate login page) to login the user & the credential will be sent to the backend server in the `HttpBasic` format.
- In above scenarios, Spring Security by default is not going to generate the `JSESSION ID`. We need to explicitly tell the Spring Security framework to generate the `JSESSION ID` so that all the subsequent request can be invoked without passing the credentials.
- We can configure above using `sessionManagement` method & passing `sessionConfig.sessionCreationPolicy()`:
 - a) `SessionCreationPolicy.ALWAYS` – Always create an `HttpSession` so that it can be reused again & again to access the secured APIs.
 - b) `SessionCreationPolicy.NEVER` – Spring Security will never create an `HttpSession` but will use the `HttpSession` if it already exists.
 - c) `SessionCreationPolicy.IF_REQUIRED` – Spring Security will only create an `HttpSession` if required
 - d) `SessionCreationPolicy.STATELESS` – Spring Security will never create an `HttpSession` & it will never use it to obtain the `SecurityContext`.
- Whatever `JSESSION ID` gets generated using the Security config session configurations i.e., using `sessionCreationPolicy`, will not be automatically stored inside the `SecurityContextHolder`.
- Either we need to manually store the `JSESSION ID` inside the `SecurityContextHolder` or make the configuration so that Spring Security framework can take care for us.

POST /contact Send

Params Authorization Headers (10) Body Scripts Settings Cookies

Key	Value	Description
Connection	keep-alive	
X-XSRF-TOKEN	ecf7dbf4-8c11-447d-80c9-cc4ee78...	

200 OK • 135 ms • 637 B • Save Response

Name	Value	Domain	Path	Expires	HttpOnly	Secure
JSESSIONID	2EFA50BA3D7B5C4244C8EB9A48887C10	localhost	/	Session	true	false
XSRF-TOKEN	ecf7dbf4-8c11-447d-80c9-cc4ee7862db1	localhost	/	Session	false	false

```
@Bean no usages Shivam-Omen *
SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {
    CsrfTokenRequestAttributeHandler csrfTokenRequestAttributeH = new CsrfTokenRequestAttributeHandler();
    http.securityContext( SecurityContextConfigurer<HttpSecurity> contextConfig ->
        contextConfig.requireExplicitSave(false))
        .sessionManagement( SessionManagementConfigurer<HttpSecurity> sessionConfig ->
            sessionConfig.sessionCreationPolicy(SessionCreationPolicy.ALWAYS))
        .csrf( CsrfConfigurer<HttpSecurity> csrfConfig ->
            csrfConfig.csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
            .csrfTokenRequestHandler(csrfTokenRequestAttributeH))
}
```

- For public pages, there is no need of CSRF protection as hacker will not be able to get any sensitive data from here. He can only submit new data. Make sure all these public APIs are permitted

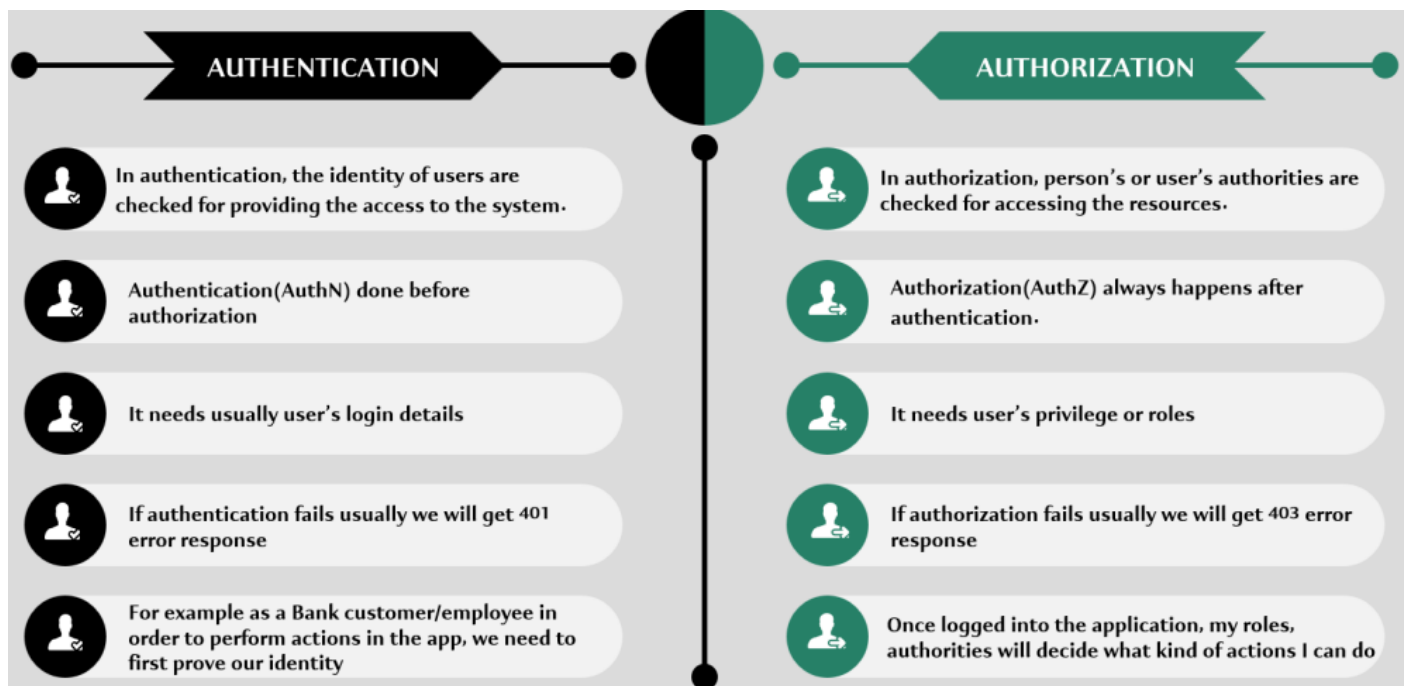
```
@Bean no usages Shivam-Omen
SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {
    CsrfTokenRequestAttributeHandler csrfTokenRequestAttributeHandler = new CsrfTokenRequestAttributeHandler();
    http.securityContext( SecurityContextConfigurer<HttpSecurity> contextConfig -> contextConfig.requireExplicitSave(false))
        .sessionManagement( SessionManagementConfigurer<HttpSecurity> sessionConfig -> sessionConfig
            .sessionCreationPolicy(SessionCreationPolicy.ALWAYS))
        .csrf( CsrfConfigurer<HttpSecurity> csrfConfig -> csrfConfig
            .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
            .csrfTokenRequestHandler(csrfTokenRequestAttributeHandler)
            .ignoringRequestMatchers(...patterns: "/contact", "/register"))
        .addFilterAfter(new CsrfCookieFilter(), BasicAuthenticationFilter.class)
        .cors( CorsConfigurer<HttpSecurity> corsConfig -> corsConfig.configurationSource( HttpServletRequest request -> {
            CorsConfiguration config = new CorsConfiguration();
            config.setAllowedOrigins(Collections.singletonList("http://localhost:4200"));
            config.setAllowedMethods(Collections.singletonList("*"));
            config.setAllowCredentials(true);
            config.setAllowedHeaders(Collections.singletonList("*"));
            config.setMaxAge(3600L);
            return config;
        }))
        .authorizeHttpRequests( AuthorizationManagerRequestMat... requests -> requests
            .requestMatchers(...patterns: "/myAccount", "/myBalance", "/myLoans", "/myCards", "/user").authenticated()
            .requestMatchers(...patterns: "/notices", "/contact", "/error", "/register", "/invalidSession").permitAll());

    http.formLogin(withDefaults());
    http.httpBasic( HttpBasicConfigurer<HttpSecurity> hbc -> hbc.authenticationEntryPoint(new CustomBasicAuthenticationEntryPoint()));
    http.exceptionHandling( ExceptionHandlingConfigurer<HttpSecurity> ehc -> ehc.accessDeniedHandler(new CustomAccessDeniedHandler()));

    return http.build();
}
```

Section 9: Implementing Authorization using Authorities, Roles

Authentication Vs Authorization



How the Authorities or Roles get stored inside Spring Security

- Authorities/Roles information in Spring Security is stored using GrantedAuthority interface. There is only one method inside this interface that returns the name of the authority or role.
- GrantedAuthority i/f defines the schema that anyone has to follow if they want to store roles or authorities' information.
- Implementation classes: SimpleGrantedAuthority (most commonly used Impl)
 - SimpleGrantedAuthority stores a string representation of an authority granted to the Authentication object.

Inside Spring Security framework, we represent the user – related data by using 2 different interfaces & its implementation classes

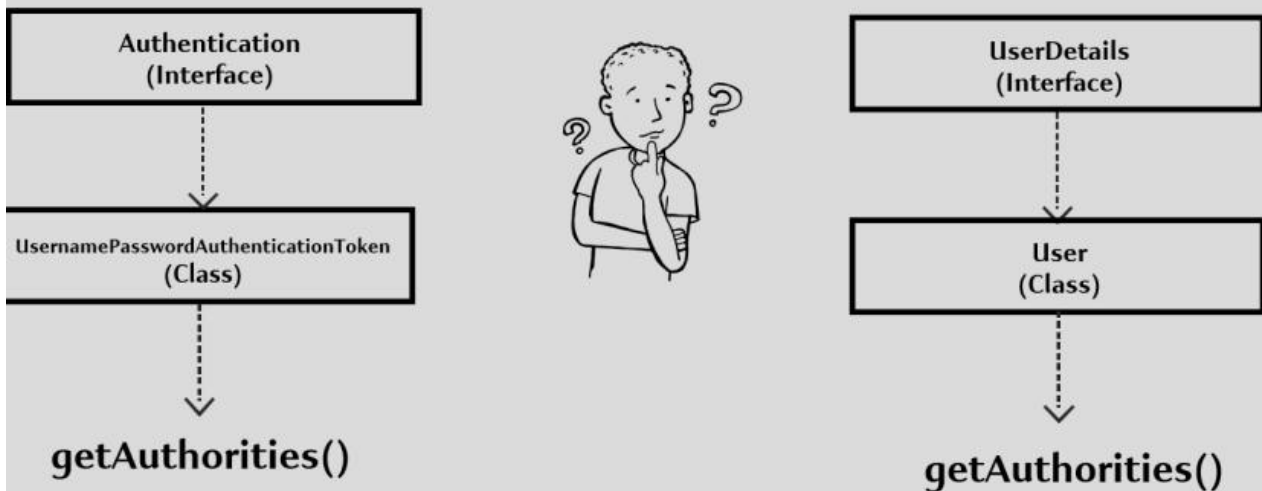
1. UserDetails i/f – During the authentication, while loading the UserDetails from the storage system, UserDetails implementation class object will be used to store user details.
2. Authentication i/f – Once authentication is successful, the logged-in user details will be stored in Authentication implementation class object.

Note: We can debug the above interfaces & implementation classes to understand how they're storing the authorities or roles information of an end user.

```
public interface GrantedAuthority {  
    String getAuthority();  
}
```

```
public final class SimpleGrantedAuthority implements GrantedAuthority {  
    private final String role;  
    public SimpleGrantedAuthority(String role) {  
        this.role = role;  
    }  
    @Override  
    public String getAuthority() {  
        return this.role;  
    }  
}
```

How does Authorities information stored inside the objects of UserDetails & Authentication interfaces which plays a vital role during authentication of the user ?



In Spring Security the authorities requirements can be configured using the following ways,

hasAuthority() — Accepts a single authority for which the endpoint will be configured and user will be validated against the single authority mentioned. Only users having the same authority configured can invoke the endpoint.

hasAnyAuthority() — Accepts multiple authorities for which the endpoint will be configured and user will be validated against the authorities mentioned. Only users having any of the authority configured can invoke the endpoint.

access() — Using Spring Expression Language (SpEL) it provides you unlimited possibilities for configuring authorities which are not possible with the above methods. We can use operators like OR, AND inside access() method.

You can extract path values from the request, as seen below by using access():

```
http.authorizeHttpRequests((authorize) -> authorize
    .requestMatchers("/cards/{name}").access(new
        WebExpressionAuthorizationManager("#name == authentication.name")))
```

You can check if the user has multiple roles, as seen below by using access():

```
.requestMatchers("/admin/**").access(allOf(hasAuthority("admin"),
hasAuthority("manager")))
```

You can check if the user has multiple roles, as seen below by using access():

```
.requestMatchers("/admin/**").access(new WebExpressionAuthorizationManager("
hasAuthority('admin') && hasAuthority('manager')"))
```

```

@Bean no usages Shivam-Omen
SecurityFilterChain defaultSecurityFilterChain(HttpSecurity http) throws Exception {
    CsrfTokenRequestAttributeHandler csrfTokenRequestAttribute = new CsrfTokenRequestAttributeHandler();
    http.securityContext((SecurityContextConfigurer<HttpSecurity> contextConfig -> contextConfig.requireExplicitSave(false))
        .sessionManagement((SessionManagementConfigurer<HttpSecurity> sessionConfig -> sessionConfig
            .sessionCreationPolicy(SessionCreationPolicy.ALWAYS))
        .csrf((CsrfConfigurer<HttpSecurity> csrfConfig -> csrfConfig
            .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse())
            .csrfTokenRequestHandler(csrfTokenRequestAttribute)
            .ignoringRequestMatchers(...patterns: "/contact", "/register")))
        .addFilterAfter(new CsrfCookieFilter(), BasicAuthenticationFilter.class)
        .cors((CorsConfigurer<HttpSecurity> corsConfig -> corsConfig.configurationSource((HttpServletRequest request -> {
            CorsConfiguration config = new CorsConfiguration();
            config.setAllowedOrigins(Collections.singletonList("http://localhost:4200"));
            config.setAllowedMethods(Collections.singletonList("*"));
            config.setAllowCredentials(true);
            config.setAllowedHeaders(Collections.singletonList("*"));
            config.setMaxAge(3600L);
            return config;
        })))
        .authorizeHttpRequests((AuthorizationManagerRequestMatcher requests -> requests
            .requestMatchers(...patterns: "/myAccount").hasAuthority("VIEWACCOUNT")
            .requestMatchers(...patterns: "/myBalance").hasAnyAuthority(...authorities: "VIEWBALANCE", "VIEWACCOUNT")
            .requestMatchers(...patterns: "/myLoans").hasAuthority("VIEWLOANS")
            .requestMatchers(...patterns: "/myCards").hasAuthority("VIEWCARDS")
            .requestMatchers(...patterns: "/user").authenticated()
            .requestMatchers(...patterns: "/notices", "/contact", "/error", "/register", "/invalidSession").permitAll());

    http.formLogin(withDefaults());
    http.httpBasic((HttpBasicConfigurer<HttpSecurity> hbc -> hbc.authenticationEntryPoint(new CustomBasicAuthenticationEntryPoint()));
    http.exceptionHandling((ExceptionHandlerConfigurer<HttpSecurity> ehc -> ehc.accessDeniedHandler(new CustomAccessDeniedHandler()));

    return http.build();
}

```

Authority Vs Role in Spring Security

Beside Authorities, Spring Security also allow us to enforce authorization with the help of ROLES concept. There is a very minute difference b/w Authority & Role.



Advantage of using Roles over Authority

- Whenever we're grouping the authorities using ROLE, the spring security related configuration would be minimal as usually in real project, there'll be 1000s of actions that an end user can perform inside an app & configuring all the 1000s of authorities using Spring Security config is going to be a tedious process.

Imp Note: While configuring ROLES in Spring Security config & Authorities table, we need to make sure all our roles have a common prefix "ROLE_". We can customize the prefix by defining a bean.

You can configure the authorization rules to use a different prefix by providing a `GrantedAuthorityDefaults` bean, as shown below:

```
@Bean
static GrantedAuthorityDefaults grantedAuthorityDefaults() {
    return new GrantedAuthorityDefaults("MYPREFIX_");
}
```

You need to expose `GrantedAuthorityDefaults` using a static method to ensure that Spring publishes it before initializing Spring Security's method security `@Configuration` classes



In Spring Security the ROLES requirements can be configured using the following ways,

hasRole() — Accepts a single role name for which the endpoint will be configured and user will be validated against the single role mentioned. Only users having the same role configured can invoke the endpoint.

hasAnyRole() — Accepts multiple roles for which the endpoint will be configured and user will be validated against the roles mentioned. Only users having any of the role configured can call the endpoint.

access() — Using Spring Expression Language (SpEL) it provides you unlimited possibilities for configuring roles which are not possible with the above methods. We can use operators like OR, AND inside `access()` method.

Note :

- **ROLE_ prefix only to be used while configuring the role in DB. But when we configure the roles, we do it only by its name.**

```
.authorizeHttpRequests( AuthorizationManagerRequestMat... requests -> requests
    /*.requestMatchers("/myAccount").hasAuthority("VIEWACCOUNT")
    .requestMatchers("/myBalance").hasAnyAuthority("VIEWBALANCE", "VIEWACCOUNT")
    .requestMatchers("/myLoans").hasAuthority("VIEWLOANS")
    .requestMatchers("/myCards").hasAuthority("VIEWCARDS")
    */
    .requestMatchers( ...patterns: "/myAccount").hasRole("USER")
    .requestMatchers( ...patterns: "/myBalance").hasAnyRole( ...roles: "USER", "ADMIN")
    .requestMatchers( ...patterns: "/myLoans").hasRole("USER")
    .requestMatchers( ...patterns: "/myCards").hasRole("USER")
    .requestMatchers( ...patterns: "/user").authenticated()
    .requestMatchers( ...patterns: "/notices", "/contact", "/error", "/register", "/invalidSession").permitAll());
```

Authorization Event

For each denied authorization, an '`AuthorizationDeniedEvent`' is triggered. You can listen these events, using the below kind of logic,

```
@EventListener
public void onFailure(AuthorizationDeniedEvent deniedEvent) {
    log.error("Authorization failed for the user : {} due to : {}",
        deniedEvent.getAuthentication().getName(),
        deniedEvent.getAuthorizationDecision().toString());
}
```

Due to the potential for `AuthorizationGrantedEvents` to generate excessive noise, they are not published by default.

Custom Filters in Spring Security

- Lots of times we will have situations where we need to perform some housekeeping activities during the authentication & authorization flow. Few such examples are:
 - Input Validation
 - Tracing, Auditing & reporting
 - Logging of input like IP address etc.
 - Encryption & Decryption.
- All such requirements can be handled using HTTP Filters inside Spring Security. We already saw some inbuilt filters in Spring Security like UsernamePasswordAuthenticationFilter, BasicAuthenticationFilter etc.
- A Filter is a component which receives requests, process its logic & handover to the next filter in the chain.
- Spring Security is based on a chain of servlet filters. Each filter has a specific responsibility & depending on the configuration, filters are added or removed. We can add our custom filters as well based on the need.

✓ We can always check the registered filters inside Spring Security with the below configurations,

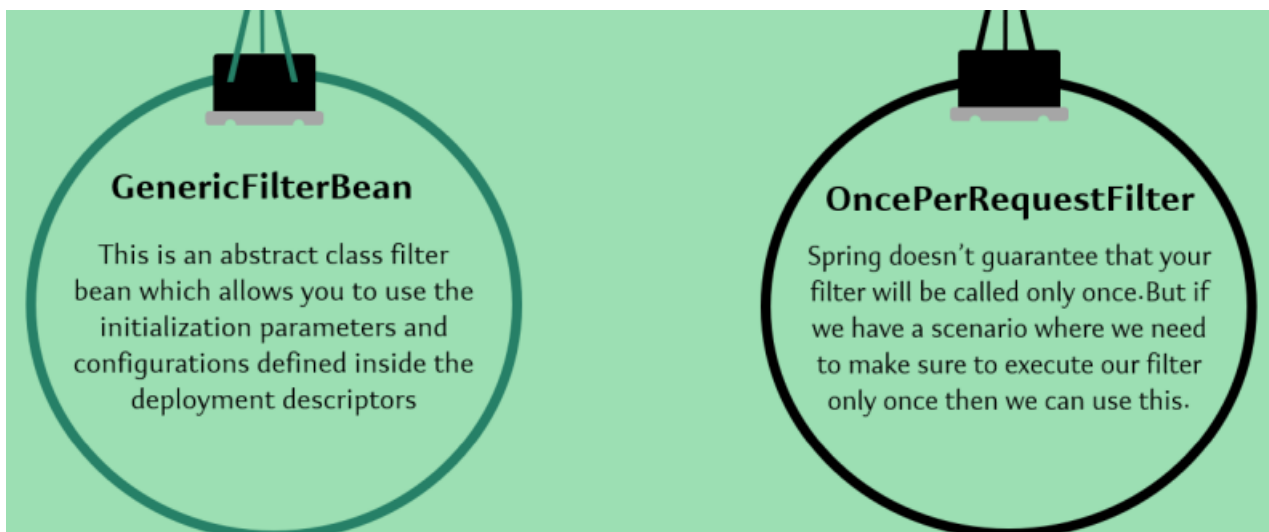
1. `@EnableWebSecurity(debug = true)` – We need to enable the debugging of the security details
2. Enable logging of the details by adding the below property in application.properties
`logging.level.org.springframework.security.web.FilterChainProxy=DEBUG`

Attached are the some of the internal filters of Spring Security that gets executed in the authentication flow,

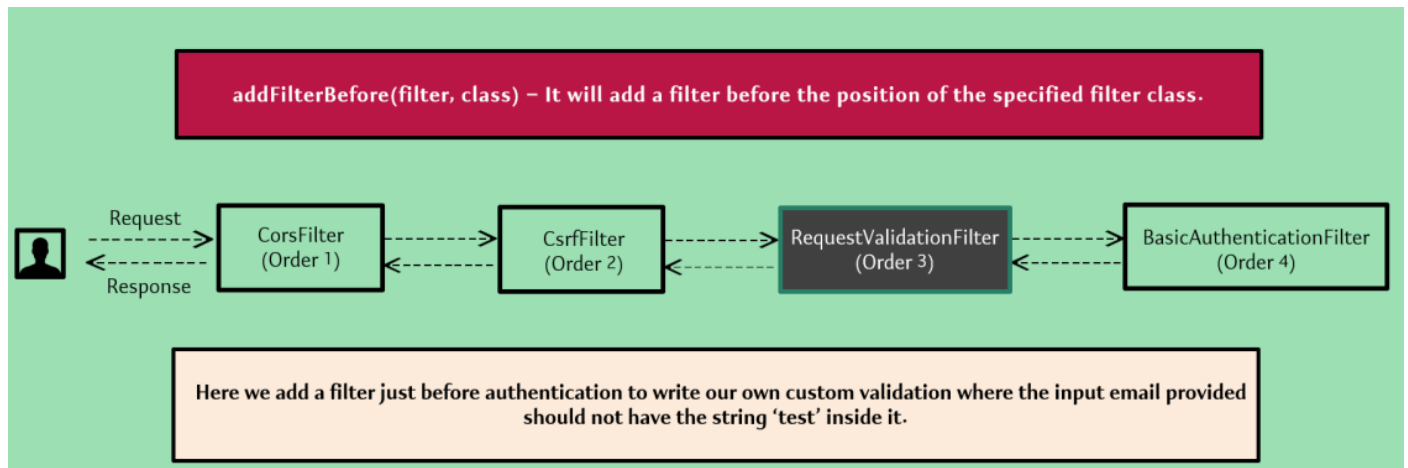
```
Security filter chain: [  
  DisableEncodeUrlFilter  
  ForceEagerSessionCreationFilter  
  ChannelProcessingFilter  
  WebAsyncManagerIntegrationFilter  
  SecurityContextPersistenceFilter  
  HeaderWriterFilter  
  CorsFilter  
  CsrfFilter  
  LogoutFilter  
  UsernamePasswordAuthenticationFilter  
  DefaultLoginPageGeneratingFilter  
  DefaultLogoutPageGeneratingFilter  
  BasicAuthenticationFilter  
  CsrfCookieFilter  
  RequestCacheAwareFilter  
  SecurityContextHolderAwareRequestFilter  
  AnonymousAuthenticationFilter  
  SessionManagementFilter  
  ExceptionTranslationFilter  
  AuthorizationFilter  
]
```

- We can create our own filters by implementing the Filter i/f from jakarta.servlet package. Post that we need to override the `doFilter()` method to have our own custom logic. For deep analysis, refer `FilterChainProxy` class & `VirtualFilterChain`.

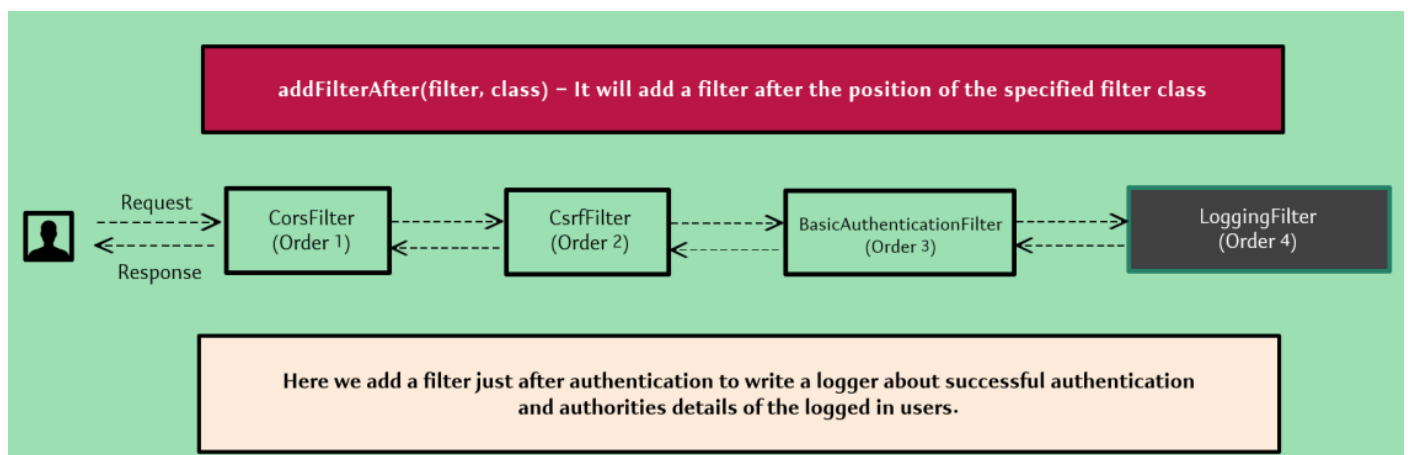
- ✓ You can add a new filter to the spring security chain either before, after, or at the position of a known one. Each position of the filter is an index (a number), and you might find it also referred to as “the order.”
- ✓ Below are the methods available to configure a custom filter in the spring security flow,
 - `addFilterBefore(filter, class)` – adds a filter before the position of the specified filter class
 - `addFilterAfter(filter, class)` – adds a filter after the position of the specified filter class
 - `addFilterAt(filter, class)` – adds a filter at the location of the specified filter class



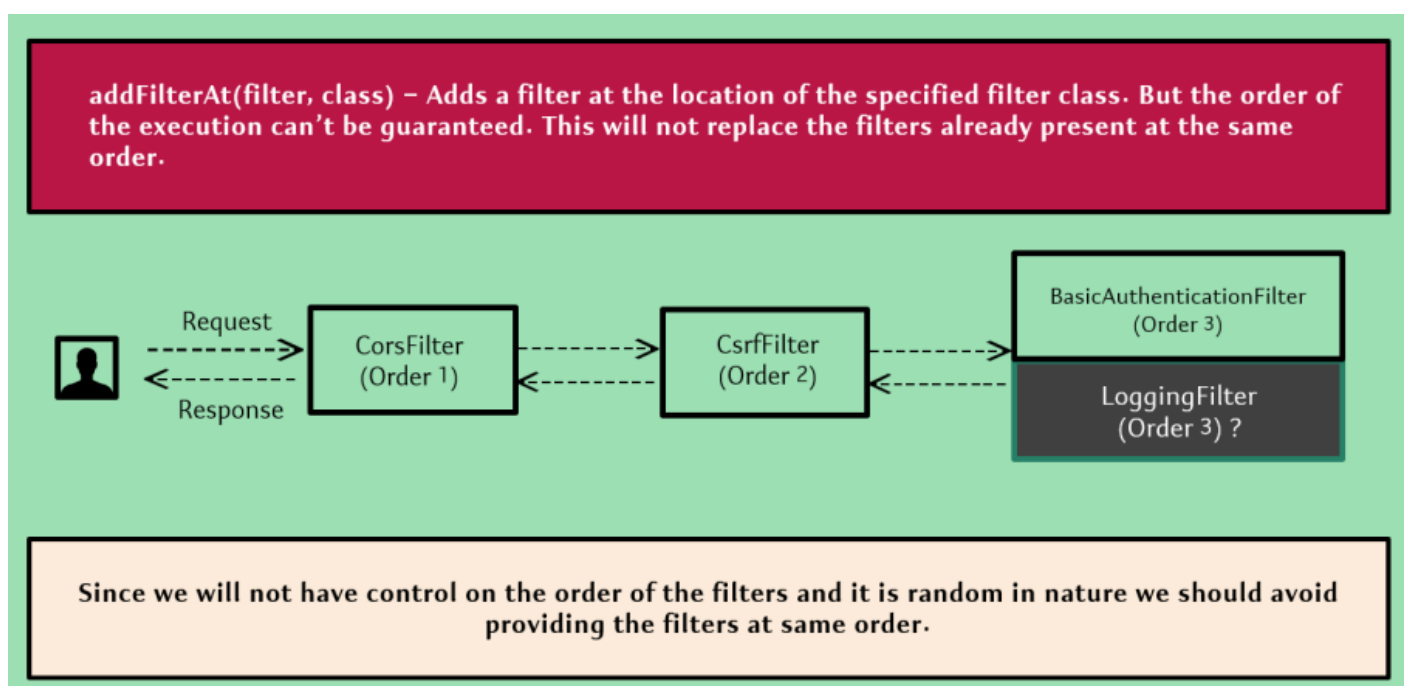
Add Filter Before in Spring Security



Add Filter After in Spring Security



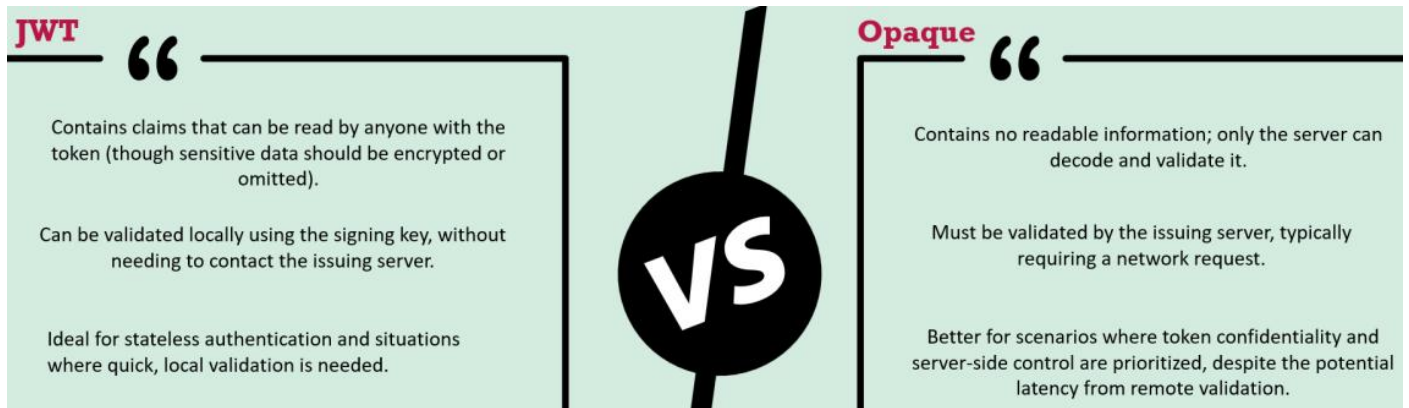
Add Filter At in Spring Security



Token Based Authentication using JSON Web Token (JWT)

Opaque Tokens Vs JSON Web Token (JWT)

- In general, there are 2 token formats:
 1. Opaque Tokens
 2. JSON Web Tokens (JWT)



1. Opaque Tokens

- Opaque tokens are typically long, random strings with no inherent meaning or structure that can be decoded. Used to reference authentication information stored on the server side.
- Since they contain no embedded information, they require a call to the Authorization Server or a dedicated introspection endpoint to validate and obtain user information.
- Suitable for scenarios where token validation by a central server is feasible, such as within a secure internal network.

2. JWT Tokens

- A JWT is a JSON string that contains all the claims and information it represents & is certified by a signature from the issuer.
- JWTs are self-contained token in JSON format that consists of 3 parts needed for verification within the token itself: Header, Payload & Signature, encoded in Base64 URL.
- JWT tokens are ideal for stateless, distributed systems where quick token validation is needed without frequent server calls. (can be validated locally by verifying the token's signature using a public key, without needing a server call)

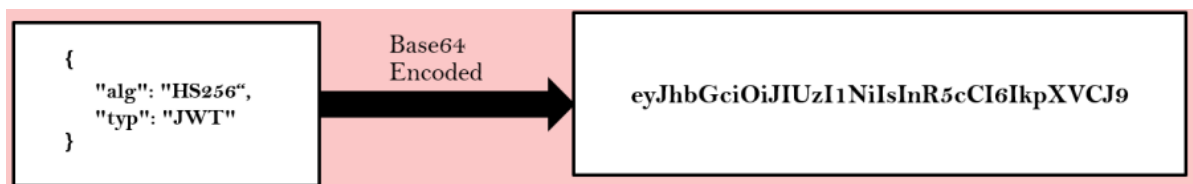
A JWT token has 3 parts each separated by a period(.) Below is a sample JWT token,

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtciI6IjZSL6IkpvaGUiOiJkaWY6IiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

1. Header
2. Payload
3. Signature (Optional)

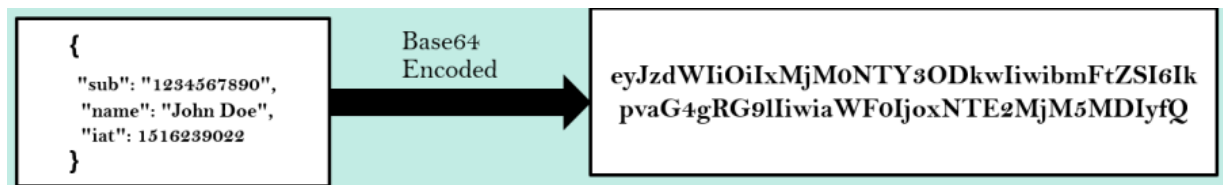
a) JWT Header

- Inside the JWT header, we store metadata/info related to the token. If we chose to sign the token, the header contains the name of the algorithm that generates the signature



b) JWT Payload/Body

- In the body, we can store details related to user, roles etc, which can be used later for AuthN & AuthZ.
- Though there is no such limitation what we can send & how much we can send in the body, but we should put our best efforts to keep it as light as possible.



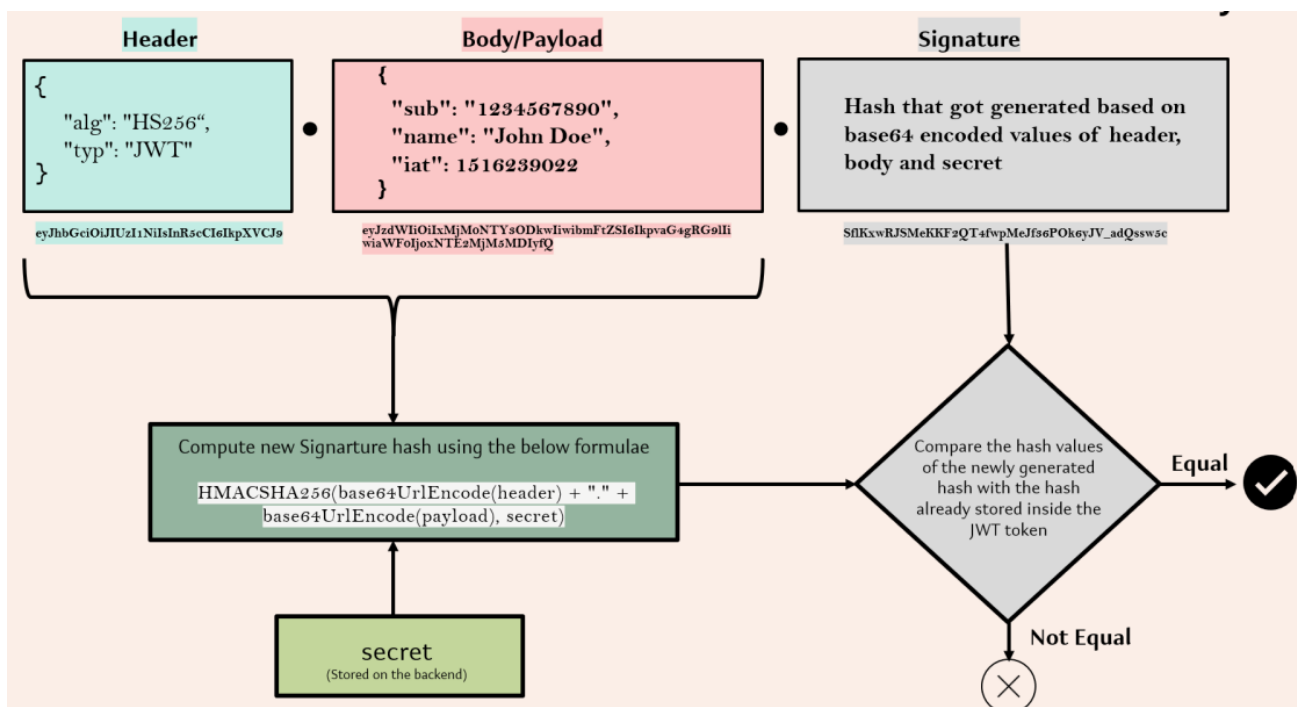
c) JWT Digital Signature

- Digital signature is optional in the token if the party that you want to share JWT token is internal but not open in the web.
- To make sure that no one tampered data on the network, we can send the signature of the content when initially the token is generated.
- To create the signature part, we have to take the encoder header & the encoded payload, a secret, the algorithm specified in the header & sign that.

✓ For example if you want to use the HMAC SHA256 algorithm, the signature will be created in the following way:

$\text{HMACSHA256}(\text{base64UrlEncode}(\text{header}) + "." + \text{base64UrlEncode}(\text{payload}), \text{secret})$

✓ The signature is used to verify the message wasn't changed along the way, and, in the case of tokens signed with a private key, it can also verify that the sender of the JWT is who it says it is.



- If we want to play with JWT tokens & put these concepts into practice, we can use <https://www.jwt.io/> debugger to decode, verify & generate JWTs.

JWT
Debugger

Introduction

Libraries

As...

ENCODED VALUE

JSON WEB TOKEN (JWT)

COPY CLEAR

Valid JWT

Signature Verified

eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG91IiwiaWF0Ij0iOnRydwUsIm1hdCI6MTUxNjIzOTYmN0. KMUFsIDTnFmyG3nMiGM6H9FNfUROf3wh75mq3p-QV30

ENABLE AUTO-FOCUS

☐ Enable auto-focus

DECODED HEADER

JSON

CLAIMS TABLE

COPY COPY

```
{  "alg": "HS256",  "typ": "JWT"}
```

DECODED PAYLOAD

JSON

CLAIMS TABLE

COPY COPY

```
{  "sub": "1234567890",  "name": "John Doe",  "admin": true,  "iat": 1516239022}
```

JWT SIGNATURE VERIFICATION (OPTIONAL)

Enter the secret used to sign the JWT below:

SECRET

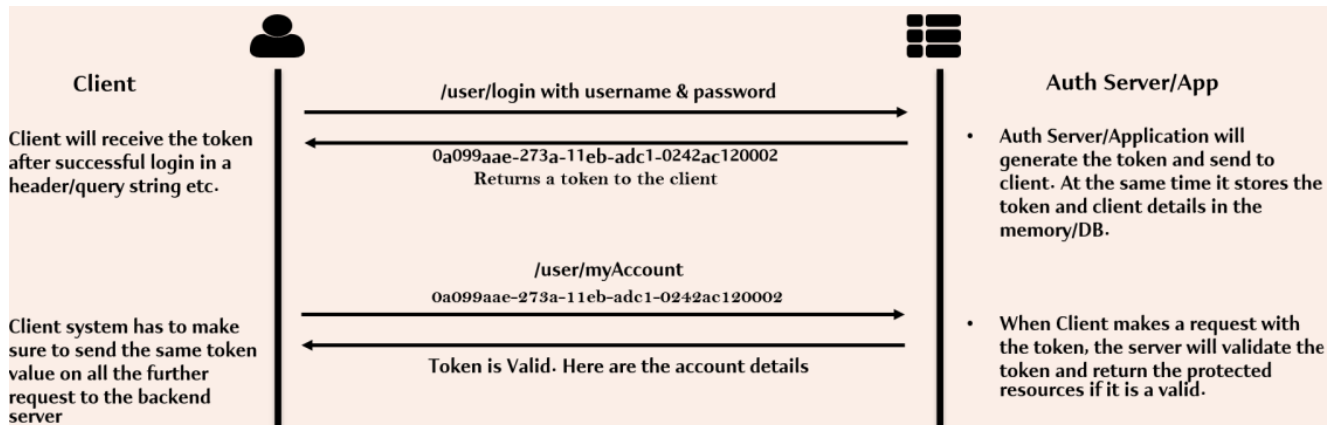
COPY CLEAR

Valid secret

a-string-secret-at-least-256-bits-long

Role of Tokens in AuthN & AuthZ

- A token can be a plain string of UUID format or it can be of type JWT usually that get generated when the user authenticated for the first time during login.
- On every request to a restricted resource, the client sends the access token in the query string or Authorization header. The server then validates the token & if it's valid, return the secure resource to the client.



Token Advantages

1. Security

- Limited expose of the user credentials inside the network.
- Tokens can be revoked during any suspicious activities without invalidating the user credentials.

2. Expiration

- Tokens can have specific expiration times set, ensuring tokens are valid only for a predefined duration.

3. Self-contained

- Tokens are self-contained & carry all the necessary information about the user, roles/authorities etc.

4. Reusability

- Tokens can be used across different domains & services, making them suitable for Single Sign-On (SSO) systems.

5. Cross-Platform Compatibility

- Tokens can be used across various platforms & devices, including web applications, mobile apps, & IOT devices.

6. Statelessness

- The token contains all the information to identify the user, **eliminating the need for the session state**. If we use a load balancer, we can pass the user to any server, instead of being bound to the same server we logged in on.

Security Configuration to use JWT Tokens

Step 1: We need to add the dependency for the library supporting JWT generation & validation. (Refer GitHub repo section11- pom.xml)

Step 2: As of now, we were telling to the Spring Security framework to generate the JSESSION ID always but now since we want to go with the JWT advanced token format, we need to convey not to generate JSESSION ID. We can achieve this by using STATELESS session creation policy.

```
http.sessionManagement( SessionManagementConfigurer<HttpSecurity> sessionConfig -> sessionConfig
    .sessionCreationPolicy(SessionCreationPolicy.STATELESS))
```

By using STATELESS policy, the JWT token will not be stored inside the session. Once the token will be generated, it will be sent back to UI app & UI will send this token for further request.

This makes our application more secure compared to JSESSION ID approach.

Step 3: We will remove the sessionContext related changes as it was required when we generate the JSESSION ID

Step 4: We need to expose the header to the UI application/Client (deployed at different origin) as JWT token will be sent as the Response header to the client.

```
.cors( CorsConfigurer<HttpSecurity> corsConfig -> corsConfig.configurationSource( HttpServletRequest request -> {
    CorsConfiguration config = new CorsConfiguration();
    config.setAllowedOrigins(Collections.singletonList("http://localhost:4200"));
    config.setAllowedMethods(Collections.singletonList("*"));
    config.setAllowCredentials(true);
    config.setAllowedHeaders(Collections.singletonList("*"));
    config.setExposedHeaders(List.of("Authorization"));
    config.setMaxAge(3600L);
    return config;
}))
```


Step 5: How we're going to generate & validate a JWT token

JWT token Generation Filter

- We would want to generate the JWT token once the initial login is completed. We need to create a new filter extending OncePerRequestFilter.
- In all upcoming request, we don't want our filter to be invoked, because for all the upcoming requests, we want to validate the token but not generate. We need to override below method to convey Spring Security the same.
- Add the logic for token generation in doFilterInternal method.

```
Overriding this method to not execute the filter in certain scenarios.  
  
If return false, the filter will be executed.  
If return true, the filter will not be executed.  
  
We want to execute this filter only for the "/user" endpoint (during login operation)  
  
@Override 4 usages  
protected boolean shouldNotFilter(HttpServletRequest request) throws ServletException {  
    return !request.getServletPath().equals("/user");  
}
```

JWT token validation Filter

- We would want to validate the token every time the request comes to the backend. We need to create a new filter extending OncePerRequestFilter.
- In all upcoming request except the during login operation, we want to invoke this filter. We need to add below config:

```
Overriding this method to not execute the filter in certain scenarios.  
  
If return false, the filter will be executed.  
If return true, the filter will not be executed.  
  
We want to execute this filter for all the request except the "/user" endpoint (login operation)  
  
@Override 4 usages  
protected boolean shouldNotFilter(HttpServletRequest request) throws ServletException {  
    return request.getServletPath().equals("/user");  
}
```

- Add the logic for token validation in doFilterInternal method.

Step 6: Add Token Generation & Validation filter to the Security configuration.

- Add Token Generation Filter after login i.e., BasicAuthenticationFilter
- Add Token Validation Filter before BasicAuthenticationFilter so that we will inform Spring Security framework that authentication is already successful, no need to perform authentication one more time.

```
.addFilterAfter(new JWTTokenGeneratorFilter(), BasicAuthenticationFilter.class)  
.addFilterBefore(new JWTTokenValidationFilter(), BasicAuthenticationFilter.class)
```

JWT Token received as part of Response Header to UI.

Name	Headers	Preview	Response	Initiator	Timing	Adblock	Cookies
☐ user							
☒ user	▼ Response Headers		☐ Raw				
☒ favicon.ico	Access-Control-Allow-Credentials		true				
	Access-Control-Allow-Origin		http://localhost:4200				
	Access-Control-Expose-Headers		Authorization				
	Authorization		eyJhbGciOiJIUzI1NiU9.eyJpc3MiOiJFYXp5IEJhbmsiLCJzdWIiOiJKV1QgVG9rZW4iLCJ1Ij09ImlhdCI6MTY0MjQ0MjQwMywMywiZXhwIjoxNzUzNDcyNDZfQ7uMRlow4aoN8				

JWT Token sent to BE as part of Request Header.

myAccount?id=1

myAccount?id=1

favicon.ico

Request Headers

Accept

Accept-Encoding

Accept-Language

Authorization

application/json, text/plain, */*

gzip, deflate, br, zstd

en-US,en;q=0.8

eyJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJFYXp5IEJhbmsiLCJzdWl0IjKV1QgVG9rZW4iLCJ1c2VybmFtZSI6ImhhdCI6MTc1MzQ0MTJlIjN5IiwiaXNjaXNzUzNDcyMjc3IjQ6bnNhbnBoPSI6LGH9Ds-3Rr6zmLOanZ

GET

/user

Params

Authorization

Headers (8)

Body

Scripts

Settings

Auth Type

Basic Auth

Username

happy@example.com

Password

.....

The authorization header will be automatically generated when you send the

Body

Cookies (1)

Headers (16)

Test Results

200 OK

157 ms

854 B

Vary

Access-Control-Request-Headers

Set-Cookie

XSRF-TOKEN=da31e37e-ca21-4089-b71e-dda15e94d95c; Path=/

Authorization

eyJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJFYXp5IEJhbmsiLCJzdWl0IjKV1QgVG9rZW4iLCJ1c2VybmFtZSI6ImhhdCI6MTc1MzQ0MTJlIjN5IiwiaXNjaXNzUzNDcyMjc3IjQ6bnNhbnBoPSI6LGH9Ds-3Rr6zmLOanZ

GET

/myAccount?id=1

Params

Authorization

Headers (8)

Body

Scripts

Settings

Accept-Encoding

Connection

Authorization

Key

gzip, deflate, br

keep-alive

eyJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJFYXp5IEJhbmsiLCJzdWl0IjKV1QgVG9rZW4iLCJ1c2VybmFtZSI6ImhhdCI6MTc1MzQ0MTJlIjN5IiwiaXNjaXNzUzNDcyMjc3IjQ6bnNhbnBoPSI6LGH9Ds-3Rr6zmLOanZ

Body

Cookies

Headers (15)

Test Results

JSON

Preview

Visualize

1 {

2 "customerId": 1,

3 "accountNumber": 1865764534,

4 "accountType": "Savings",

5 "branchAddress": "123 Main Street, New York",

6 "createDt": "2025-07-21"

7 }

Publish an AuthenticationManager bean for Custom Authentication

- As of now inside our application, we're supporting the login operation using the HTTP Basic Authentication format & FormLogin.
- Anyone who wants to login to our application, first they need to invoke the /user api & pass basic Authentication credentials & since this is a secure API on backend side, the Spring Security framework will take care of Authenticating the end user with credentials we passed.
- Apart from this FormLogin or Http Basic Authentication, we may get a requirement to accept the credentials inside the request body / header, without following any standards of form login/Http Basic format. In this kind of scenarios whatever Spring Security framework provides is not going to help us.
- We need to build our own REST API operation that accepts the user credentials wherever we want & inside this API we need to invoke the Authentication manually. Once the Authentication is completed, the JWT token or any other token value we want to send inside the Request body.

```
@PostMapping("/apiLogin") no usages Shivam-Omen
public ResponseEntity<LoginResponseDTO> apiLogin (@RequestBody LoginRequestDTO loginRequest) {
    String jwt = "";
    Authentication authentication = UsernamePasswordAuthenticationToken.unauthenticated(loginRequest.username(),
        loginRequest.password());
    Authentication authenticationResponse = authenticationManager.authenticate(authentication);
    if(null != authenticationResponse && authenticationResponse.isAuthenticated()) {
        if (null != env) {
            String secret = env.getProperty(ApplicationConstants.JWT_SECRET_KEY,
                ApplicationConstants.JWT_SECRET_DEFAULT_VALUE);
            SecretKey secretKey = Keys.hmacShaKeyFor(secret.getBytes(StandardCharsets.UTF_8));
            jwt = Jwts.builder().issuer("Eazy Bank").subject("JWT Token")
                .claim("name: username", authenticationResponse.getName())
                .claim("name: authorities", authenticationResponse.getAuthorities().stream().map(
                    GrantedAuthority::getAuthority).collect(Collectors.joining(" ", ")))
                .issuedAt(new Date())
                .expiration(new Date((new Date()).getTime() + 300000000))
                .signWith(secretKey).compact();
        }
    }
    return ResponseEntity.status(HttpStatus.OK).header(ApplicationConstants.JWT_HEADER, jwt)
        .body(new LoginResponseDTO(HttpStatus.OK.getReasonPhrase(), jwt));
}
```

Default value for eraseCredentialsAfterAuthentication is true

providerManager.setEraseCredentialsAfterAuthentication(false) means provider manager will not erase the password inside the Authentication object.

```
@Bean no usages Shivam-Omen
public AuthenticationManager authenticationManager(UserDetailsService userDetailsService, PasswordEncoder passwordEncoder) {
    EazyBankUsernamePwdAuthenticationProvider authenticationProvider =
        new EazyBankUsernamePwdAuthenticationProvider(userDetailsService, passwordEncoder);
    ProviderManager providerManager = new ProviderManager(authenticationProvider);
    providerManager.setEraseCredentialsAfterAuthentication(false);
    return providerManager;
}
```

POST /api/login

Params Authorization Headers (9) **Body** Scripts Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON** ▾

```

1 {
2   "username": "happy@example.com",
3   "password": "EazyBytes@12345"
4 }

```

Body Cookies (1) Headers (16) Test Results ↻ 200 OK • 404 ms • 1.02 KB • 🌐

{ } JSON ▾ ▶ Preview 🔄 Visualize ▾

```

1 {
2   "status": "OK",
3   "jwtToken": "eyJhbGciOiJIUzI1NiJ9.eyJpc3MiOiJFYXp5IEJhbmsiLCJzdWIiOiJKV1QgVG9rZW4iLCJ1c2VybmFtZSI6ImhhcHB5QGV4YW1wbGUuY29tIiwiaXV0aG9yaXRpZXM0IjI6fVVFUuiIsImh0dCI6MTc1MzQ1MDM2MSwiZXhwIjoxNzUzNDgwMzYxZQ.dHoc6QRxQuIsPV2G8f80JNSOP-midomwa9v085Ry2yQ"
4 }

```

Method Level Security

- As of now we've applied authorization rules on the API paths/URLs using Spring Security but Method level security allows to apply the Authorization rules at any layer of an application like in service layer or repository layer etc.
- Method Level Security can be enabled using the annotation `@EnableMethodSecurity` on the configuration class.
- Method Level Security also helps in applying authorization rules even in the non-web applications like Desktop apps where we will not have any endpoints.
- Method Level Security provides the below approaches to apply the Authorization rules & executing our business logic:
 - Invocation Authorization** – Validates if someone can invoke a method or not based on their roles/authorities.
 - Filtering Authorization** – Validates what a method can receive through its parameters & what the invoker can receive back from the method post business logic execution.
- Spring Security will use the aspects from the AOP module & have the interceptors in b/w the method invocation to apply the authorization rules configured.
- Method Level Security offers below 3 different styles for configuring the authorization rules on top of the methods:
 - The **prePostEnabled** property (default value as true) enables Spring Security **@PreAuthorize** & **@PostAuthorize** annotations.
 - The **securedEnabled** property enables **@Secured** annotation
 - The **jsr250Enabled** property enables **@RoleAllowed** annotation

```

@Configuration
@EnableMethodSecurity(prePostEnabled = true, securedEnabled = true, jsr250Enabled = true)
public class ProjectSecurityConfig {
    ...
}

```

✓ `@Secured` and `@RoleAllowed` are less powerful compared to `@PreAuthorize` and `@PostAuthorize`

@PreAuthorize

- Using invocation authorization we can decide if a user is authorized to invoke a method before the method executes (preauthorization) or after the method execution is completed (postauthorization). For filtering the parameters before calling the method we can use Prefiltering,

```
@Service
public class LoansService {

    @PreAuthorize("hasAuthority('VIEWLOANS')")
    @PreAuthorize("hasRole('ADMIN')")
    @PreAuthorize("hasAnyRole('ADMIN','USER')")
    @PreAuthorize("#username == authentication.principal.username")
    public Loan getLoanDetails(String username) {
        return loansRepository.loadLoanDetailsByUsername(username);
    }
}
```

```
@Repository 2 usages  ⬆ Shivam-Omen *
public interface LoanRepository extends CrudRepository<Loans, Long> {

    @PreAuthorize("hasRole('USER')") 1 usage  ⬆ Shivam-Omen
    // @PreAuthorize("hasRole('ROOT')") // Negative Case
    List<Loans> findByCustomerIdOrderByStartDtDesc(long customerId);
}
```

- Referring above code snippet, the method findByCustomer... will be invoked only if the given logged in user has "USER" role.
- In case the Preauthorization failure, 403 error will be thrown by the Spring Security framework while it is trying to invoke the method. So, during the API invocation, there are no issues but while trying to invoke the method, behind the scenes, Spring Security framework will perform the Authorization checks using AOP.

@PostAuthorize

- For applying postauthorization rules below is the sample configuration,

```
@Service
public class LoanService {

    @PostAuthorize("hasAuthority('VIEWLOANS')")
    @PostAuthorize("hasRole('ADMIN')")
    @PostAuthorize("hasAnyRole('ADMIN','USER')")
    @PostAuthorize("returnObject.username == authentication.principal.username")
    public Loan getLoanDetails(String username) {
        return loanRepository.loadLoanByUsername(username);
    }
}
```

```
@GetMapping("/myLoans") no usages  ⬆ Shivam-Omen
@PostAuthorize("hasRole('USER')")
// @PostAuthorize("hasRole('ROOT')") // Negative Case
public List<Loans> getLoanDetails(@RequestParam long id) {
    List<Loans> loans = loanRepository.findByCustomerIdOrderByStartDtDesc(id);
    if (loans != null) {
        return loans;
    } else {
        return null;
    }
}
```

- @PostAuthorize will not stop method execution, instead it will stop sending the response from the annotated method if the authorization check fails.

@PreFilter

- If we have a scenario where we don't want to control the invocation of the method but we want to make sure that the parameters sent and received to/from the method need to follow authorization rules or filtering criteria, then we can consider filtering.
- For filtering the parameters before calling the method we can use PreFilter annotation. But please note that the filterObject should be of type Collection interface.

```
@RestController
public class ContactController {

    @PreFilter("filterObject.contactName != 'Test'")
    public List<Contact> saveContactInquiryDetails(@RequestBody
        List<Contact> contacts)
    {
        // business logic
        return contacts;
    }
}
```

- Refer PreFilterAuthorizationMethodInterceptor & PostFilterAuthorizationMethodInterceptor for in-depth understanding.

@PostFilter

- For filtering the parameters after executing the method we can use PostFilter annotation. But please note that the filterObject should be of type Collection interface.

```
@RestController
public class ContactController {

    @PostFilter("filterObject.contactName != 'Test'")
    public List<Contact> saveContactInquiryDetails(@RequestBody List<Contact>
        contacts)
    {
        // business logic
        return contacts;
    }
}
```

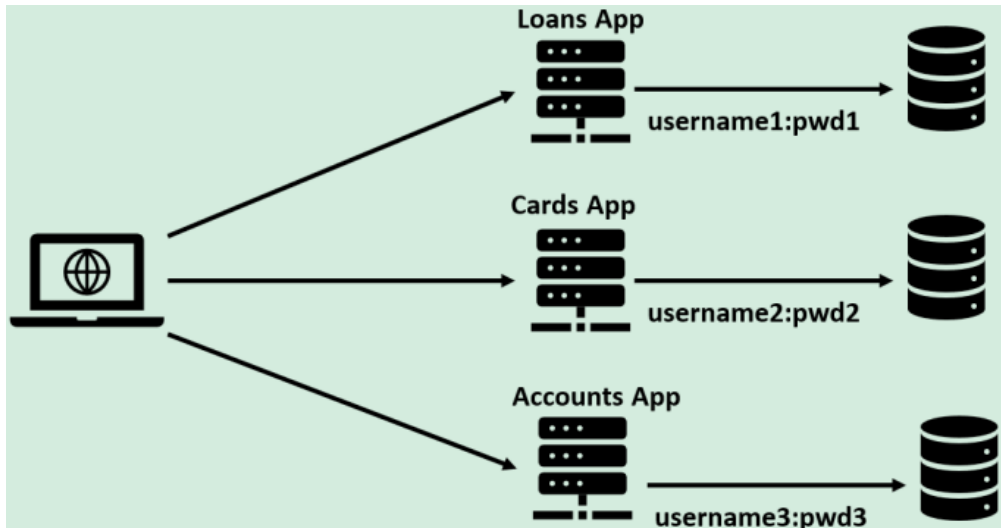
- We can use the @PostFilter on the Spring Data repository methods as well to filter any unwanted data coming from the database.

```
@PostMapping("/contact") no usages new *
//@PreFilter("filterObject.contactName != 'Test'")
@PostFilter("filterObject.contactName != 'Test'")
public List<Contact> saveContactInquiryDetails(@RequestBody List<Contact> contacts) {
    List<Contact> returnContacts = new ArrayList<>();
    if(!contacts.isEmpty()) {
        Contact contact = contacts.getFirst();
        contact.setContactId(getServiceReqNumber());
        contact.setCreateDt(new Date(System.currentTimeMillis()));
        Contact savedContact = contactRepository.save(contact);
        returnContacts.add(savedContact);
    }
    return returnContacts;
}
```


Section 13: OAuth2 & OpenID Connect

Problems the OAuth2 trying to solve:

- If a bank has multiple websites supporting accounts, loans, cards etc. Without OAuth2, the Bank customer has to register & maintain different user profiles in all 3 systems.
- Even the AuthN & AuthZ logic, Security standards will be duplicated in all the 3 websites.
- Any future changes or enhancements around security, authentication etc. need to be done in all the places.



Q. How come, google let us use the same account in all its product? Though they're different website/apps deployed on different servers

Ans: With the help of OAuth2 implementation. OAuth2 recommend to use a separate Auth server for Authentication & Authorization.

**photos.google.com**

**Google Photos
End user**

**PhotoEditor** website that allows to edit your phtos, make albums, collage etc.



✓ **Scenario :** The end user wants to use a third-party website called PhotoEditor to edit the photos stored in their Google Photos account.

- **Without OAuth2:** The end user must share their Google account credentials with the PhotoEditor website. Using these credentials, the PhotoEditor website will invoke the Google Photos API to fetch photo details and generate albums, collages, etc.

However, this approach has a significant disadvantage: PhotoEditor could act fraudulently and perform other operations on the user's behalf, such as changing the password, changing the email address, deleting photos, and more.

- **With OAuth2:** The end user doesn't have to share their Google account credentials with the PhotoEditor website. Instead, they authorize Google to give a temporary access token to PhotoEditor with limited access permissions, such as only reading the image data.

With this approach, PhotoEditor can only read the image data and cannot perform any other operations on the user's behalf, ensuring a more secure interaction.

Introduction to OAuth2

- OAuth stands for Open Authorization. It's a free & open protocol, built on IETF standards & licenses from the Open Web Foundation.
- OAuth 2.0 is a security standard where we give one application permission to access our data in another application. The steps to grant permission or consent are often referred as a Authorization or Delegated Authorization.
- We authorize one application to access our data, or use features in another application on our behalf, without giving them our password.
- In many ways, we can think of the OAuth token as a "access card" at any office/hotel. These tokens provide limited access to someone, without handing over full control in the form of the master key.
- The OAuth framework specifies several Grant Types for different use cases, as well as a framework for creating new Grant Types:
 1. Authorization Code
 2. PKCE
 3. Client Credentials
 4. Device Code
 5. Refresh Token
 6. Implicit Flow (Legacy)
 7. Resource Owner Credentials / Password Grant (Legacy)

OAuth2 Terminology

Resource Owner (i.e., End User)

- Resource owner is the end user that owns the resources (e.g., photos)
- E.g., In the scenario of Photo Editor application, the end user who want to use the Photo Editor website to edit his photo.

Client

- Client is an application or website that interacts with the Authorization server after taking permission from the Resource owner.
- For e.g., Photo Editor website

Authorization Server

- Authorization server is responsible for the validating the client's request to access the resource based on the Resource owner consent.
- Resource owner should have an account on this server.
- For e.g., In the scenario of Photo Editor, Google Auth Server acts as the Authorization Server.

Resource Server

- Resource server is the server that hosts the APIs, service or resources that the client wants to consume.
- For e.g., In the scenario of Photo Editor, the Google Photos server (hosts APIs like /getImages etc) acts as the Resource Server.
- In smaller organization, a single server can act as both Resource Server & Auth Server.

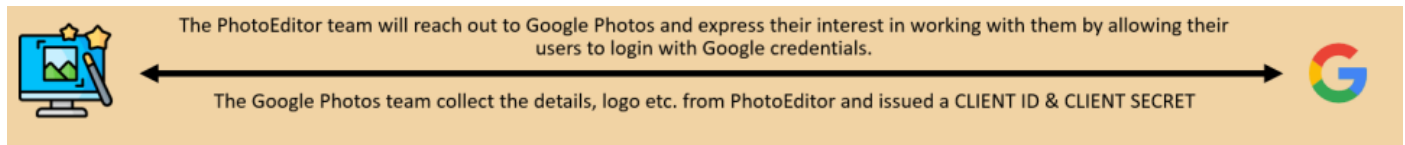
Scopes

- Scopes are the granular permissions the client wants, such as access to data or to perform certain actions.
- For e.g., In the scenario of Photo Editor, the Auth Server can issue an access token to Client with the scope of only READ IMAGES.

OAuth2 Sample Flow

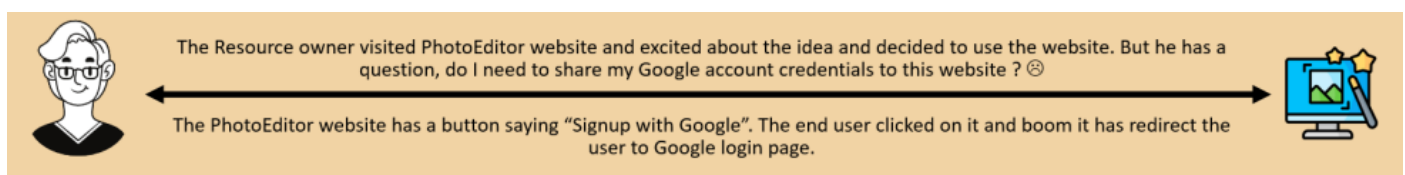
Step 1: Client Registration to Authorization Server

- The clients will register themselves with the Authorization server & post registration the Authorization server will issue them the CLIENT ID & CLIENT SECRET which will later use for client identification.



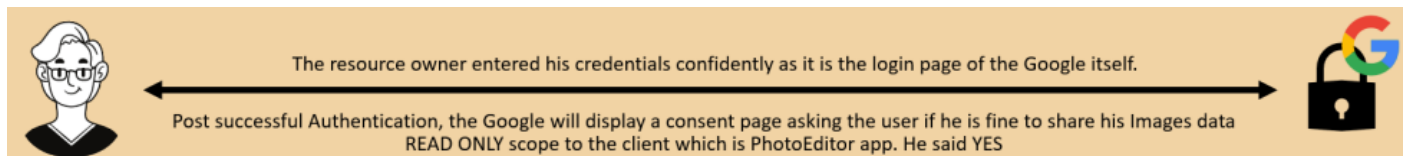
Step 2: Resource Owner/End user Signup/Sign-in to Client Website/Application

- The End user would like to use the Client application to explore client's application features & the client wants the end user to login-in via Auth Server registered credential.
- For e.g., The end user can login via Signup with Google (Auth Server) that will redirect the user to Google Login Page (Auth Server).



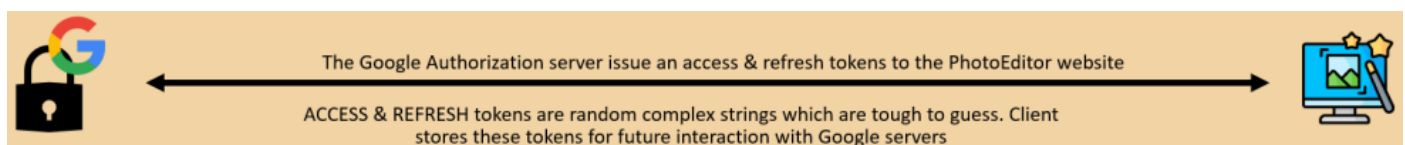
Step 3: Resource Owner Authentication with Auth Server & Resource access specification consent to the Client

- The end user will be authenticated to Auth Server and post authentication; the required access content is given to the Client based on the client's registration details.

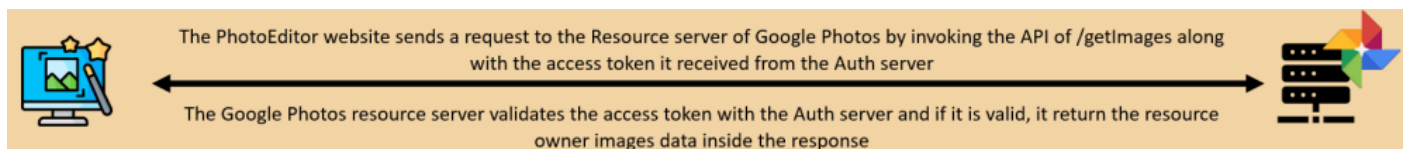


Note: From Step 4, based on the Grant Type, the Flow will vary.

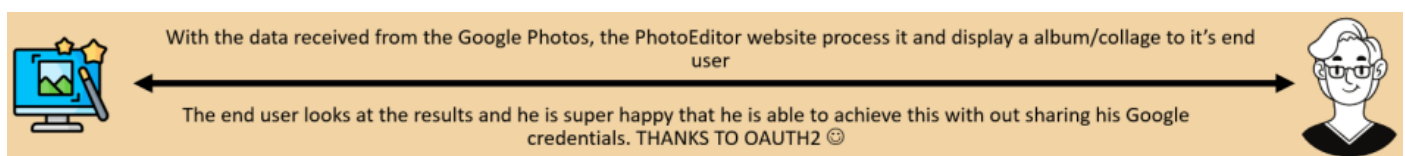
Step 4: Post Client Identification, the Authorization Server will issue tokens to the Client.



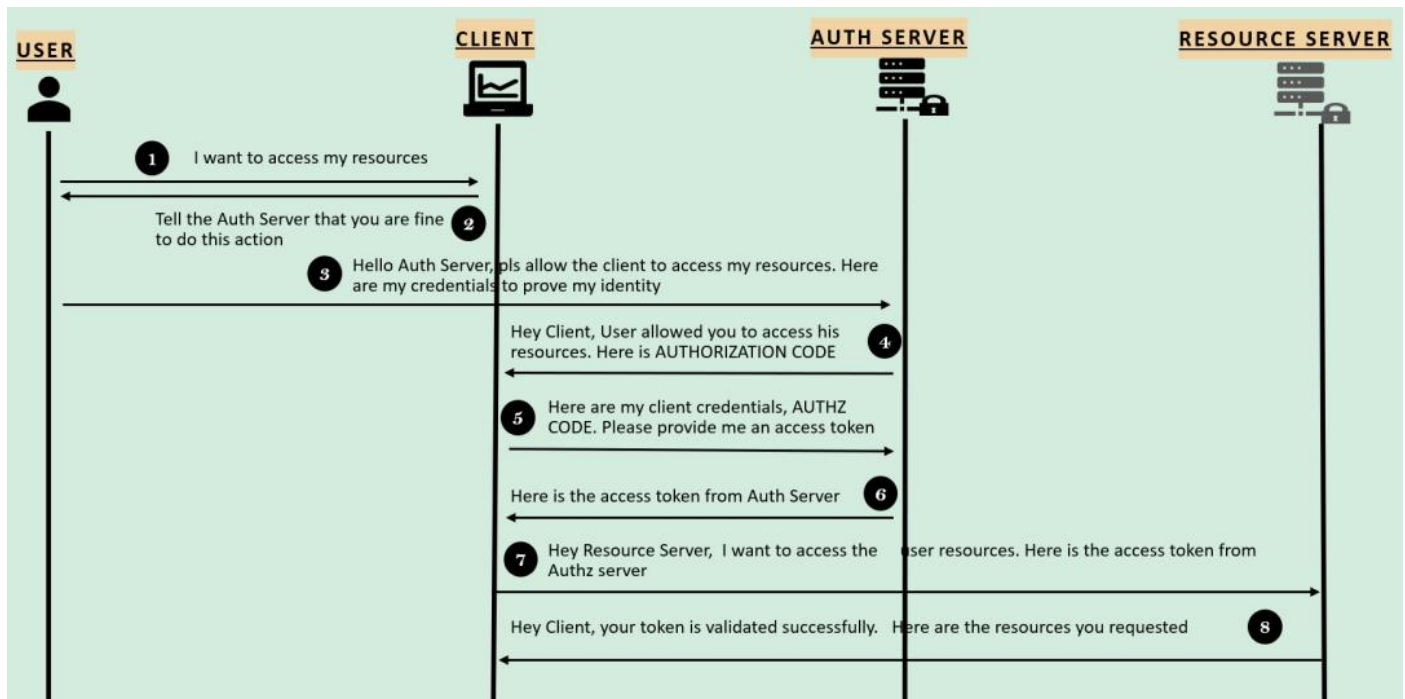
Step 5: Client application will send request (along with issued tokens) to Resource Server for the resource access



Step 6: Resource Owner/End user can explore the client's feature on the received resources & client can according perform operations.



OAuth2 Flow in Grant Type: Authorization Code



Request 1: Client to Auth Server – Will Receive **Authorization Code** from Auth Server

- In the step 2 & 3, where client is making a request to the Auth Server endpoint with the below request details:
 - 1. CLIENT ID / client_id**
 - Id which identifies the client application by the Auth Server. This will be granted to client during its registration to Auth Server.
 - 2. REDIRECT URI / redirect_uri**
 - The URI value which the Auth server needs to redirect post successful Authentication.
 - If default value is provided during the registration, then this value is optional.
 - 3. SCOPE / scope**
 - Similar to Authorities. Specifies the level of access that client is requesting like READ
 - 4. STATE / state**
 - CSRF token value to protect from CSRF attacks
 - 5. RESPONSE TYPE / response_type**
 - With the value 'code' which indicates that we want to follow Authorization Code Grant.

Request 2: Client to Auth Server – Will Receive **Access token** from Auth Server

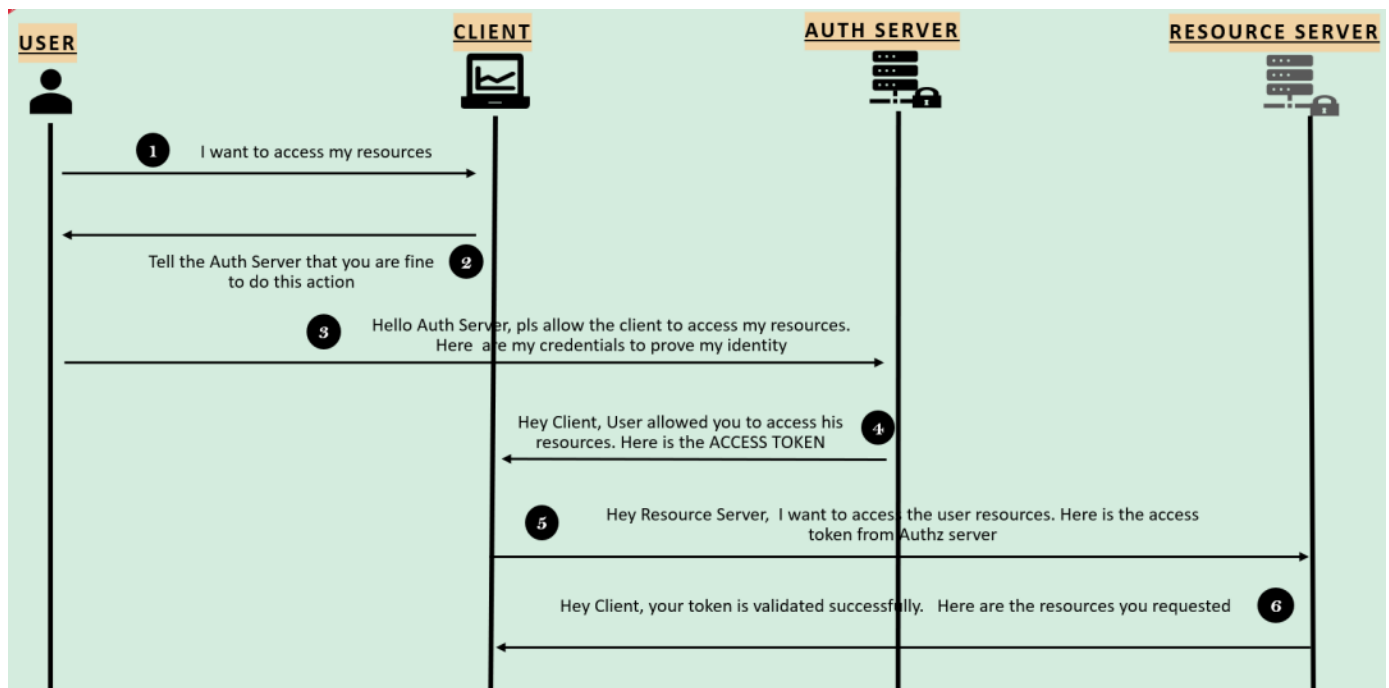
- In the step 5, where the client already received an authorization code from Auth Server, will again make a request for the Access token to Auth Server with below request values:
 - 1. CODE / code**
 - The Authorization code received in the above steps.
 - 2. CLIENT ID / client_id & CLIENT SECRET / client_secret**
 - The client credentials which are registered with the Auth Server. Please note that these are not user credentials.
 - 3. GRANT TYPE / grant_type**
 - With the value 'authorization_code' which identifies the kind of grant type is used.
 - 4. REDIRECT URI / redirect_uri**

Q. Why in the Authorization Code Grant Type, Client is making request 2 times to Authorization Server for Authorization Code & Access Token?

- In the 1st call, Authorization server will make sure that End user / Resource Owner directly interacted with it along with the credentials. If the details are correct, Auth server will send the Authorization code to Client.
- In the 2nd call, once Client receives the Authorization code, the Client has to prove its identity along with the Authorization code & Client Credentials to get the Access Token.

- Q. Why can't Auth Server directly club both the calls together & provide the token in a single step?
- There used to have another Grant Type called 'Implicit Grant Type' with the same feature but it's not recommended to use as its less secure.

OAuth2 Legacy Flow in Grant Type: Implicit Flow – Not Recommended



Request 1: Client to Auth Server – Will Receive **Access token** from Auth Server

- In the step 3, where Client is making a request to Auth Server endpoint with below request details:

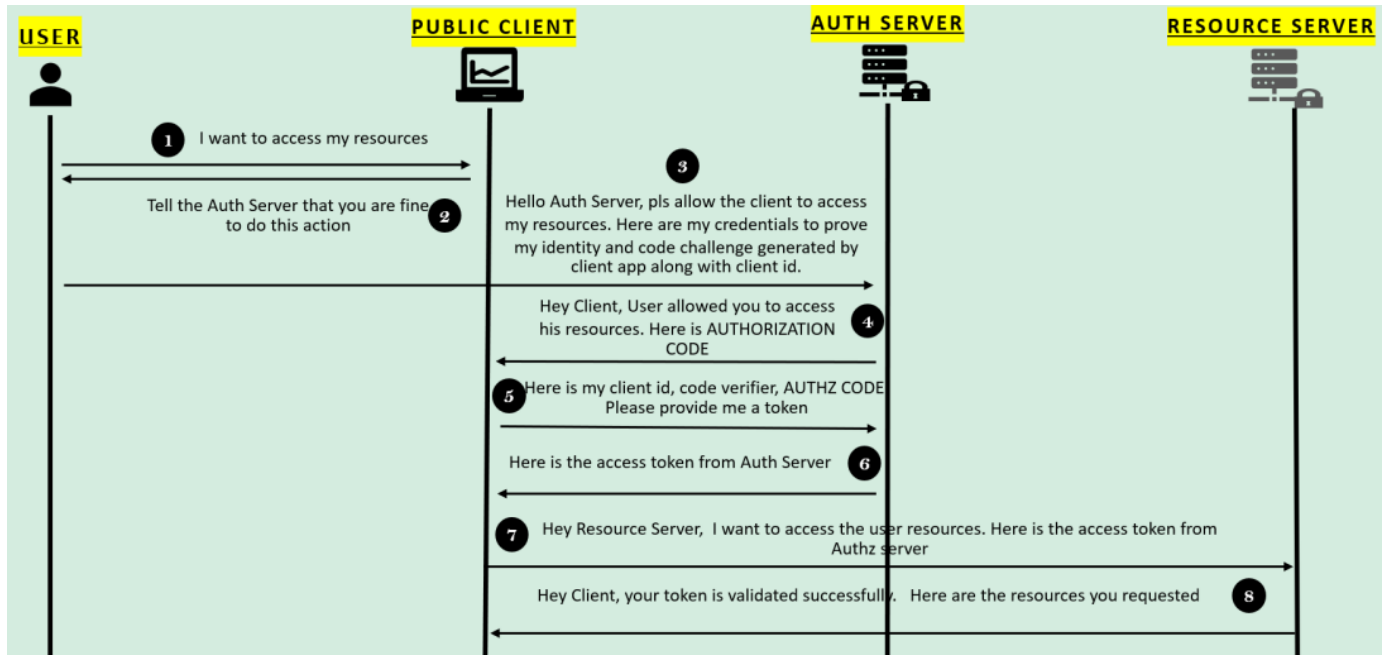
- **client_id** – the id which identifies the client application by the Auth Server. This will be granted when the client register first time with the Auth server.
- **redirect_uri** – the URI value which the Auth server needs to redirect post successful authentication. If a default value is provided during the registration then this value is optional
- **scope** – similar to authorities. Specifies level of access that client is requesting like READ
- **state** – CSRF token value to protect from CSRF attacks
- **response_type** – With the value 'token' which indicates that we want to follow implicit grant type

- If the user approves the request, the Auth server will redirect the browser back to the `redirect_uri` specified by the application, adding a token & state to the fragment part of the URL.

Note: Implicit Grant Flow is deprecated & is not recommended to use in production applications. Always use the Authorization code Grant Flow instead of Implicit Grant Flow.

OAuth2 Flow Grant Type: Authorization Code with Proof Key for Code Exchange (PKCE)

- When public clients (e.g., Native and Single – Page Applications) request Access Token, some additional security concerns are posed that are not mitigated by the Authorization Code Flow alone. This is because public clients can't securely store a Client Secret.
- Given these situations, OAuth 2.0 provides a version of the Authorization Code Flow for public client applications which makes use of a Proof Key for Code Exchange (PKCE)



- In the step 2 & 3**, where client is making a request to Auth Server endpoint with below request details:

- `client_id` – the id which identifies the client application by the Auth Server. This will be granted when the client registers first time with the Auth server.
- `redirect_uri` – the URI value which the Auth server needs to redirect post successful authentication. If a default value is provided during the registration then this value is optional
- `scope` – similar to authorities. Specifies level of access that client is requesting like READ
- `state` – CSRF token value to protect from CSRF attacks
- `response_type` – With the value 'code' which indicates that we want to follow authorization code grant
- `code_challenge` - XXXXXXXXX – The code challenge generated as previously described
- `code_challenge_method` - S256 (either plain or S256)

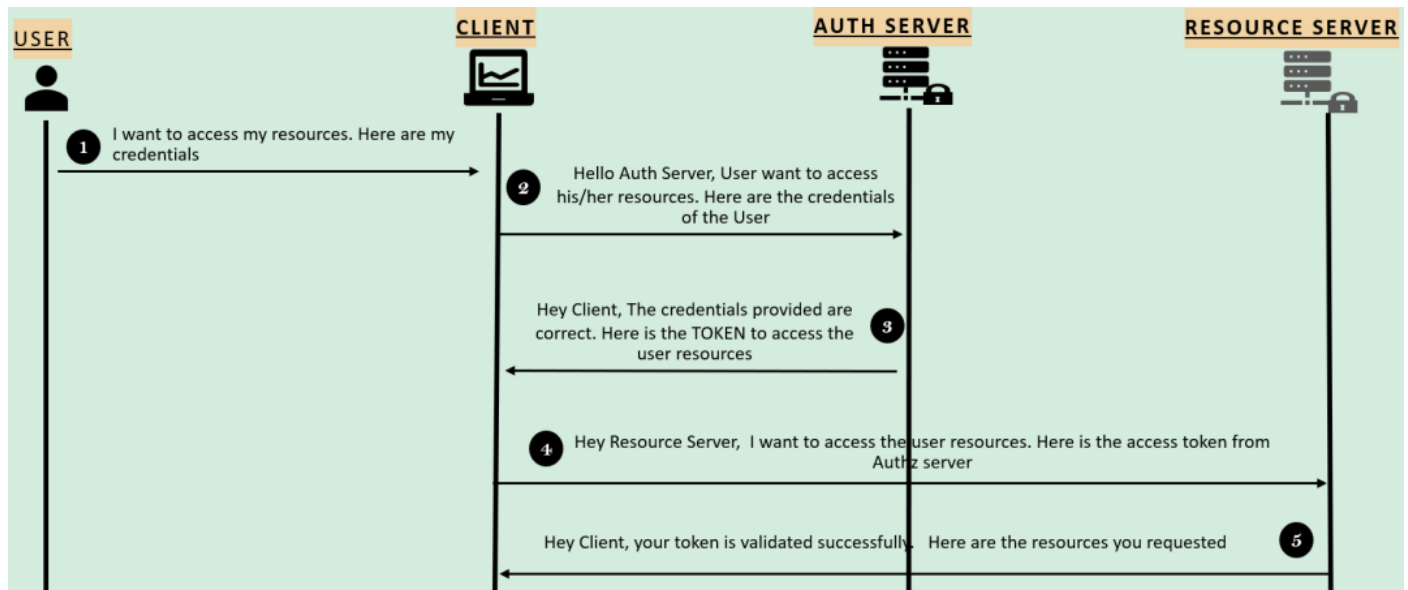
- In the step 5**, where client after received an authorization code from Auth Server, it will again make a request to Auth Server for a token with the below values:

- `code` – the authorization code received from the above steps
- `client_id` & `client_secret` (optional) – the client credentials which are registered with the auth server. Please note that these are not user credentials
- `grant_type` – With the value 'authorization_code' which identifies the kind of grant type is used
- `redirect_uri`
- `code_verifier` – The code verifier for the PKCE request, that the app originally generated before the authorization request.

- The **PKCE – Enhanced Authorization Code Flow** follows below steps:

- Once user clicks login, client app creates a cryptographically-random `code_verifier` and from this generates a `code_challenge`.
- `code challenge` is a Base64-URL-encoded string of the SHA256 hash of the code verifier.
- Redirects the user to the Authorization Server along with the `code_challenge`.
- Authorization Server stores the `code_challenge` and redirects the user back to the application with an authorization code, which is good for one use.
- Client App sends the authorization code and the `code_verifier` (created in step 1) to the Authorization Server.
- Authorization Server verifies the `code_challenge` and `code_verifier`. If they are valid it responds with ID Token and Access Token (and optionally, a Refresh Token).

OAuth2 Flow Legacy Grant Type: Password Grant/Resource Owner Credentials Grant Type

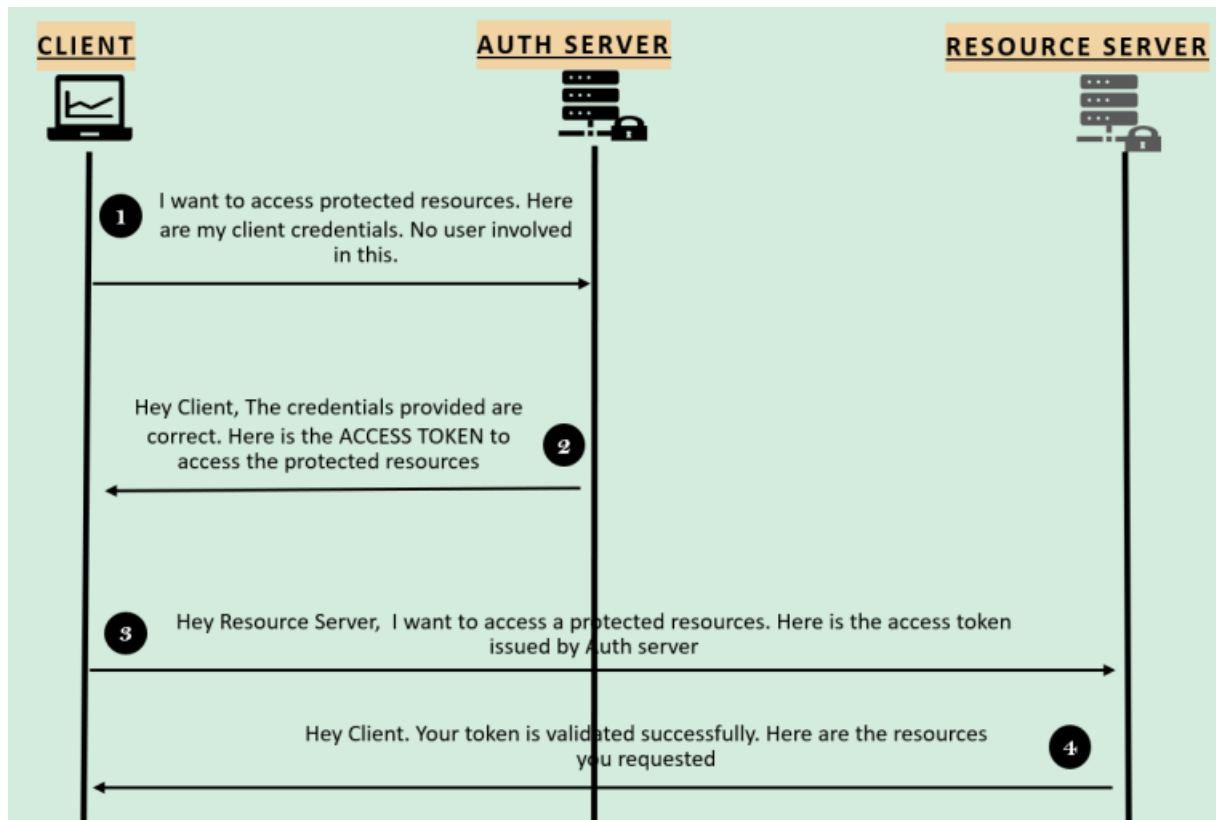


- In the step 2, where Client is making a request to Auth Server endpoint have below request details:

- **client_id** & **client_secret** – the credentials of the client to authenticate itself.
- **scope** – similar to authorities. Specifies level of access that client is requesting like READ
- **username** & **password** – Credentials provided by the user in the login flow
- **grant_type** – With the value 'password' which indicates that we want to follow password grant type

- We use this Authentication flow only if the Client, Authorization Server & Resource Servers are maintained by the same organization.
- This flow will be used followed by the Enterprise applications who want to separate the Auth Flow & Business Flow. Once the Auth Flow is separated different applications in the same organization can leverage it.

OAuth2 Flow Grant Type: Client Credentials

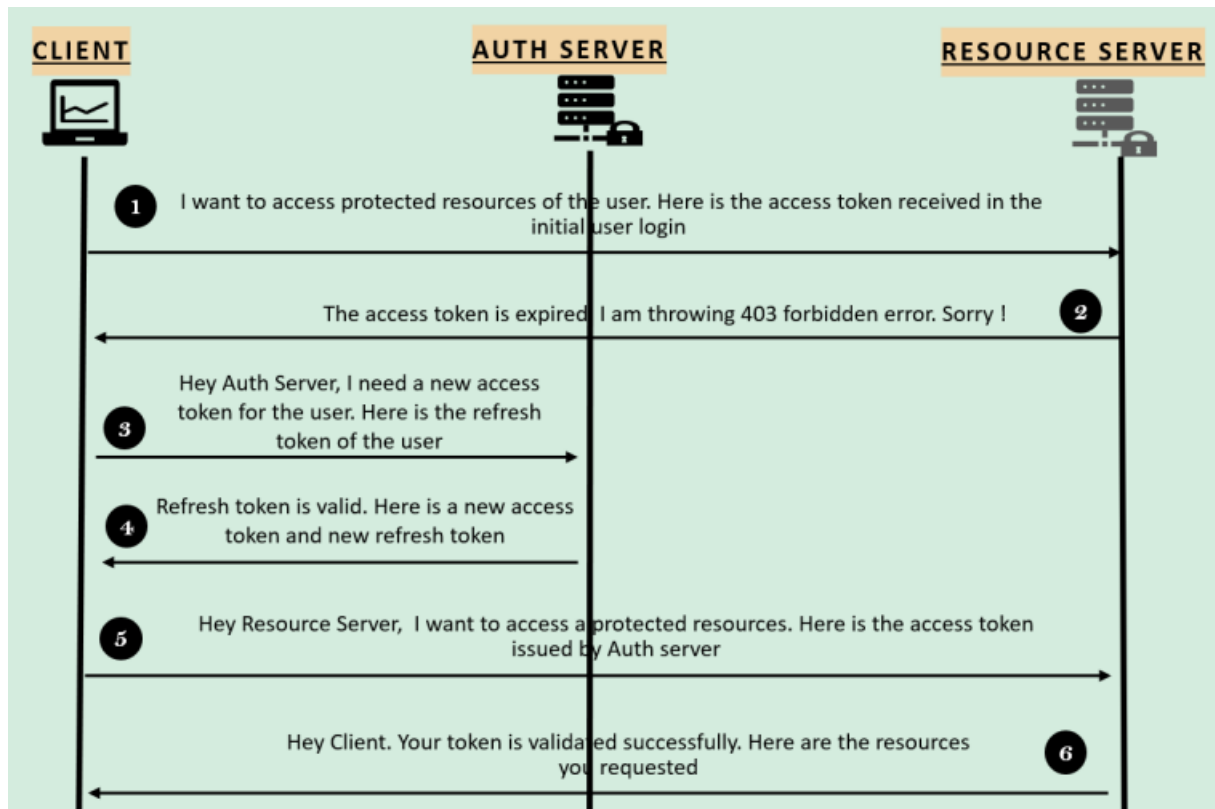


- In the step 1, where Client is making a request to Auth Server endpoint, have below request details:

- **client_id** & **client_secret** – the credentials of the client to authenticate itself.
- **scope** – similar to authorities. Specifies level of access that client is requesting like READ
- **grant_type** – With the value '**client_credentials**' which indicates that we want to follow client credentials grant type

- This is the simplest grant type flow in OAuth2.
- We use this Authentication flow only if there is no user & UI involved. Like in the scenarios where 2 different applications want to share data b/w them using backend APIs

OAuth2 Flow Grant Type: Refresh Token



- In the step 3, where Client is making a request to Auth Server endpoint with below request details:

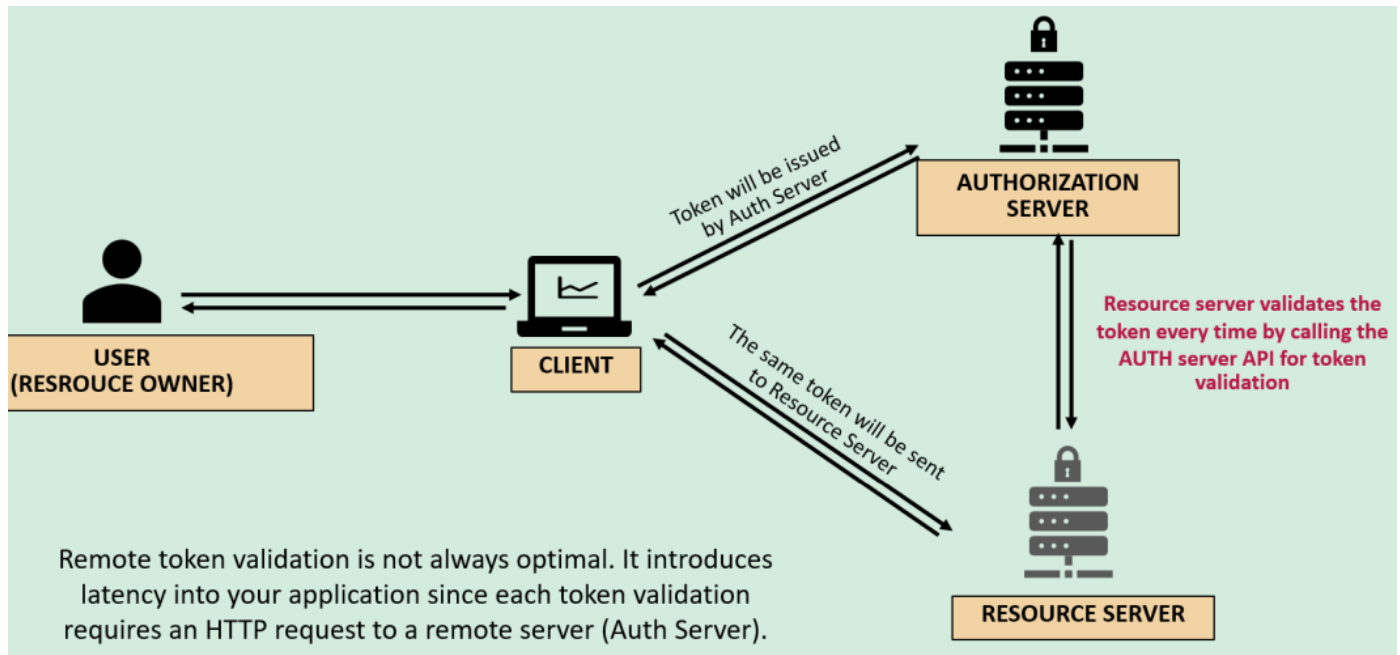
- client_id** & **client_secret** – the credentials of the client to authenticate itself.
- refresh_token** – the value of the refresh token received initially
- scope** – similar to authorities. Specifies level of access that client is requesting like READ
- grant_type** – With the value 'refresh_token' which indicates that we want to follow refresh token grant type

- This flow will be used in the scenarios where the Access Token of the user is expired. Instead of asking the user to login again & again, we can use the Refresh Token which originally provided by the AuthZ server to reauthenticate the user.
- Though we can make our Refresh tokens to never expire but it's not recommended considering scenarios where the tokens can be stolen if we always use the same token.
- offline_access** scope is used to get a Refresh Token that never expires.

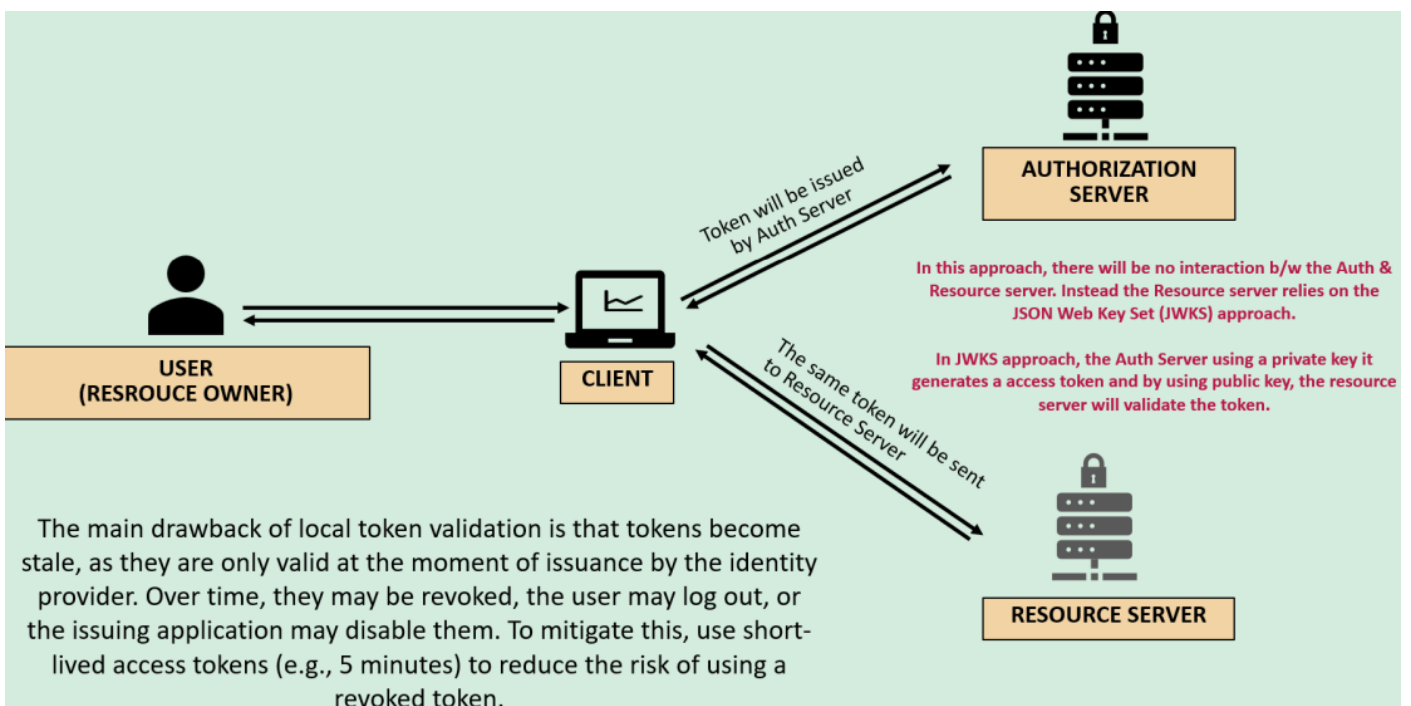
Note: We can use <https://oauth.net/2/>, <https://www.oauth.com/>, <https://www.oauth.com/playground/> to explore all the in-depth OAuth2 details & grant types.

How Resource Server Validates the Tokens issued by Authorization Server

Validating Access Tokens Remotely

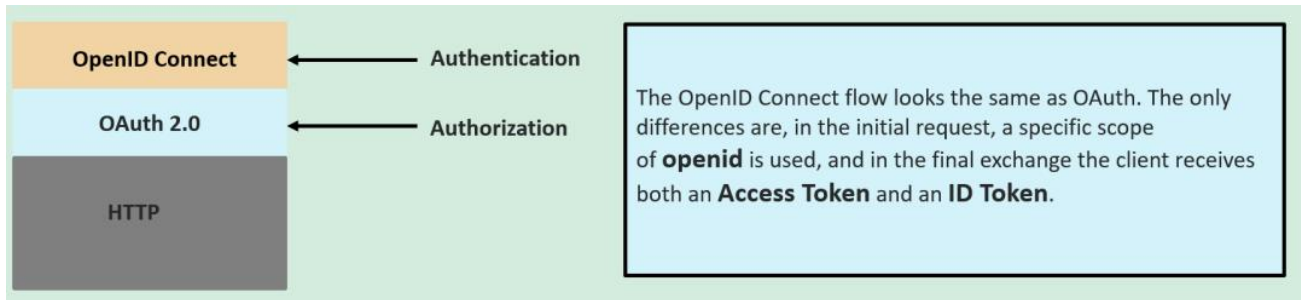


Validating Access Tokens Locally



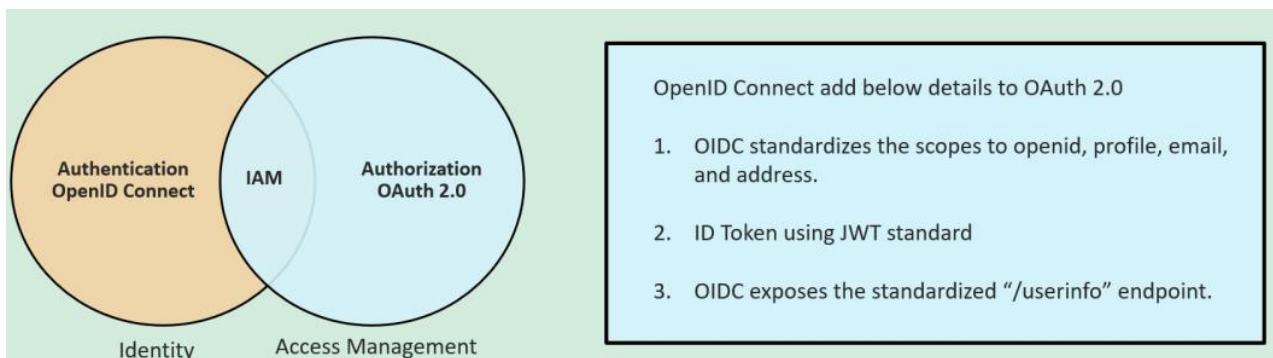
OpenID Connect

- OpenID Connect is a protocol that sits on the top of the OAuth 2.0 framework. While OAuth2 provides Authorization via an access token containing scopes, OpenID Connect provides Authentication by introducing a new ID token which contains a new set of information and claims specifically for Identity.
- With the ID token, OpenID Connect brings standards around sharing identity details among the applications.



Why is OpenID Connect important?

- Identity is the key to any application. At the core of modern authorization is OAuth 2.0 but OAuth 2.0 lacks an authentication component. Implementing OpenID Connect on the top of OAuth 2.0 completes an IAM (Identity & Access Management) strategy.
- As more & more applications need to connect with each other and more identities are being populated on the internet, the demand to be able to share these identities is also increased.
- With OpenID Connect, applications can share the identities easily and standard way.



OAuth2 Demo

2 different approaches:

1. Allow the secured APIs access to the end user via the social logins like GitHub, Facebook, Google etc.
2. Allow the secured APIs access via our own Authorization Server

Note: Whenever we enable the OAuth 2.0 login, we need to give clue to our Spring Security framework on which Authorization server we're trying to use, whether we're using social logins or our own Auth Server.

- To give this clue, we have to create a bean of type **ClientRegistrationRepository** & its implementations like **InMemoryClientRegistrationRepository** and **SupplierClientRegistrationRepository**.
- Using these repository class, we're going to store all our Authorization server related details in form of **ClientRegistration** object.

```
Choose Implementation of ClientReg

InMemoryClientRegistrationRepository (org.springframework.security.oauth2.client.registration)
SupplierClientRegistrationRepository (org.springframework.security.oauth2.client.registration)

public interface ClientRegistrationRepository {

    Returns the client registration identified by the provided registrationId, or null if not found.
    Params: registrationId - the registration identifier
    Returns: the ClientRegistration if found, otherwise null

    ClientRegistration findByRegistrationId(String registrationId);
}
```

```
public final class ClientRegistration implements Serializable {

    private static final long serialVersionUID = SpringSecurityCoreVersion.SERIAL_VERSION_UID;

    private String registrationId;

    private String clientId;

    private String clientSecret;

    private ClientAuthenticationMethod clientAuthenticationMethod;

    private AuthorizationGrantType authorizationGrantType;

    private String redirectUri;

    private Set<String> scopes = Collections.emptySet();

    private ProviderDetails providerDetails = new ProviderDetails();

    private String clientName;

    private ClientSettings clientSettings;
```


1. Allow the secured APIs access to the end user via the social logins like GitHub, Facebook, Google etc.

- Whenever we want to implement social logins inside our Spring Boot web application, we can leverage one of the enum class available i.e., **CommonOAuth2Provider**

```
public enum CommonOAuth2Provider {

    GOOGLE {

        @Override
        public Builder getBuilder(String registrationId) {
            ClientRegistration.Builder builder = getBuilder(registrationId,
                ClientAuthenticationMethod.CLIENT_SECRET_BASIC, DEFAULT_REDIRECT_URL);
            builder.scope("openid", "profile", "email");
            builder.authorizationUri("https://accounts.google.com/o/oauth2/v2/auth");
            builder.tokenUri("https://www.googleapis.com/oauth2/v4/token");
            builder.jwkSetUri("https://www.googleapis.com/oauth2/v3/certs");
            builder.issuerUri("https://accounts.google.com");
            builder.userInfoUri("https://www.googleapis.com/oauth2/v3/userinfo");
            builder.userNameAttributeName(IdTokenClaimNames.SUB);
            builder.clientName("Google");
            return builder;
        }
    },

    GITHUB {...},

    FACEBOOK {...},

    OKTA {...};
}
```

We can configure social login in 2 ways in our spring boot application

- Using defining beans of ClientRegistrationRepository with oauth2Login config
- Using predefined configuration of ClientRegistrationRepository with oauth2Login config

```
spring.security.oauth2.client.registration.github.client-id=${GITHUB_CLIENT_ID:0v23LiGyLB1m1TcK3z4W}
spring.security.oauth2.client.registration.github.client-secret=${GITHUB_CLIENT_SECRET:d0dd3013ae0e4bdbc09fc48d5f1e518ede400ce2}
```

2. Allow the secured APIs access via Authorization Server

- Whatever Auth server that are available with the help of social logins like GitHub, Facebook, LinkedIn etc., all these Auth servers have a very serious drawback/limitation due to which these Auth servers can't be used inside an Enterprise organization.
- The limitations are:
 - We will not have any control on the social login Auth servers. We can't create a role or authority based on our business requirements inside these Auth servers.
 - Whatever user related information that we're getting from the social logins, we just have to accept them & build our business logic using them.
- But in real applications, we can't live with these kinds of restrictions, we may want to register thousands of end users with respective authorities & roles defined inside the Auth server.
- That's why the Enterprise applications prefer to build their own Auth server where they have complete control.

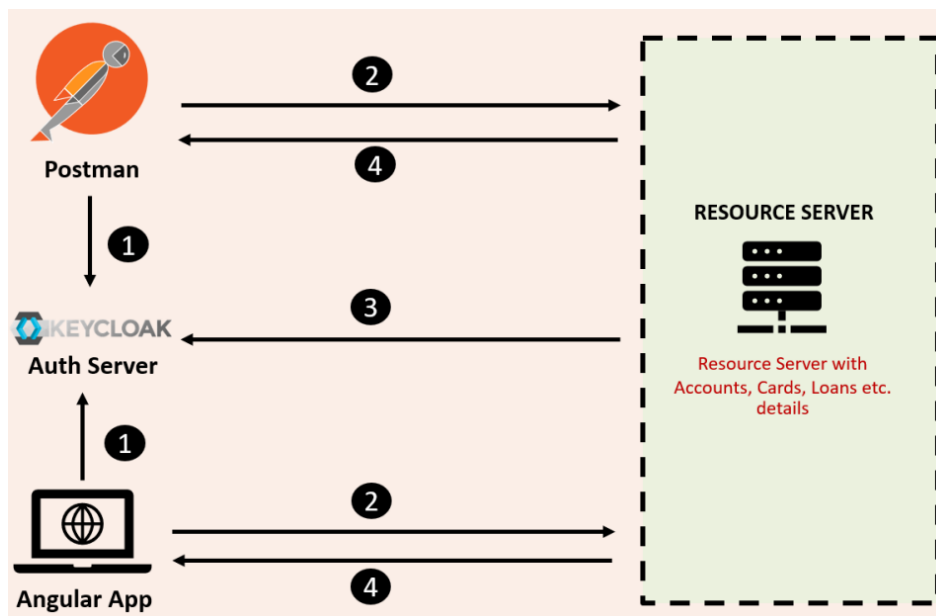
a) Resource Server – Our Spring boot application with all the protected APIs & resources

b) Client – Postman, Angular Application

c) Auth Server can be:

- 1) Commercial/Open-Source Product supporting OAuth 2 & Open ID standards like KeyCloak, Okta. All cloud providers like GCP, AWS, Azure have products around the OAuth 2 & Open ID Connect.
- 2) Build Custom Authorization Server from Scratch with help of Spring Auth Server library

Implementing OAuth2 using KeyCloak Auth Server Flow



Step 1: Client applications will reach out to the KeyCloak Server to get an access token. When they're trying to get the access token from the Auth server, they can involve End user if they're following the Authorization Code or PKCE flow otherwise if they're following Client Credential grant flow, they don't have to involve any end user.

Step 2: Once the Access token is received from the Auth Server, these client apps will send the token to the Resource server as part of Request.

Step 3: Once the Access token is received by the Resource Server, depending on the token type (JWT or Opaque), the Resource server will validate the token with Auth server (KeyCloak)

Step 4: If Access token is valid, the Resource server will send the secure resources as response to the client applications.

Intro to KeyCloak Auth Server

- As discussed earlier, OAuth2 & Open ID are just standard or specifications. They are not actual product which can be leveraged by any organization. That's why they are simply guidelines/standards/specifications.
- Based on these OAuth2 standards, bigger organizations like Facebook, GitHub, LinkedIn etc. they started building their own Auth server for their own internal purpose but this may not be possible for all the organizations to build their Auth server, that's where Organizations can use already built Auth Server Products based on this specification like KeyCloak (Open Source), Okta (Commercial), AuthO, Amazon Cognito.
- KeyCloak is an open-source product built & maintained by Cloud Native Computing Foundation. We can setup this Auth Server inside our local/dev/production system regardless of the complexity of the application.
- **Refer** <https://www.keycloak.org/> for details on KeyCloak.
- **User Federation** means we can store all the user credentials inside storage systems like Relational DB, LDAP, Active Directory & we can point our Auth Server to these storage systems for Authentication & Authorization.
- If needed, we can also customize the login page of the Auth Server based upon our own organization look & feel requirement.

Setting KeyCloak in our local system

We can refer <https://www.keycloak.org/guides> for setting up Keycloak. We as Developer are not responsible to set up the Keycloak server, either Devops team or Platform team or Deployment team will setup in various environment & **will provide the URLs to the developers.**

Step 1: Docker image

- `docker run -p 8180:8080 -d -e KEYCLOAK_ADMIN=admin -e KEYCLOAK_ADMIN_PASSWORD=admin quay.io/keycloak/keycloak start-dev`
- **start-dev:** We're telling to the key cloak to start using a development profile/mode i.e., embedded H2 database or in-memory database will be used to store all the end user details & all other configurations.

Step 2: KeyCloak Admin Console Access link: <http://localhost:8180/> with Credential as admin, admin

Step 3: Set up own Realm

- A Realm is a space that is used to manage a set of users, credentials, roles & groups in a specific environment.
- There is a very good chance we're not going to maintain the same users, credentials etc across multiple environments, to support these kinds of scenarios, Every Auth server built on the top of OAuth2 will have a concept called Realm.

Step 4: Registering Client details inside the KeyCloak Auth Server

- The Client details will vary based on OAuth2 grant types set up.

a) Client Credential Grant type flow – No End user involved (Backend app to Resource server communication)

- Refer above slides for this grant type flow.
- During the client registration to Auth Server, Auth server will give Client Id & Client Secret which can be leveraged by the 3rd Party app/client to get an access token later.
- Same Access token will be sent to Resource server & post validation; the protected resources will be sent back to client.

Create your own client in the newly created realm by providing client Id

Client type ?

OpenID Connect

Client ID * ?

eazybankapi

Name ?

eazybankapi

Description ?

Eazy Bank 3rd Party Client App

Always display in UI ?

Off

Client authentication ?

On

Authorization ?

Off

Authentication flow

☐ Standard flow ?

☐ Direct access grants ?

☐ Implicit flow ?

☒ Service accounts roles ?

☐ Standard Token Exchange ?

☐ OAuth 2.0 Device Authorization Grant ?

☐ OIDC CIBA Grant ?

PKCE Method ?

Choose...

eazybankapi

OpenID Connect

Clients are applications and services that can request authentication of a user.

Settings

Keys

Credentials

Roles

Client scopes

Service accounts roles

Sessions

Advanced

General settings

Client ID * ?

eazybankapi

Name ?

eazybankapi

Description ?

Eazy Bank 3rd Party Client App

Always display in UI ?

Off

Jump to

General

Access

Capabilities

Login

Client Id is eazybankapi

eazybankapi

OpenID Connect

Enabled

Clients are applications and services that can request authentication of a user.

Settings

Keys

Credentials

Roles

Client scopes

Service accounts roles

Sessions

Advanced

Events

Client Authenticator ?

Client Id and Secret

Save

Client Secret

Wn9xrBLn78Eq8mPqdG1MV7KhYY6LUNBK

Regenerate

We can create roles using Realm roles & then assign the role using service-account-roles in client

The screenshot shows the 'Manage' sidebar on the left with 'Realm roles' highlighted. The main panel displays a search bar 'Search role by name' with a green 'Create role' button. Below, a list of roles is shown with 'ADMIN' and 'USER' highlighted in green.

Role name
ADMIN
USER
default-roles-eazybankdev ?
offline_access
uma_authorization

The screenshot shows the 'Service accounts roles' tab. It includes a search bar 'Search by name', a checked 'Hide inherited roles' checkbox, and a green 'Assign role' button. Below is a table listing roles with checkboxes for selection.

☐ Name	Inherited
☐ default-roles-eazybankdev	False
☒ ADMIN	False
☐ USER	False

We can access the below endpoint to know all the URLs

The screenshot shows the 'Configure' sidebar on the left with 'Realm settings' highlighted. The main panel displays configuration options for 'Unmanaged Attributes', 'Signature algorithm', and 'Endpoints'.

Unmanaged Attributes ?	Disabled
Signature algorithm SAML IdP metadata ?	Choose...
Endpoints ?	OpenID Endpoint Configuration SAML 2.0 Identity Provider Metadata

For e.g., <https://accounts.google.com/.well-known/openid-configuration>

Public Keys to validate the Access Token locally using jwks_uri (Fetched from Auth Server)

```
# Resource server - Auth Server Communication Link to validate the access token
spring.security.oauth2.resourceserver.jwt.jwk-set-uri=${JWK_SET_URI:http://localhost:8180/realms/eazybankdev/protocol/openid-connect/certs}
```

For opaque token, each request to the Resource Server will be validated with Auth Server.

Creating Client & User details inside KeyCloak for Auth Code Grant Flow

Client type ?

OpenID Connect

Client ID * ?

eazybankclient

Name ?

eazybankclient

Description ?

Client to see the demo of Auth Code Grant Type Flow

Always display in UI ?



Off

Client authentication ?



On

?

Authorization ?



Off

Authentication flow



Standard flow ?



Direct access grants ?



Implicit flow ?



Service accounts roles ?



Standard Token Exchange ?



OAuth 2.0 Device Authorization Grant ?



OIDC CIBA Grant ?

PKCE Method ?

Choose...

Root URL ?

Home URL ?

Valid redirect URIs ?

*

[+ Add valid redirect URIs](#)

Valid post logout
redirect URIs ?

[+ Add valid post logout redirect URIs](#)

Web origins ?

[+ Add web origins](#)

Clients are applications and services that can request authentication of a user.

Settings	Keys	Credentials	Roles	Client scopes	Sessions	Advanced	Events
----------	------	-------------	-------	---------------	----------	----------	--------

Client Authenticator
?

Client Id and Secret

Save

Client Secret

3EmjCYHclTfimucXtvHEJhaScYi3T9et

Refer <https://www.keycloak.org/documentation> <https://www.keycloak.org/docs-api/latest/rest-api/index.html> for more details on how to configure users through APIs.

Manage	General	
Clients		
Client scopes		
Realm roles		
Users	Username *	happy@example.com
Groups	Email	happy@example.com
Sessions	First name	Happy
Events	Last name	Example